Gottfried Wilhelm Leibniz Universität Hannover
Institut für Informationsverarbeitung

Diplomarbeit

Genome Compression

Jan Voges
Matrikelnummer: 2593770

Betreuer:	Dr.-Ing. M. Munderloh
Erstprüfer:	Prof. Dr.-Ing. J. Ostermann
Zweitprüfer:	Prof. Dr.-Ing. B. Rosenhahn

Hannover, September 2015

Die Richtlinien im *Merkblatt zur Ausführung von Abschlussarbeiten am Institut für Informationsverarbeitung* sind Bestandteil der Aufgabenstellung. Eine anderweitige Verwendung der Arbeit, insbesondere die Bekanntgabe an Dritte, bedarf der Zustimmung des Erstprüfers.

Ich erkläre hiermit, dass ich die vorliegende Arbeit selbständig angefertigt und keine anderen als die angegebenen Quellen und Hilfsmittel verwendet habe. Ich stimme der Verwertung meiner Arbeit durch das Institut in den folgenden Punkten zu:

- Aufnahme der gedruckten und der elektronischen Fassung der Arbeit in die Institutsbibliothek,
- Vervielfältigung der gesamten Arbeit oder von Auszügen für Lehrzwecke und
- Wiedergabe der Arbeit durch Bild- und Tonträger

Hannover, den 23.09.2015

Institut für
Informationsverarbeitung
Leibniz Universität Hannover
Prof. Dr.-Ing. J. Ostermann

Appelstr. 9A - 30167 Hannover
Sekretariat: +49 511 762 - 5316
Telefax: +49 511 762 - 5333
E-mail: office@tnt.uni-hannover.de
URL: http://www.tnt.uni-hannover.de

23.03.2015

Diplomarbeit

für

Jan Voges

2593770

Genome Compression

Die jüngsten Fortschritte in der Entwicklung von Genom-Sequenziermaschienen erlauben eine kostengünstige Sequenzierung großer Mengen Genmaterial in immer kürzerer Zeit. Gleichzeitig sorgt die zunehmende Verbreitung dieser Sequenziermaschinen für immer neue Anwendungsfelder. Allerdings ist der Sequenzierungsprozess nicht fehlerfrei. Aus diesem Grund wird das sequenzierte Genom mehrfach (ca. 150 Mal) ausgelesen. Dies führt zu sehr großen Datenmengen im Terabyte Bereich, die mit aktueller PC-Hardware schwer zu handhaben sind. Weiterhin ist absehbar, dass zukünftig die Menge an sequenziertem Genmaterial stärker ansteigen wird als die jeweils zur Verfügung stehende Speicherkapazität.

Am Institut für Informationsverarbeitung wird deshalb an einem Verfahren zur Kompression von sequenzierten Geninformationen gearbeitet. Die Geninformationen bestehen dabei aus verschiedenen Teilen, die jeweils eine eigene Statistik aufweisen und daher getrennt betrachtet und komprimiert werden müssen.

Herr Voges erhält daher die Aufgabe, Verfahren aus der Informationstheorie zu analysieren und auf Eignung zur Codierung und Kompression von sequenzierten Genmaterial zu prüfen, zu kombinieren, anzupassen oder zu erweitern. Die vielversprechendsten Verfahren sind zu implementieren und mit dem Stand der Technik zu vergleichen. Bei dem entwickelten Verfahren soll dabei ein wahlfreier Zugriff auf Teile der komprimierten Informationen möglich sein (Random Access), um ein effizientes Arbeiten mit den komprimierten Daten zu ermöglichen.

Prof. Dr.-Ing. J. Ostermann

Preface

Conventions Throughout this document, the following conventions are used:

- Units of information (most commonly bits and bytes) will have binary prefixes, multiplying the unit by powers of two. For example 1 GiB (one gigibyte) are exactly $1 \cdot 2^{30} = 1 \cdot 1024^3 = 1\,073\,741\,824\ B$ (bytes) in contrast to 1 GB (one gigabyte), which represents $1 \cdot 10^9 = 1 \cdot 1000^3 = 1\,000\,000\,000\ B$.
- Words in the running text which need to be emphasised because of their special or outstanding meaning, are written in *italic* (*cursive*) letters.
- Code listings, URLs, file extensions within the running text, and congenial expressions are written in a `typewriter` font.
- Generally, mathematical symbols are explained within the running text. The following conventions are used throughout this report:
  - Sets are written in the "blackboard bold" font. For example, the symbol $\mathbb{R}$ denotes the set of real numbers.
  - Alphabets (in the context of information theory), or discrete finite sets, are denoted with symbols in a calligraphic font, for example $\mathcal{A}$.
  - Vectors are denoted by underlined *lowercase* Latin letters, for example $\underline{v}$. Only in obvious cases, the underlining is omitted.
  - Matrices are denoted by underlined *uppercase* Latin letters such as $\underline{M}$.
- Compression results are presented according to one of the following definitions.
  - Compression Ratio (CR): $CR = \frac{Compressed\,Size}{Uncompressed\,Size}$
  - Compression Factor (CF): $CF = \frac{Uncompressed\,Size}{Compressed\,Size}$
  - Space Savings (SpS): $SpS = 1 - CR$

MPEG Contributions Within the scope of this thesis, I attended the 112th MPEG (Moving Pictures Experts Group) meeting in Warsaw, dated 22 to 26 June 2015. In collaboration with Dr.-Ing. Marco Munderloh (the supervisor of this thesis), I prepared an input document with the title "Approaches to SAM File Compression" [VM15] and presented the document (containing excerpts from Sections 4.2 (Block-based Compression of Nucleotide Sequences) and 4.3 (Predictive Coding of Quality Scores) in a plenary meeting. Moreover, I presented the software framework

from Section 5.1. By now, ongoing correspondence with some MPEG members has been established, with a main focus on discussing the implementation and application of novel compression schemes, and the results obtained thereby.

MPEG assigns unique identifiers to output documents from their meetings. Those identifiers are mentioned within the running text and within the bibliography. Generally, MPEG output documents are available through [Chi15].

Provisional Patent Application Most of the compression approaches mentioned in Chapter 4 (Novel Approaches to Genome Compression) have been summarised in a provisional patent application (in the United States of America) with the title "Predictive Coding of Genomic Information Data" [VMO15]. In collaboration with the EZN and an appointed patent attorney, there is a continuing evaluation of the proposal.

Supplementary Material The DVD accompanying this report contains the LaTeX code for this document, including most of the references as PDF/DOCX files, and most of the figures. Furthermore, the DVD contains the software that has been developed in the scope of this thesis (the software is explained in Section 5.1 and Appendices A and C). The top-level directory structure of the DVD is shown in Listing 1.

```
latex/        // LaTeX code, including the references
samsplit/     // Samsplit software collection
tsc/          // Tsc software collection
```

Listing 1: *Supplementary material provided on the DVD accompanying this report.*

Danksagung An dieser Stelle möchte ich mich bei allen Personen bedanken, die mich bei der Erstellung dieser Arbeit unterstützt haben. Mein besonderer Dank geht an Herrn Prof. Dr.-Ing. Jörn Ostermann, der mir die Erstellung der vorliegenden Diplomarbeit am Institut für Informationsverarbeitung ermöglichte. Außerdem danke ich Herrn Dr.-Ing. Marco Munderloh, der diese Arbeit fachlich betreut hat. Seine hilfreichen Anregungen haben sehr zum Gelingen dieser Arbeit beigetragen.

Acronyms

AC Arithmetic Coding. 17–19, 47, 62, 73, 74
AhG Ad-hoc Group. In the context of this thesis, within MPEG, AhGs are established with the purpose to evaluate predefined topics. The work done by AhGs might lead to a full standardization process. V, 2, 29, 43, 44, 46, 70, 71, 78
ASCII American Standard Code for Information Interchange. II, 28, 31, 33, 34, 52

BGZF Blocked GNU Zip Format. 37, 41
bp base pair(s). XXIX, XXXI, 24
BWA Burrows Wheeler Aligner. 29
BWT Burrows Wheeler Transform. 53

CABAC Context Adaptive Binary Arithmetic Coding. 18
cdf Cumulative distribution function. 9
CF Compression Factor. iii
CfE Call for Evidence. 43
CfP Call for Proposals. 43
CR Compression Ratio. iii, 74

DCT Discrete Cosine Transform. 21, 22, 56, 57
DDBJ DNA Data Bank of Japan. 72
DFT Discrete Fourier Transform. 22
DNA Deoxyribonucleic acid. Molecule containing the genetic information of an organism. XXIX–XXXII, 1, 7, 22–25, 27, 29, 30, 34, 41, 42, 46, 77
DVD Digital Versatile Disc or Digital Video Disc. iv, I, VII

EBI European Bioinformatics Institute (`http://www.ebi.ac.uk/`). 31, 73
EZN ErfinderZentrum Norddeutschland (Inventors' Centre North Germany). iv

GEO Gene Expression Omnibus. 73
GIDB Genomic Information Data Base. See Appendix D. 5, 43, 52, 55–58, 60, 71, 72, 79

HDD Hard Disk Drive. 40
HEVC High Efficiency Video Coding. 18

IEC International Electrotechnical Commission (`http://www.iec.ch/`). XXX
INSDC International Nucleotide Sequence Database Collaboration. 72, 73
ISO International Organization for Standardization (`http://www.iso.org/`). XXX, 43
IUPAC International Union of Pure and Applied Chemistry. 28, 29, 31, 77

KLT Karhunen-Loeve-Transform. 22

LZMA Lempel-Ziv-Markov Algorithm. 54, 55
LZW Lempel-Ziv-Welch (algorithm). 19

MIPS Million Instructions Per Second. 40
MPEG Moving Pictures Experts Group. iii–v, V, XXX, 2, 5, 18, 29, 32, 34, 36, 40, 43–46, 52, 70–73, 78
mRNA messenger RNA. XXXII, XXXIII, 23

NCBI US National Center for Biotechnology Information. 73
NGS Next-Generation Sequencing. XXXI, XXXII, 1, 24, 25, 27, 28, 31, 39–41, 43, 48, 51, 77

PCC Pearson product-moment Correlation Coefficient. XXIX, 10
pdf Probability density function. 10, 11
pmf Probability mass function. 9–11, 16

RA Random Access. 49
RNA RiboNucleic Acid. vi, XXX–XXXII, 1, 23, 27, 31, 34, 41

SNV Single Nucleotide Variant. XXXII, 43
SOLiD Sequencing by Oligonucleotide Ligation and Detection. 25, 26
SpS Space Savings. iii

Contents

1. Introduction **1**
1.1. Motivation . . . 2
1.2. Report Structure . . . 4

2. Theoretical Foundations **7**
2.1. Statistics . . . 7
2.2. Source Coding . . . 13
2.3. Genomics and Sequencing Technologies . . . 22
2.4. File Formats . . . 31

3. Prior Work in Genome Compression and Storage **39**
3.1. Problem Definition . . . 39
3.2. Standardization Efforts . . . 43
3.3. Requirements for Genomic Information Compression . . . 44
3.4. Overview about Genome Compression . . . 46

4. Novel Approaches to Genome Compression **51**
4.1. Statistical Analysis . . . 51
4.2. Block-based Compression of Nucleotide Sequences . . . 59
4.3. Predictive Coding of Quality Scores . . . 64
4.4. Quantized Quality Scores . . . 66

5. Results **69**
5.1. Software Framework . . . 69
5.2. MPEG Genomic Information Database . . . 70
5.3. Results: Block-based Compression Of Nucleotide Sequences . . . 73

6. Summary and Outlook **77**

A. Samsplit **I**
A.1. Samsplit . . . I
A.2. MATLAB Scripts and Tntlib . . . II

B. SAM/BAM File Processing **V**
B.1. Getting SAMtools . . . V
B.2. Accessing SAM/BAM Content . . . V
B.3. BAM Indexing . . . VI
B.4. SAM/BAM Conversion . . . VI

C. The Tsc Framework **VII**

D. Genomic Information Database **IX**

E. Tsc Benchmark Results **XIII**

Bibliography **XIX**

Glossary **XXIX**

Index **XXXV**

List of Figures **XXXIX**

List of Tables **XLI**

List of Listings **XLV**

1. Introduction

Deoxyribonucleic acid (DNA) is a molecule existent in all known life forms[1]. It contains the genetic information of an organism in the form of nucleotide sequences. The nucleotide sequences contain information about the synthesis of proteins which in turn control the biological development of an organism, ultimately yielding the wide variety of life forms. Watson and Crick first described the double helix structure of the DNA molecule in 1953 in their paper "Molecular structure of nucleic acids: A structure for deoxyribose nucleic acid" [WC53]. Since then, determining the exact nucleotide sequence of an organism (its "genome") in order to examine its function and structure has been the major goal of a scientific field known as "genomics" today. The process of expressing the nucleotide sequence in a DNA molecule as a sequence of characters is known as "sequencing".

In 1977, Frederick Sanger published a DNA sequencing method, which was adopted amongst others in the "first generation" of sequencing applications [SNC77]. Sanger sequencing became one of the main tools for the human genome project which was started in 1990 and reached its climax with the sequencing of the complete human genome in 2001. Amongst others, this project stimulated the development of the so-called "next-generation sequencing" (NGS) tools, based on massively parallel analysis and yielding a high throughput. Due to these NGS technologies, the sequencing of huge amounts of genetic information has become affordable [WBL13, Kah11]. On account of this flood of data, IT costs may become a major obstacle compared to sequencing costs. High-performance compression of genomic data is required to reduce the storage size and transmission costs. Sophisticated compression systems closing the gap between sequencing and storage costs would help the scientific community in order to gain increased knowledge and to bring genome sequencing into every-day diagnosis and therapy.

NGS sequencing machines do not read out an entire genome; instead, they produce short random fragments of nucleotide sequences known as "sequence reads". Usually, a so-called quality score, expressing the confidence that a particular nucleotide has actually been found, is associated with each nucleotide in a sequence read. This raw sequencing data generated by NGS machines is usually stored in FASTQ files. Reads which have been assembled[2] in order to reconstruct the donor genome are called "mapped reads". This mapped sequencing data is usually stored in SAM files which compared to FASTQ files contain additional mapping information as well as

[1] An exception are RNA viruses.

[2] Generally, there are two flavours of the assembly process. If the mapping is done without an external reference, the process is called de-novo assembly, otherwise, it is called reference-based assembly.

possible additional meta information. Mapped sequencing data represented in SAM files contains more redundancy, as typically multiple sequence reads are mapping to the same location on the donor genome. The average amount of reads mapping to the same location is referred to as *coverage*.

It has been shown by Tembe et al. [TLS10] and Deorowicz and Grabowski [DG11], that splitting the data into separate streams (mainly for sequence reads and quality scores) and compressing them independently, yields significant gains over general-purpose (e.g. dictionary-based [ZL77]) programs (e.g. gzip [GA15]). All recent contributions to the topic of FASTQ and SAM file compression employ this scheme of separately compressing the sequence reads and quality scores (as well as the other fields present in FASTQ or SAM files).

Recently, various institutions initiated investigations regarding genomic information compression. For example, the Moving Pictures Experts Group (MPEG) appointed an ad-hoc group (AhG) to investigate genetic information processing and in particular compression. In October 2014, MPEG issued a "White paper on genomic information compression and storage"[3] [AMC+14], with the goal of starting a standardisation effort in genomic information processing.

Based on statistical analysis performed on a reference data set (issued by the appointed MPEG AhG on genome compression), new proposals for sequence and quality score compression have been developed in the scope of this thesis. A major requirement for this work was to ensure *random access* to the compressed data (for reasons discussed in detail in the following sections).

Before starting to concentrate on the details regarding genomic information processing and compression with Chapter 2.3 (Genomics and Sequencing Technologies), the following sections illustrate the thread running through this thesis and give an overview about the topics covered.

1.1. Motivation

Genome sequencing is a topic that has long since reached public attention. Even daily press provides articles about genomics from time to time. For example: A research group from Decode, an Icelandic genetics firm, sequenced the whole genome of 2,363 Icelanders and gained a deep insight into how the diversity in the genomes affects human diversity [GHG+15]. This series of papers, published in the journal Nature Genetics, has reached widespread attention, e.g. in The New York Times [Zim15] and the German Spiegel [Lun15]. Yet, the field of genomic information processing still provides a wide variety of challenges for the scientific community. As already mentioned, there exists a still expanding gap between sequencing and

[3] MPEG output document number N15047.

storage costs, which is putting pressure on the development of sophisticated genomic information compression algorithms. The following paragraphs provide a overview of the current status of genomic information compression.

The sequenced human genome, stored as a FASTQ file, consumes about 1.5 TiB of hard disk storage. The current de-facto standard for the compression of FASTQ files is however plain general dictionary-based compression such as e.g. gzip [GA15][4]. A "gzipped" FASTQ file has a size of for example about 370 GiB, yielding space savings[5] of approximately 75%. Recent and more sophisticated tools such as Quip [JRPK12] yield slighter better space savings, reaching about 80%.

The human genome stored as a SAM file (containing additional meta data) even consumes about 3 TiB. The de-facto standard for the compression of SAM files - the tool SAMtools - produces so-called BAM files [SAM14]. Roughly speaking, SAMtools employs modified, block-based gzip compression. A structured overview about the obtainable compression ratios is given in Table 1.1.

File type	**Size**	**Tool**	**Compressed size**	**CR**
FASTQ	1.5 TiB	gzip	500 GiB	33%
		Quip	300 GiB	20%
SAM	3 TiB	BAM	700 GiB	23%

Table 1.1.: *Roughly approximated compression ratios for current de-facto standard tools.*

Besides the obtainable compression ratios, there are a number of other criteria for evaluating compression algorithms. The following list gives a generally accepted overview about data compression criteria.

- **Compression ratio:** The main goal of data compression (technically denoted as *source coding*) is to reduce the data size, i.e. to encode the data using less bits.
- **Lossless versus lossy compression:** Lossless compression only aims at removing *redundancy* from the data. Lossy compression schemes additionally try to eliminate the *irrelevance* contained in the data[6].
- **Speed:** The speed of the compression process respectively the decompression process is primarily determined by the computational complexity of the

[4] Gzip is employing the DEFLATE [Deu96] compression algorithm which in turn employs LZ77 (Lempel-Ziv 77) [ZL77] compression in combination with Huffman [Huf52] coding.

[5] *Space savings* is defined as $1 - {}^{Compressed\,Size}/_{Uncompressed\,Size}$.

[6] For example, in the terms of digital audio compression, certain frequency domains are not audible for the human ear ($f \geq 20\ kHz$) or are masked by adjacent frequency domains. This data is irrelevant for the perception of the audio signal and can be transmitted with a lower accuracy (this is known as *quantization*.

employed algorithms. In practice, implementation issues play another non-negotiable role.

- **Memory usage:** Especially, predictive coding schemes require the usage of more or less memory. Generally speaking, the availability of huge memory capacities enhances the obtainable compression ratios.
- **Random access:** The ability to directly access (and decompress) a compressed data element (e.g. a block) is called *random access*. Structuring the compressed data into independent blocks imposes a deterioration on the obtainable compression ratios. However, when regarding relatively large data sizes, random access is in a variety of cases a mandatory requirement and a trade-off between the block sizes and the compression ratios has to be found.

Concerning the above mentioned criteria for data compression, and the enormous and steadily growing amount of sequencing data, there is a need for superior genomic information compression algorithms, particularly in terms of random access. Except for SAMtools, genomic information compression tools that have especially been tailored to this specific application, do exist, but have not been widely adopted yet. As pointed out in Chapter 3, a variety of novel tools has recently reached attention of the scientific community. Nevertheless, each of these tools has certain drawbacks and does not provide a complete solution to the task. It can be identified, that there is still a need for more efficient genomic information compression.

In the scope of this thesis, I developed several novel approaches for genomic information compression. First, I evaluated the compression mechanisms of existing tools. Second, novel approaches improving the obtainable compression criteria (random access, memory usage, etc.) have been developed and tested.

1.2. Report Structure

Before discussing the details of genomic information compression, Chapter 2 gives a brief but comprehensive overview about the theoretical foundations necessary for the understanding of this report. Sections 2.1 and 2.2 provide the mathematical background before the fields of genomics, sequencing technologies, and the associated file formats will be examined in more detail in Sections 2.3 and 2.4.

Chapter 3 then introduces the special needs of genome compression, providing an overview about processing pipelines and the standardisation efforts yet started. Selected existing tools are presented and compared to provide a baseline for novel approaches.

Chapter 4 presents the approaches to genomic information compression developed in the scope of this thesis. In particular, Section 4.1 provides statistical analyses, yielding the rationales behind the developed algorithms.

Finally, Chapter 5 then first introduces the software framework that has been developed for the evaluation of the developed compression approaches. Before presenting the obtained results, the MPEG Genomic Information Database (GIDB) is presented and explained.

The work done in the scope of this thesis, and by other researchers, raised new questions about genomic information compression. Together with a summary, this outlook is presented in Chapter 6.

2. Theoretical Foundations

The following sections give an overview about the mathematics and theorems necessary for the understanding of the presented approaches to genomic information compression. The most important and fundamental theorems of the theory of probability are presented in Section 2.1. Section 2.2 introduces the basic concepts of modelling discrete information sources. The selection of mathematical symbols has been done according to Musmann et al. [MO11, MO12, MO*12].

Section 2.3 then introduces the basic biological terms regarding genomic information and presents a short history of DNA sequencing as well as an overview about contemporary sequencing and post-processing workflows. Finally, Section 2.4 phases in the actual file formats used for genomic information storage.

2.1. Statistics

2.1.1. Probability

The probability $P = P(u)$ of some event u is introduced according to the axiomatic definition by Kolmogoroff.

Let u be an *elementary event* from the event space $\mathcal{A} = \{a_1, \dots a_k, \dots, a_K\}$ with the probability of occurrence $P(u)$. Then all elementary events a_k together with all associated probabilities $P(a_k)$ form a so-called *ensemble* U. All $P(u)$ are probabilities, if the following equations hold.

$$0 \leq P(u) \leq 1 \tag{2.1}$$

$$\sum_u P(u) = 1 \tag{2.2}$$

Let us now consider the joint event of N elementary events $(u_1, \dots, u_n, \dots, u_N)$ with $u_n \in \mathcal{A} \; \forall \; n$. The N elementary events u_n can be obtained by performing N parallel and simultaneous experiments or by performing N repetitions of the same experiment. All possible K^N joint events $(u_1, \dots, u_n, \dots, u_N)$ with the joint probabilities $P(u_1, \dots, u_n, \dots, u_N)$ form a so-called *joint ensemble* $(U_1, \dots, U_n, \dots, U_N)$ with the following properties.

$$0 \leq P(u_1, \ldots, u_n, \ldots, u_N) \leq 1 \tag{2.3}$$

$$\sum_{u_1} \cdots \sum_{u_N} P(u_1, \ldots, u_n, \ldots, u_N) = 1 \tag{2.4}$$

Example: Flipping coins Consider the flip of two fair coins. Let r and s be discrete random variables associated with the outcome of flipping the first coin respectively flipping the second coin. If a coin displays "head", then r respectively s is 1, and is 0 otherwise. All possible outcomes are $(r = 0, s = 0), (r = 0, s = 1), (r = 1, s = 0), (r = 1, s = 1)$, with $P(r, s) = 1/4 \; \forall \; r, s \in \{0, 1\}$ and $\sum_r \sum_s P(r, s) = 1$, as all outcomes are equally probable.

Let us now consider that an event u_N is dependent from a previous joint event $(u_1, \ldots, u_n, \ldots, u_{N-1})$. The *conditional probability* of the occurrence of u_N given $(u_1, \ldots, u_n, \ldots, u_{N-1})$ is denoted as $P(u_N \,|\, u_1, \ldots, u_n, \ldots, u_{N-1})$.

$$0 \leq P(u_N \,|\, u_1, \ldots, u_n, \ldots, u_{N-1}) \leq 1 \tag{2.5}$$

$$\sum_{u_N} P(u_N \,|\, u_1, \ldots, u_n, \ldots, u_{N-1}) = 1 \tag{2.6}$$

The connection of conditional and joint probabilities is given by $P(u_N, u_{N-1}) = P(u_N \,|\, u_{N-1}) \cdot P(u_{N-1})$, which directly conducts Bayes' theorem [GS97].

$$P(u_N, u_{N-1}) = \frac{P(u_{N-1}, u_N) \cdot P(u_N)}{P(u_{N-1})} \tag{2.7}$$

If the occurrence of the event u_N is not related with the occurrence of the previous event u_{N-1}, the two events are said to be *statistically independent*[1].

$$P(u_N \,|\, u_{N-1}) = P(u_N) \tag{2.8}$$

$$P(u_N, u_{N-1}) = P(u_N) \cdot P(u_{N-1}) \tag{2.9}$$

Generally speaking, the ensembles $(U_1, \ldots U_n, \ldots, U_N)$ are statistically independent, if Equation 2.10 holds.

[1] Referring to the coin example above: Flipping two fair coins generates statistically independent events r and s with $P(r, s) = P(r) \cdot P(s) = 1/2 \cdot 1/2 = 1/4$.

$$P(u_1, \dots u_n, \dots, u_N) = \prod_{n=1}^{N} P(u_n) \tag{2.10}$$

In terms of handling random experiments in the sections hereinafter, it is beneficial to define so-called discrete random variables. Let us define a discrete random variable x as a function which is assigning a real number $x = x(u),\ x \in \mathbb{R}$ to every elementary event $u \in \mathcal{A}$ with $\mathcal{A}$ containing a finite number of K elements. The *probability mass function* (pmf) $P(x = x_k)$ with $k = 1, 2, \dots, K$ is then the function that gives the probability that $x = x_k$. Following from the pmf, we can define the *cumulative distribution function* (cdf) of x as shown in Equation 2.11.

$$P(x \leq X) = \sum_{x_k \leq X} P(x = x_k) \tag{2.11}$$

According to the probability axioms in Equation 2.1, the sum of all probabilities given by the pmf must yield one.

$$P(x \leq \infty) = \sum_{x_k \leq \infty} P(x = x_k) = 1 \tag{2.12}$$

As an example, a pmf and the corresponding cdf are shown in Figure 2.1.

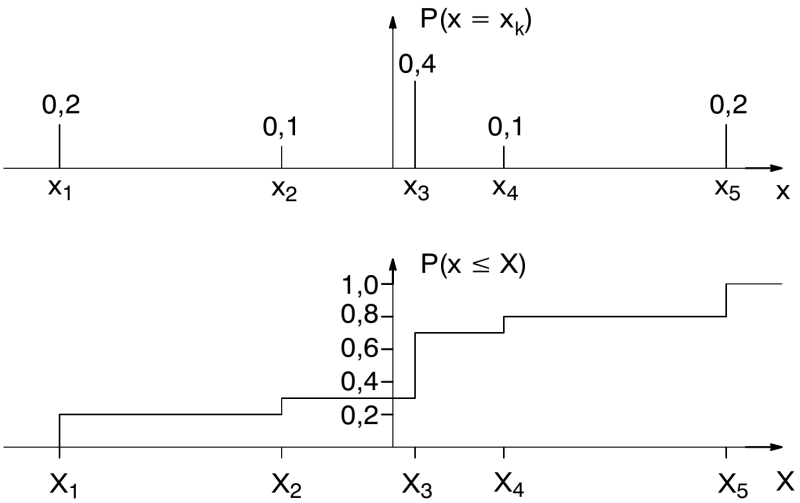


Figure 2.1.: *The figure on the top shows the pmf of a discrete random variable assigning values to $K = 5$ elementary events. The figure on the bottom shows the corresponding cdf. Note that $\sum_{k=1}^{5} P(x = x_k) = 1$. The Figure has been taken from [MO12].*

In the case of continuous random variables, the pmf is being replaced by the *probability density function* (pdf) $p_x(x)$ and the discrete sums in the formulas above are replaced by integrals. The following sections use in some cases continuous random variables. However, in the scope of this thesis only discrete random variables are of interest.

2.1.2. Correlation and Statistical Independence

Let us now consider a continuous random variable x. The n-th moment of x is denoted as $E\left[x^n\right]$ and shown in Equation 2.13.

$$E\left[x^n\right] = \int_{x=-\infty}^{+\infty} x^n p_x(x)\, dx \tag{2.13}$$

The first moment ($n = 1$) of x is known as the *linear mean* m_x.

$$m_x = E\left[x^1\right] = E\left[x\right] = \int_{x=-\infty}^{+\infty} x\, p_x(x)\, dx \tag{2.14}$$

With the linear mean m_x, we can now define the n-th *central moment* μ_n.

$$\mu_n = E\left[(x - m_x)^n\right] = \int_{x=-\infty}^{+\infty} (x - m_x)^n\, p_x(x)\, dx \tag{2.15}$$

The second central moment ($n = 2$) is known as *variance* $\sigma_x^2 = E\left[(x - m_x)^2\right]$. The positive square root is known as *standard deviation* σ_x. According to the definition of the n-th moment respectively central moment, we can define the $(n+m)$-th joint moment respectively joint central moment for two random variables x and y. The joint moment for $n = m = 1$ is known as *correlation*.

$$E\left[x \cdot y\right] = \int_{x=-\infty}^{+\infty} \int_{x=-\infty}^{+\infty} x \cdot y \cdot p_{xy}(x, y)\, dx\, dy \tag{2.16}$$

With the joint central moment for $n = m = 1$ being known as *covariance* $\mu_{11} = E\left[(x - m_x) \cdot (y - m_y)\right]$, we can define the Pearson product-moment correlation coefficient (PCC) expressing the strength of the *linear*[2] relationship between x and y.

$$\rho_{xy} = \frac{\mu_{11}}{\sigma_x \cdot \sigma_y}\ ,\ |\rho_{xy}| \leq 1 \tag{2.17}$$

[2] Anscombe showed with his quartet of graphs, that the PCC fails to express higher-order correlations [Ans73].

If the two variables are uncorrelated, then $\rho_{xy} = 0$. If $|\rho_{xy}| = 1$, the distribution of x and y shows a clear linear dependence. Moreover, if the two variables x and y are statistically independent, then they are uncorrelated, as shown in Equation 2.18. However, it is important to note, that the inverse does not hold, i.e. if two variables x and y are correlated, one can *not* conclude that they are statistically dependent (nevertheless, correlation indicates a predictive relationship that can be exploited in practice).

$$\begin{aligned} &p_{xy}(x,y) = p_x(x) \cdot p_y(y) \\ &\Rightarrow E\left[x \cdot y\right] = E\left[x\right] \cdot E\left[y\right] = m_x \cdot m_y \\ &\Rightarrow \rho_{xy} = \frac{\mu_{11}}{\sigma_x \cdot \sigma_y} = \frac{E\left[(x - m_x) \cdot (y - m_y)\right]}{\sigma_x \cdot \sigma_y} = \frac{E\left[x \cdot y\right] - m_x \cdot m_y}{\sigma_x \cdot \sigma_y} = 0 \end{aligned} \tag{2.18}$$

2.1.3. Random Processes

In the previous sections, we introduced a discrete random variable x, which assigns a real number $x = x(u),\ x \in \mathbb{R}$ to every elementary event u from the event space $\mathcal{A}$. If in addition the discrete random variable depends on the time t, i.e. $x = x(u,t),\ t = k \cdot T,\ k \in \mathbb{N},\ T > 0$, we call this a *discrete random process*, in short $x(t)$. The same holds for continuous random variables ($t \in \mathbb{R}$), the process is then called *continuous random process*. If now the pmf respectively the pdf is invariant concerning time shifting (and thus the statistical properties - such as e.g. the expected value - of the process are), then the random process is denoted as being *stationary*, as shown in Equation 2.19.

$$p_{x(t_1),\ldots x(t_N)}(x_1,\ldots,x_N) = p_{x(t_1+c),\ldots x(t_N+c)}(x_1,\ldots,x_N) \tag{2.19}$$

Therefore, we can conclude, that for a stationary process the expected value is not time-dependent: $E\left[x(t)\right] = E\left[x\right]$. If now the time-based expected value equals the expected value over a set of realisations of the random process, i.e. $E\left[x\right] = \bar{x}$, then the process is called *ergodic*.

2.1.4. Correlation Functions and Power Spectral Density

As noted in Equation 2.16, the joint moment $E\left[x \cdot y\right]$ is known as correlation. Concerning two random processes, the corresponding expression $E\left[x(t_1) \cdot y(t_2)\right]$ is known as *cross correlation* (function) $R_{xy}(t_1,t_2)$. For stationary processes with $t_1 = t$ and $t_2 = t - \tau$, this reduces to $R_{xy}(\tau) = E\left[x(t) \cdot y(t-\tau)\right]$. Eventually, for ergodic processes (in this case a discrete process), the cross-correlation can be computed by convolution of the two signals. Equation 2.20 shows the cross-correlation for

a continuous (ergodic) random process whereas Equation 2.21 presents the cross-correlation (series) for a discrete random process $x(t_m) = x(m)$ with the discrete time steps $m = k \cdot T, \;\; k \in \mathbb{Z}, \; T > 0$.

$$R_{xy}(\tau) = (x \star y)(\tau) = \int_{t=-\infty}^{-\infty} x(t) \cdot y(t-\tau)\, dt \tag{2.20}$$

$$R_{xy}(n) = (x \star y)(n) = \sum_{t=-\infty}^{-\infty} x(m) \cdot y(m-n) \tag{2.21}$$

By computing the cross-correlation of a process $x(m)$ with the process itself, one obtains the *auto-correlation*[3] (series) $R_{xx}(n)$.

The *power spectral density* $S_{xx}(\omega)$ of a time-discrete stationary process $x(n)$ is then defined as the FOURIER transform of the auto-correlation series $R_{xx}(n)$.

$$S_{xx}(\omega) = \sum_{n=-\infty}^{+\infty} R_{xx}(n) \cdot e^{-j\omega n} \tag{2.22}$$

The auto-correlation series of a stationary (discrete) process $x(m)$ is a real and even series, i.e. $R_{xx}(n) = R_{xx}(-n)$. Moreover the maximum is at $n = 0$ ($R_{xx}(n) \leq R_{xx}(0)$). Important to note is, that the power of an average-free process $x(m)$ can be computed either as second moment of the process, or with the auto-correlation series or by evaluating the power spectral density.

Another important aspect is the relation of the auto-correlation series respectively the power spectral density to information sources with or without memory. A source emitting a message with statistically independent symbols (and thus *without* memory) would produce the delta impulse as auto-correlation series. Therefore, the power spectral density would be constant and vice versa. We will exploit this knowledge later on (in Chapter 4).

2.1.5. The Discrete Markov Process

A special - and in the following sections of outstanding importance - discrete random process is the finite, discrete MARKOV process [Mar06]. Such a process has a finite number of states $\mathcal{S} = (s_1, \ldots, s_j, \ldots, s_J)$, and a finite number of events $\mathcal{A} = (a_1, \ldots, a_k, \ldots, a_K)$. Upon occurrence of an event $u_n \in \mathcal{A}$ from a series of events $(u_1, \ldots, u_n, \ldots, u_N)$, the process transitions from its current state $r_n \in \mathcal{S}$ to

[3] In practice, one can only estimate the sequence, because only a finite segment of the theoretically infinite-length random process is available.

its next state r_{n+1}. If the probability that state r_N emits the symbol u_N depends on the $N-1$ preceding events $(u_1, \ldots, u_{N-1})$, the process is said to have the order $(N-1)$ whilst the states are defined by the $(N-1)$ preceding events.

2.2. Source Coding

2.2.1. The Discrete and Memoryless Source

Let us consider a discrete source, emitting N symbols $(u_1, \ldots, u_n, \ldots, u_N)$ (a message) with the symbols u_n being taken from an alphabet $\mathcal{A}$ containing K symbols $()a_1, \ldots, a_k, \ldots, a_K =$ whereas the symbol a_k occurs with the probability[4] $P(a_k)$, with $\sum_{k=1}^{K} P(a_k) = 1$.

In case of a *memoryless* source, the selected symbols u_n are *statistically independent*. This means, that the probability of a symbol u_N is independent from the probabilities of any previously sent symbols $(u_1, \ldots, u_{N-1})$: $P(u_N \,|\, u_1, \ldots, u_{N-1}) = P(u_N)$.

The information content $I(a_k)$ of a symbol a_k can be measured regarding its probability of occurrence $P(a_k)$. If the symbol a_k had the probability of occurrence $P(a_k) = 1$, then this symbol would not contain any information, as it is the only symbol from the K symbols that will ever be sent. The symbol a_k would carry no information $(I(a_k) = 0)$.

Vice versa, this can be formulated as: If there is a high *uncertainty* about the occurrence of a symbol a_k, this symbol does have a small probability of occurrence $P(a_k) \ll 1$, and thus contains much information. We can now define the information content $I(a_k)$ of a symbol a_k with the probability of occurrence[5] $P(a_k)$.

$$I(a_k) = -log_2(P(a_k)) \tag{2.23}$$

Computing the average information content of the alphabet $\mathcal{A}$ with K symbols results in the definition of the *entropy*[6] H.

$$H(\mathcal{A}) = -\sum_{k=1}^{K} P(a_k) \cdot log_2\left(P(a_k)\right) \tag{2.24}$$

[4] As noted in Section 2.1.1, the symbols a_k together with the probabilities $P(a_k)$ form a so-called *ensemble*.

[5] As in order to compute the information content the binary logarithm is used, the information content is measured in bit.

[6] The entropy is denoted with a Greek Eta (H). The terms of *information* and *entropy* in the context of information theory have first been formalised by Claude E. Shannon in "A Mathematical Theory of Communication" [Sha48].

An alphabet containing K symbols, and in which the occurrences of all symbols are equally probable ($P(a_k) = {}^1\!/_K \ \forall\ k \in \{1, \ldots, K\}$), yields the largest entropy, also denoted as *decision content* H_0.

$$H_{max}(\mathcal{A}) = -\sum_{k=1}^{K} \frac{1}{K} \cdot log_2\left(\frac{1}{K}\right) = log_2(K) = H_0 \tag{2.25}$$

If now all u are symbols selected from $\mathcal{A}$ with the probabilities $P(u)$, then these symbols and probabilities form another so-called *ensemble* U. The entropy of U can then be written as shown in Equation 2.26.

$$H(U) = -\sum_{u} P(u) \cdot log_2\left(P(u)\right) \tag{2.26}$$

Example: The information content of the human genome The human genome viewed from the perspective of information theory is simply a (very long) sequence of nucleotide symbols. The nucleotides are selected from an alphabet $\mathcal{G}$ with $g_1 = A, g_2 = C, g_3 = G, g_4 = g_K = T$. The human genome is estimated to consist of about 3.2 billion base pairs in total. Thus we can write the human genome as a sequence (or message) $(u_1, \ldots, u_n, \ldots, u_N)$ with $N \approx 3.2 \cdot 10^9$. Assuming, that the probabilities of occurrence are equal ($P(g_k) = {}^1\!/_4\ \forall\ 1 \leq k \leq K = 4$), we can then compute the entropy $H(\mathcal{G})$ of the human genome.

$$H(\mathcal{G}) = log_2(K) = 2\ {}^{bit}\!/_{symbol} \tag{2.27}$$

The information contained in all N symbols is therefore:

$$N \cdot H(\mathcal{G}) \approx 3.2 \cdot 10^9\ symbol \cdot\ 2\ {}^{bit}\!/_{symbol} = 6.4 \cdot 10^9\ bit = 6.4\ Mib = \underline{\underline{800\ MiB}} \tag{2.28}$$

2.2.2. The Source Coding Theorem

It is possible to assign K code words $\underline{x}_k$ with the lengths n_k to the K source symbols $(a_1, \ldots, a_k, \ldots, a_K)$, such that the mean code word length $\bar{n} = \sum_{k=1}^{K} P(a_k) \cdot n_k$ lies between the following bounds (D is the number of possible code symbols; in case of binary coding, $D = 2$). Equation 2.29 is also known as (Shannon's [Sha48]) *source coding theorem.*

$$\frac{H(U)}{log_2(D)} \leq \bar{n} < \frac{H(U)}{log_2(D)} + 1 \tag{2.29}$$

For the usual case of binary coding ($D = 2 \Rightarrow log_2(D) = 1$), the theorem can be reduced.

$$H(U) \leq \bar{n} < H(U) + 1 \tag{2.30}$$

The theorem implies to assign code words with the lengths n_k to the symbols a_k such that Equation 2.31 holds. Simply speaking, symbols with a high probability of occurrence are assigned short code words and vice versa.

$$I(a_k) \leq n_k < I(a_k) + 1 \tag{2.31}$$

The lower bound (optimal case) of $n_k = I(a_k)\ \forall\ k$ can thus only be reached if all probabilities $P(a_k)$ are integral powers of 2. In all others cases, the coding introduces *redundancy*. The redundancy R of a code with the mean code word length $\bar{n}$ can be computed as shown in Equation 2.32.

$$R = \bar{n} - H(U) \tag{2.32}$$

It can be shown, that the HUFFMAN algorithm yields the code with the smallest redundancy [Huf52]. When constructing a code, one has to take care of the fact, that the code can unambiguously be decoded. A class of codes known as *prefix-free codes*[7] conforms to this requirement[8]. To form a prefix-free code, the K code word lengths $n_1, \ldots, n_k, \ldots, n_K$ must comply according to Equation 2.33, which is known as KRAFT's inequality.

$$\sum_{k=1}^{K} D^{-n_k} \leq 1 \tag{2.33}$$

Example: Nucleotide sequence entropy Assume that we have a nucleotide sequence containing the $K = 5$ symbols[a] $a_1 = A, a_2 = C, a_3 = G, a_4 = T, a_5 = N$. The decision content of such a source is $H_0 = log_2(K) \approx 2.3219\ bit$. We now assume the following symbol distribution.

[7] The code word representing some particular symbol is never a prefix of the code word representing any other symbol.

[8] Actually, it can be shown that all unambiguously reversible codes can be represented by a prefix-free code.

Symbol	Absolute frequency	Relative frequency
A	12	0.3
C	8	0.2
G	13	0.325
T	5	0.125
N	2	0.05
Σ	40	1

This symbol distribution yields an entropy of approximately $H = 2.1036\ {}^{bit}/_{symbol}$ and thus has a redundancy of $R = H_0 - H = 0.2183\ {}^{bit}/_{symbol}$.

[a] The symbol "N" is sometimes used by sequencing devices if no decision about the base call can be made.

2.2.3. Entropy Coding

Generally, entropy coding schemes assign unique code words $\underline{x}(u)$ to the symbols u occurring in a message. Whereas the symbols all have the same length, the code words have a variable length, assigning the shortest code words to the most likely symbols, according to the source coding theorem. The following sections briefly introduce arithmetic coding as a method being fully independent from the characteristics of the message, and Golomb [Gol66] respectively Rice [RP71] codes as entropy coding schemes that can be employed when the source characteristics are roughly known in advance.

An alternative (or additional) notation of the source coding theorem shall be mentioned at this point. The upper bound of the theorem can be arbitrarily reduced by encoding blocks of L source symbols.

$$\frac{H(U)}{log_2(D)} \leq \bar{n} < \frac{H(U)}{log_2(D)} + \frac{1}{L} \tag{2.34}$$

2.2.3.1. Huffman Coding

As Huffman coding [Huf52] is not being relevant in detail in the scope of this thesis, the algorithm shall only be briefly explained here. The algorithm works by building a frequency-sorted binary tree to assign variable-length code words $\underline{x}(u)$ to the symbols u. The algorithm has been shown to be optimal among methods encoding symbols separately (with a known probability distribution) and precisely produces a redundancy-free code if all symbol probabilities are integral powers of two. However, a Huffman code might not be optimal, if the pmf is unknown.

Arithmetic coding can e.g. adapt to the input statistics and provide better compression ratios. Furthermore, in the case of HUFFMAN coding, the complete "code word table", determining the association among symbols and code words has to be transmitted in order to decode the bit stream.

2.2.3.2. Arithmetic Coding

HUFFMAN coding produce a code word for each single symbol. Arithmetic coding (AC) differs from this and other forms of entropy coding, as it encodes an entire message of length L into a single number. The algorithm for arithmetic coding has the advantage that no "HUFFMAN table" (of size K^L) has to be stored. Instead, only the K (initial[9]) probabilities have to be transmitted.

Let us assume that we have again K possible source symbols $(a_1, \ldots, a_K)$ with the probabilities $P(a_1), \ldots, P(a_K)$. To compute the (binary) code word $\underline{x}(a_k)$ we now define the accumulated probability P_k for the symbol a_k.

$$P_k = \sum_{\kappa=1}^{k-1} P(a_\kappa) \tag{2.35}$$

Then, we write out P_k as a binary number.

$$P_k = C(a_k) = \sum_{i=1}^{\infty} c_i \cdot 2^{-i} \tag{2.36}$$

According to the theorem of source coding, we may assign only n_k bits to $C(a_k)$.

$$-\,log_2 P(a_k) \leq n_k < -log_2 P(a_k) + 1 \tag{2.37}$$

Therefore, we only use the first n_k bits from $C(a_k)$ and write this number as $C'(a_k)$.

$$C'(a_k) = \sum_{i=1}^{n_k} c_i \cdot 2^{-i} \tag{2.38}$$

Now, $C'(a_k)$ might be smaller than P_k ($C'(a_k) \leq P_k$). Thus, we construct the code word $\underline{x}(a_k)$ according to the following equation.

$$\underline{x}(a_k) = \begin{cases} C'(a_k) & for \quad C'(a_k) = P_k \\ C'(a_k) + 2^{-n_k} & for \quad C'(a_k) < P_k \end{cases} \tag{2.39}$$

[9] In case of *adaptive* AC.

This scheme can now be applied to a block of L symbols $(u_1, \ldots, u_L)$ by subsequently dividing the accumulated probability intervals as roughly depicted in Figure[10] 2.2 (actually, the figure shows the decoding of a message of length $L = 3$, but the principle should be clear).

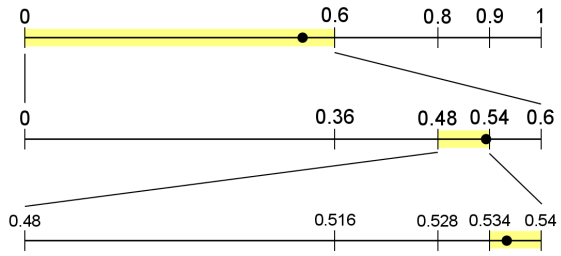


Figure 2.2.: *The code word* 0.538 *(decimal) has been received. Given the accumulated probabilities P_k and knowing the message length $L = 3$, the message can be decoded by subsequently dividing the accumulated probability intervals. Since arithmetic codes are* not *prefix-free, either the message length must be known, or an end-of-block symbol must be agreed upon.*

With AC, the possibility of *adaptive* entropy coding arises. By counting the relative symbol frequencies during the encoding process, the accumulated probability division can be adapted meanwhile. A practical example is *context-adaptive binary arithmetic coding* (CABAC), used for example in the H.264/MPEG-4 AVC and HEVC video coding standards [MSW03].

2.2.3.3. Golomb and Rice Coding

Alphabets following a geometric distribution will have a Golomb code as an optimal prefix-free code, whilst being more memory-efficient and faster as e.g. a Huffman code. The Golomb code was introduced in 1966 by Solomon W. Golomb [Gol66].

A non-negative integer $n \in \mathbb{N}$ with $\mathbb{N} = \mathbb{N}^+ \cup \{0\}$ can then be then be coded as a quotient q and a remainder r using a Golomb parameter $b \in \mathbb{N}^+$.

$$q = \left\lfloor \frac{n}{b} \right\rfloor \tag{2.40}$$

$$r = n - q \cdot b \tag{2.41}$$

$$c = \lceil log_2(b) \rceil \tag{2.42}$$

[10]The example has been taken from `https://en.wikipedia.org/wiki/Arithmetic_coding` (accessed: 2015-09-16). The image itself is public domain.

A RICE code is a GOLOMB code with b being a power of 2 ($b = 2^p$ with $p \in \mathbb{N}$). The computations for q and r can then be simplified.

$$q = n \gg p \tag{2.43}$$
$$r = n \wedge (b - 1) \tag{2.44}$$

In case of RICE coding [RP71], we need exactly $log_2(b) = log_2(2^p) = p$ bits to store the remainder. RICE codes are often used for coding prediction residues. Such residues tend to fall into a two-sided geometric distribution. RICE codes closely approximate the HUFFMAN code here, but without the overhead of having to transmit the HUFFMAN table. We will come back to RICE codes in Section 4.3.

Use with signed integers: GOLOMB and RICE codes have been constructed to code non-negative integers. By applying a simple overlap-and-interleave scheme, RICE coding can be applied to two-sided geometric distributions.

$$x' = 2x \;\text{ for }\; x \geq 0 \tag{2.45}$$
$$x' = -2x - 1 \;\text{ for }\; x \leq 0 \tag{2.46}$$

2.2.3.4. Dictionary-based Coding

Another class of entropy coding algorithms are the so-called dictionary-based coding schemes. Just as AC, theses schemes try to adapt to the statistics of the source. Briefly speaking, these algorithms "eat" the symbols emitted by the source one by one and build up a dictionary (i.e. a string table) containing already encountered strings. Upon encountering a string that has already been stored in the dictionary, the string is encoded as a reference to the dictionary.

These type of algorithms are suitable for memoryless sources as well as sources with memory (introduced in Section 2.2.4.1) and usually approach the source entropy quite well as the block length L is continuously increasing during the encoding process.

Two well-known variants are for example implemented in gzip [GA15] (using the DEFLATE algorithm which is based on the LZ77 algorithm, which is additionally employing a sliding window to generate the dictionary [ZL77], combined with HUFFMAN coding) and the Unix tool compress using the Lempel-Ziv-Welch (LZW) algorithm.

2.2.4. Predictive Coding

2.2.4.1. The Discrete Source with Memory

The discrete source *with* memory (i.e. a MARKOV source) shall be explained regarding a nucleotide sequence $(u_1, \ldots, u_n, \ldots, u_N)$. The probability of occurrence of a symbol u_n depends on the previously sent symbols (i.e. the events are statistically dependent); in this example, let us assume that we have a memory size of $M = 2$, i.e. the conditional probability for the occurrence of symbol u_N is $P(u_N \,|\, u_{N-1}, u_{N-2})$. The nucleotides are selected from an alphabet $\mathcal{A}$ containing $K = 4$ elements. Hence, there are $M^K = 2^4 = 16$ possible combinations for the symbols in memory. These possible combinations are called *states* $(s_1, \ldots s_j, \ldots, s_{(M^K)})$. The series of states being traversed while emitting the N symbols u_N is $(r_1, \ldots, r_n, \ldots, r_N)$. Each state s_j emits a symbol u_n with the conditional probability $P(u_n \,|\, s_j)$. The entropy of one state s_j can thus be computed as shown in Equation 2.47.

$$H(U|s_j) = -\sum_n P(u_n|s_j) \cdot log_2 \left(P(u_n|s_j)\right) \tag{2.47}$$

The process of entropy coding such a source is done by constructing a prefix-free code for every state s_j. The entropy of the entire source can then be computed as shown in Equation 2.48.

$$H(U) = \sum_j H(U|s_j)P(s_j) \tag{2.48}$$

2.2.4.2. Lossless Predictive Coding

However, the statistical dependence of the MARKOV source can be exploited by transmitting the difference of the current symbol u_N and a prediction value $u_N^{(p)}$ as shown in Figure 2.3.

The goal of predictive coding is to minimise the entropy $H(U_N - U_N^{(p)})$. In most cases, the formulation with M^K states $s_j = (u_1, \ldots, u_{N-1})$ can even be neglected as the conditional probabilities $P(u_N - u_N^{(p)} \,|\, u_1, \ldots, u_{N-1})$ become similar. Thus, after prediction, in most cases, only *one* entropy coder is necessary as shown in Figure 2.3.

To obtain the prediction values without any prior knowledge, the predictor selects in each state s_j the symbol a_i with the greatest conditional probability as shown in Equation 2.49 (this is known as *maximum-likelihood predictor*). The difference $(u_n - u_N^{(p)})$ then has the greatest probability.

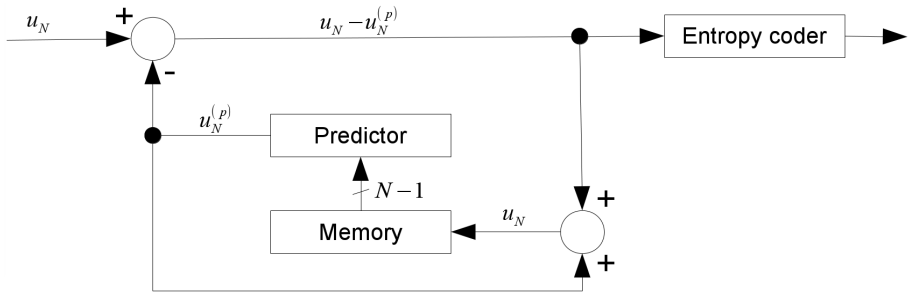


Figure 2.3.: *Schematic of a predictive encoder using the $N-1$ previously sent symbols $(u_1, \ldots, u_{N-1})$ to generate a prediction value $u_N^{(p)}$.*

$$P(u_N^{(p)} = a_i \,|\, s_j) \geq P(u_N = a_k \,|\, s_j) \tag{2.49}$$

2.2.5. Transform Coding

The *discrete cosine transform* (DCT) has been used for preliminary investigations regarding the compression of nucleotide sequences in Section 4.1.4. Therefore, the principles of transform coding are briefly summarised in this section.

In a variety of use cases, it is beneficial to obtain another representation of the data emitted by the source. This can be accomplished using *transform coding*. The digital source signal is splitted into blocks, with each block containing N symbols: $\underline{x} = (x_1, \ldots, x_n, \ldots x_N)^T$, $x_n \in \mathbb{C}$. Let us now define a (square) transformation matrix $\underline{T}$ with N rows and N columns.

$$\underline{T} = \begin{pmatrix} t_{1,1} & t_{1,2} & \cdots & t_{1,N} \\ t_{2,1} & t_{2,2} & \cdots & t_{2,N} \\ \vdots & \vdots & \ddots & \vdots \\ t_{N,1} & t_{N,2} & \cdots & t_{N,N} \end{pmatrix} \in \mathbb{C}^{N \times N} \tag{2.50}$$

We can now compute the N *transform coefficients* $\underline{c} = (c_1, \ldots c_m, \ldots, c_N)^T$ as shown in Equation 2.51.

$$\underline{c} = \underline{T}\,\underline{x} \;\Leftrightarrow\; \underline{x} = \underline{T}^{-1}\underline{c} \tag{2.51}$$

If now $\underline{T}^{-1} = \underline{T}^{*T}$, then the transform $\underline{T}$ is called a *unitary* transform. If $\underline{T}$ is a real square matrix ($\underline{T} \in \mathbb{R}^{N \times N}$) then the transform is called *orthogonal*. The goal of such transformations is to produce statistically independent transform coefficients (i.e. to decorrelate the data).

$$H(\underline{c}) = \sum_{m=1}^{N} H(c_m) \tag{2.52}$$

However, in theory only the *Karhunen-Loeve-Transform* (KLT) produces statistically independent transform coefficients [RY01]. The KLT matrix in defined as $\underline{T}_{KLT}^{-1} = (\underline{t}_1, \ldots, \underline{t}_n, \ldots \underline{t}_N)$ with $\underline{t}_n$ being the eigenvectors of the covariance matrix $\underline{R}$ as shown in Equation 2.53.

$$\underline{R}\,\underline{t}_m = \lambda_m\,\underline{t}_m \;\text{ with }\; \underline{R} = E\left[\underline{x}\,\underline{x}^T\right] \tag{2.53}$$

However, the KLT depends on the actual signal, as it is computed using the covariance matrix.

It can be shown that the (unitary) discrete Fourier transform (DFT) produces uncorrelated coefficients c_m for large N and stationary processes. However, computations are more efficient using the discrete cosine transform (DCT). The DCT is a widely used transform (e.g. in the JPEG image compression standard [PM93]). The detailed mathematical derivation is omitted here. However, there are several variants of the DCT. The one implemented in MATLAB[11] is known as DCT-IV with the coefficients shown in Equation[12] 2.54.

$$t_{mn} = \sqrt{\frac{2-\delta_{(m-1)}}{N}}\, cos\left(\frac{\pi}{N}\left(n-\frac{1}{2}\right)(m-1)\right) \;\text{ with }\; n,m = 1,\ldots,N \tag{2.54}$$

The two-dimensional DCT of e.g. an image block $\underline{X}^{N\times N}$ can be computed as shown in Equation 2.55.

$$\underline{C} = \underline{T}\,\underline{X}\,\underline{T}^T \tag{2.55}$$

2.3. Genomics and Sequencing Technologies

Before explaining the DNA and sequencing technologies in detail, the most fundamental terms are explained in the following paragraph.

The field of genetics is the study of genes and associated processes such as heredity. Genes are made up of deoxyribonucleid acid (DNA). The total amount of genes of an

[11] For example: In MATLAB, the DCT matrix of size N can easily be computed using the function `dctmtx` from the Signal Processing Toolbox.

[12] With $\delta_{(m-1)}$ yielding 1 for $m = 1$ and 0 otherwise.

organism is known as genome, which in order contains the complete genetic information of an individual organism. The genome is stored in one or more chromosomes. Humans for example have diploid (having pairs of homologous chromosomes) chromosomes whereas e.g. common wheat might be diploid, tetraploid, or hexaploid. In the case of eukaryotic cells, the chromosomes reside in an organelle called cell nucleus. Some viruses in turn encode their genomic information in an RNA genome (these viruses are accordingly called RNA viruses). The term genomics designates a discipline in genetics which applies bioinformatics and DNA sequencing methods to sequence, assemble, and analyse the function and structure of genomes. The main goals of genomics are: the identification of the complete genome of organism, the comparison among different organisms, the study of mutations, and the identification of gene functions.

2.3.1. DNA

DNA is the molecule carrying the genetic information. Normally, DNA consists of two so-called strands forming a double helix as shown in Figure 2.4. The strands are made up of atomic units called nucleotides. Besides two other components[13], a nucleotide contains a so-called nucleobase. This can be either adenine (A), cytosine (C), guanine (G), or thymine[14] (T). Adenine and thymine, as well as cytosine and guanine form so-called base pairs (the human DNA consists for example of about 3.2 billion base pairs[15]). Simply speaking, genes are nucleotide sequences that get expressed in the real world.

An overview, zooming in from the level of a single eukaryotic cell to the DNA molecule, is shown in Figure 2.4.

Genes can influence the phenotype of an organism. During the processes of transcription and translation, a messenger RNA (mRNA) sequence is synthesised along one strand of the DNA. This copy controls the protein synthesis in the ribosomes. During protein synthesis, a chain of amino-acid is synthesised which then folds into a protein. There is a direct relation between nucleotide sequences and the proteins generated. One amino-acid is coded by three subsequent nucleotides, called codons, for example “UUU” and “UUC” code the amino-acid Phenylalanine.

2.3.2. Sequencing Technology

First of all, “sequencing” is the process of determining the order of nucleotides in DNA (or RNA). Current sequencing technologies obtain the genomic information in

[13] These components are a monosaccharide sugar called deoxyribose and a phosphate group.
[14] In the case of RNA, thymine is replaced by uracil (U).
[15] As already shown in Section 2.2.1, this yields an information content of about 800 MiB for the human genome.

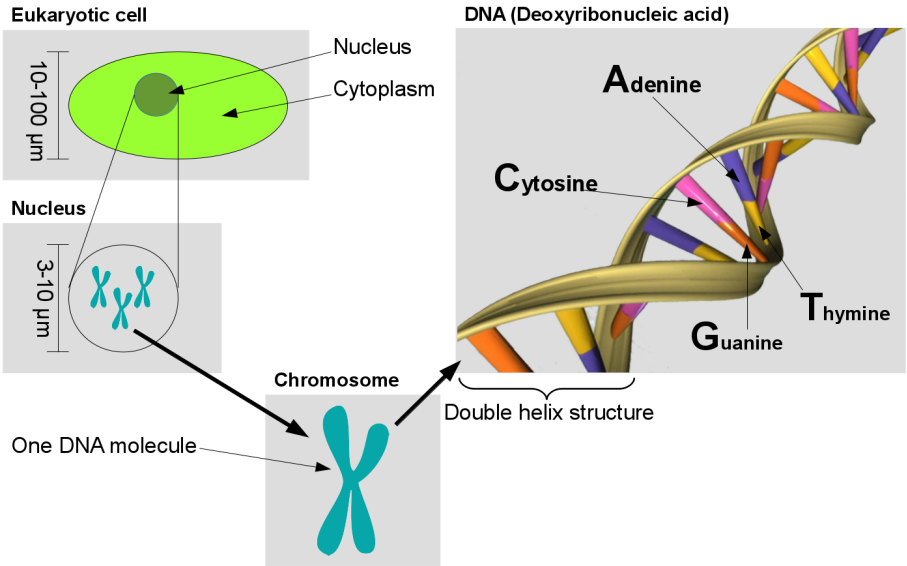


Figure 2.4.: *This figure shows where the DNA is located within a (eukaryotic) cell. In the first step, we zoom in into the nucleus, which contains the chromosomes. One chromosome contains exactly one DNA molecule, brought in shape by proteins (the complex of DNA and these proteins is then called chromatin). In the last step, we see the double helix structure of the DNA molecule. The four base types are represented by the coloured bars connecting the two strands.*

the form of so-called "reads". These are random fragments or genomic data. NGS technologies produce reads with a length of about one hundred up to thousands of base pairs. In general, shorter reads are more accurate and can be produced in much larger volumes. Mainly, there are two types of reads being produced: single reads and paired reads. Depending on the sequencing technology used, two reads can form a pair with the information that both reads have been produced from the same strand of DNA with a known distance (about 500-1,000 bp) between them.

Large-scale sequencing has numerous applications. According to Alberti et al. [AMC+14], some among them are: diagnostics, personalised medicine, and drug discovery. In the terms of personalised medicine, large-scale sequencing could be an important tool to "predict risks and drive preliminary therapeutic interventions". Concerning the second point, a deep insight into gene functions might improve and accelerate the process of drug discovery. According to Alberti et al., "medications and dosages could be optimised according to the genetic profile of individual patients" additionally.

Before talking about the so-called next-generation sequencing (NGS) technologies, a summary of the historical development of DNA sequencing and a comparison of

NGS technologies is provided in the following paragraphs. Underlying information[16] has been taken from Liu et al. [LLL+12].

2.3.2.1. First-generation Sequencing Technology

In 1977, Frederick Sanger published a DNA sequencing method which was based on the so-called "chain-termination method" [SNC77]. At that time, radioactive materials were required for the process of DNA sequencing. The so-called Sanger sequencing had (at that time) a high efficiency and (relatively) low radioactivity and was adopted in the "first generation" of sequencing applications. A few years later, in 1986, Applied Biosystems introduced the first automatic sequencing machine (working with the principle of "capillary electrophoresis"). Sanger sequencing as well as "capillary" sequencing machines became the main tools for the human genome project which ultimately yielded the complete sequencing of the human genome in 2001 [SSHP93].

Amongst others, this project stimulated the development of the so-called "next-generation sequencing" (NGS) tools, based on massively parallel analysis and yielding a high throughput and reduced cost. With NGS systems, the subsequent analysis of the sequencing data became the main bottle-neck in genetic information analysis.

2.3.2.2. Next-generation Sequencing Technology

Since the 1990s, several new methods for DNA sequencing were developed and implemented. The 454 (launched in 2005), Solexa's Genome Analyzer (released in 2006), and SOLiD (Sequencing by Oligo Ligation Detection) from Agencourt were the first massively parallel sequencing systems following the Sanger method. Most of the founder companies have been purchased by other companies, with the majority of them forming the NGS industry landscape today. Illumina Inc., Life Technologies (Thermo Fisher Scientific), 454 Lifesciences (subsidiary of Roche) and Pacific Biosciences of California, Inc. are to this point the manufactures selling the largest part of the sequencing machines today. Specifically, at the time of this thesis, Illumina holds the greatest market share in sequencing technology. At the end of 2014, Illumina machines accounted for more than 90 percent of all DNA data produced[17].

Without providing too much biochemical details, the following sections shall give an overview about the swift and emerging market of sequencing technologies at the example of the historical development of Illumina and the "sequencing by synthesis" method.

[16] Some information has additionally been taken from `https://en.wikipedia.org/wiki/DNA_sequencing`.

[17] According to the MIT Technology Review.

The term "sequencing by synthesis" designated the process of taking a single strand of DNA and synthesising the complementary strand enzymatically. Besides Illumina, a technology known as "pyrosequencing" incorporates the "sequencing by synthesis" approach, too. This method was first published by Ronaghi and Nyrén in 1996 [RKP+96]. The commercialisation of pyrosequencing was first pushed by Pyrosequencing AB. Later on, the pyrosequencing technology was licensed to 454 Life Sciences (amongst others). 454 finally developed a platform for large-scale sequencing through parallel pyrosequencing in 2004. However, 454 was purchased by Roche Diagnostics, which announced the discontinuation of this technology in 2013. Of course, there are other sequencing approaches besides "sequencing by synthesis". For example "sequencing by ligation", incorporated in the SOLiD system developed by Life Technologies, or ion semiconductor sequencing, using a semiconductor based detection system, developed by Ion Torrent Systems.

The "sequencing-by-synthesis" technology now used by Illumina was originally developed by Shankar Balasubramanian and David Klenerman at the University of Cambridge. They founded the company Solexa in 1998. In 2004, Solexa acquired the company Manteia Predictive Medicine, rendering Solexa able to use Manteia's massively parallel sequencing technology, based on signal amplification by using "DNA clusters". The first Solexa sequencer, the Genome Analyzer, was launched in 2006. In 2007, Solexa was purchased by Illumina[18].

Brown [Bro12] provides a good explanation of the Solexa/Illumina sequencing method: During the sequencing process, "only a single base can be added [...] to each growing DNA copy strand. The sequencing reaction is conducted simultaneously on a very large number [...] of different template molecules spread out on a solid surface. The terminator [...] contains a fluorescent label, which can be detected by a camera. Only a single fluorescent colour is used, so each of the four bases must be added in a separate cycle of DNA synthesis and imaging. [...] the images are recorded and the terminators are removed. [...] Since single bases are added to all templates in a uniform fashion, the sequencing process produces a set of DNA sequence reads of uniform length."

2.3.3. Comparison of Next-Generation Sequencing Technology

Modern sequencing devices such as for example Illumina's HiSeq series are able to extract large amounts of genetic information in a reasonable time. For example, in one run, machines from the HiSeq series produce about 5,000 million reads with an approximate read length of 150 base pairs [Ill15]. Because several reads map to the same location on the donor genome (i.e. they "overlap"), this data contains a huge amount of redundancy. Because sequencing technologies are more or less error-prone, this redundancy is naturally needed for error corrections. Neverthe-

[18] `https://www.illumina.com/`

less, this redundancy can be (temporarily) eliminated during compression to reduce the storage costs of NGS sequencing data. However, sequence data compression should ideally be lossless. The redundancy present in the data is on the one hand a statistical indication for the correctness of the reads and on the other hand, small differences in the donor genome may indicate important information regarding the resulting phenotypes.

Important manufacturers providing the sequencing technology described above are for example: Illumina Inc., Life Technologies, and Pacific Biosciences. All of these manufacturers use different sequencing technologies yielding different performances regarding for example the time per run. A short overview of some specific sequencing device specifications is shown in Table[19] 2.1.

Platform	**Illumina HiSeq 2000**	**PacBio RS**	**IonTorrent PGM**
Read length	up to 150 bp	~1,500 bp	~200 bp
Error rate	0.26%	12.86%	1.71%
Reads per run	up to 6 billion	~50,000	up to 80 million
Time per run	11 days	2 hours	2 hours
Cost*	0.05-0.15	0.13-0.60	1

Table 2.1.: *The technical specifications of three major sequencing technologies are compared. (* Cost per 1 million bases in US dollar)*

Each of the presented technologies has certain advantages and disadvantages. For example, Illumina's equipment is rather expensive but has the potential for a high sequence yield. PacBio machines have a moderate throughput but are rather fast and produce quite long reads. IonTorrent technology is in between the aforementioned.

To provide a practical example regarding the sequencing costs: The sequencing of a distinct human genome (containing about 3.2 billion base pairs) using an Illumina machine at a cost of approximately 0.10 USD per 1 million base pairs yields an investment of approximately 300 USD.

Two specific terms regarding reads shall additionally be briefly explained here. If a sequencing machine extracts two reads from the same long DNA strand, these reads are called a mate pair. A couple of reads extracted from a (short) DNA fragment with opposite orientation are called paired ends.

2.3.4. Representation of Genomic Information

Generally, genomic information, i.e. the nucleotide sequence of the analysed DNA (or RNA) molecule is expressed as a sequence of characters representing the encountered nucleotides respectively nucleobases. In the case of human DNA, this yields

[19]Data taken from [LLL+12] and [QSC+12].

a string composed of the (ASCII) characters "A", "C", "G", and "T". However, all NGS technologies only have a certain confidence, that a specific nucleotide has actually been found. Usually, this confidence is expressed by the additional so-called quality scores explained in Section 2.3.4.1. Another possible representation, unifying the nucleotide sequences and the quality scores, is the IUPAC notation introduced in Section 2.3.4.2. However, the majority of NGS sequencing machines produces quality scores.

2.3.4.1. Quality Scores

The large quantities of data produced by NGS machines, which are more or less error-prone, have required the development of new methods and programs for sequence analysis. As a major step, extending the "nucleotide string", the "quality scores" have been introduced; their development can be traced back to the year 1995 [BS95]. The first widely adopted quality scores were produced by a program called "Phred", linking the base-calling error probability p logarithmically to the so-called "Phred quality scores", shown in Equation 2.56.

$$Q_{Phred} = -10 \cdot log_{10}(p) \tag{2.56}$$

Quality scores currently produced by sequencing machines are explained in Section 2.4.2 (most machines store a modified quality value as an ASCII character). Sequencing processes based on *capillary electrophoresis* display their results typically as so called electropherogram, which is illustrating the principles behind quality scores quite well. A sample electropherogram is shown in Figure 2.5.

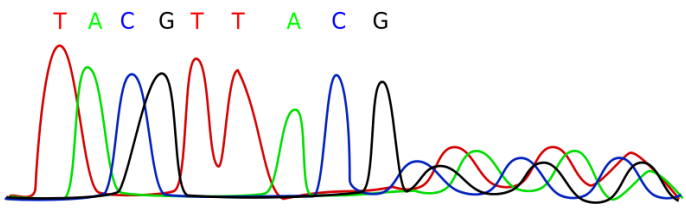


Figure 2.5.: *Schmematic diagram of an electropherogram giving the idea of how quality scores are generated. Picture was available for unlimited commercial use from Fabricatorz LLC.*

2.3.4.2. IUPAC Notation

Another representation of genomic information in contrast to the combination of nucleotide identifiers and quality scores is the IUPAC notation. The International Union for Pure and Applied Chemistry (IUPAC) standardised a larger set of 16

symbols, including the aforementioned. The additional 12 symbols incorporate for example the information of the incertitude between multiple bases (for example, "M" means "A or C"). The special symbol "N" represents an arbitrary base (not a gap). The IUPAC notation provide the opportunity to unify genomic information representation and to abandon the various flavours of quality scores. However, the use of the IUPAC notation is still limited in the scientific community and among sequencing machine manufacturers. This is due to the fact, that depending on the specific sequencing technology, IUPAC codes make more or less sense.

For example[20], pyrosequencing and equivalent technologies (e.g. IonTorrent) observe the size of interaction a single base type at a time produces. Such processes are said to be "clocked by base type". Therefore base substitutions are rare, whereas base indels are more common. The IUPAC notation perform badly in this case: it has codes for "A" or "C", but not for "A or AA". In contrast, Illumina machines are "clocked by base": Here, we have more substitution errors and less indels[21]. IUPAC notation might be beneficially applicable in this case.

2.3.5. Alignment and Assembly

The raw sequencing data coming off a sequencing device is usually composed of random fragments of the actual nucleotide sequences and the associated quality scores. Such a nucleotide sequence together with the associated quality scores is called "read". Usually, one read only represents a rather short sequence part of the given template and may in addition contain more or less errors. Furthermore, these reads do not come off in any order, nor is there any relative order known. The process of solving this "puzzle", i.e. (re-)arranging the raw reads such that they would map to a reference genome, is known as alignment respectively assembly.

Precisely speaking, "alignment" is the process of aligning sequence reads by finding regions of similarity. If the alignment is done with respect to a (pre-constructed) reference sequence, the process is called mapping. To provide practical examples, well-known alignment programs (known as "aligners") are for example the Burrows Wheeler Aligner (BWA) [LD09] or Bowtie [LTPS09] and Bowtie 2 [LS12].

The process of merging multiple alignments to reconstruct the donor DNA sequence is called "assembly". If an external reference is used, this is called "reference-based" assembly and otherwise de-novo assembly. Tools accomplishing non-reference-based alignment respectively de-novo assembly are called "Multiple Sequence Aligners", for example MAFFT [KMKM02] or MUSCLE [Edg04]. Alignment requires fast pattern matching algorithms [CLR04]. Other proposals for genome assembly have

[20] These examples have been discussed on the email reflector of the MPEG AhG on genome compression.

[21] Note: Original Illumina (Solexa) technology was emitting four confidence values (for "A", "C", "G", "T") per call. Nowadays, there is a solely single confidence value known as quality score.

been made, e.g. by Compeau et al. who introduced DE-BRUIJN graphs as a solution for assembly [CPT11].

An overview of the processing steps of sequencing and assembly is shown in Figure 2.6.

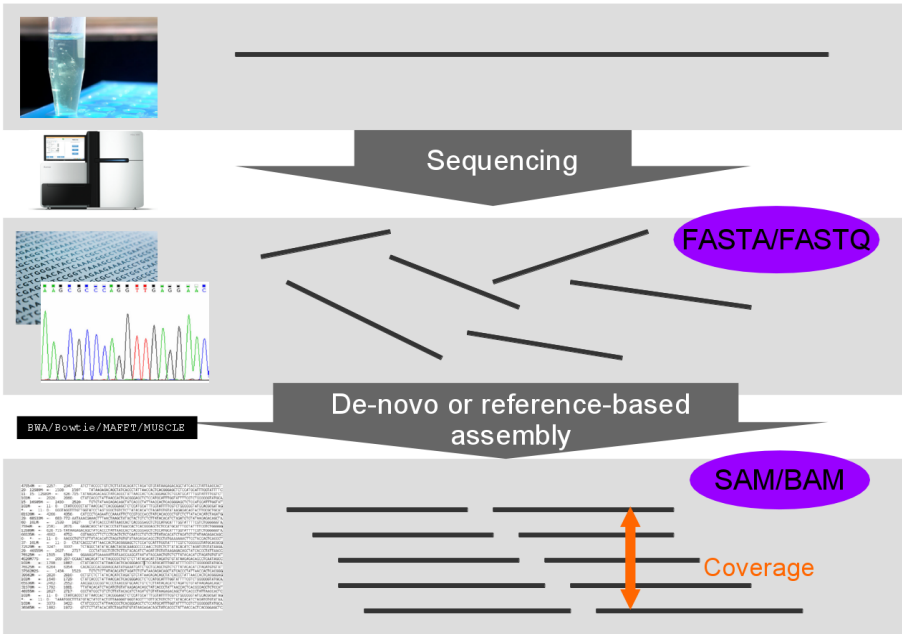


Figure 2.6.: *The (long) DNA molecule shown on the top is represented by the unsorted short reads in the second step (after sequencing). Besides the nucleotide sequences the process of sequencing produces additionally the corresponding quality scores. This raw data is usually stored in so-called FASTA (only nucleotide sequences) respectively FASTQ (containing the quality scores in addition) files as explained in detail in Section 2.4. During alignment/assembly, the reads are organised in order to hopefully reconstruct the donor genome. These mapped reads are represented in so-called SAM files, which contain the nucleotide sequences, the quality scores, and additional alignment and analysis information.*

In Figure 2.6, the term "coverage" is introduced (sometimes referred to as sequencing depth). Coverage refers to the (average) number of times a nucleotide has been sequenced. The coverage c can be computed as $c = {}^{N \cdot \bar{l}}/_{l_G}$ where N denotes the number of sequence reads, $\bar{l}$ denotes the average read length, and l_G denotes the length of the original genome. For clinical usage, a coverage of more than about 150 is usually required. Due to this relatively high coverage the question arises, if in terms of compression all reads must be preserved. The answer is "yes" (all reads must be preserved). Compression does not care about the semantics of the reads. A single base not matching onto the reference genome during mapping might be a

sequencing error or else a critical mutation in the donor genome.

2.4. File Formats

The major amount of raw sequencing data available today in public repositories is provided in two similar file formats: FASTA and FASTQ[22]. These textual[23] formats have been developed in the past, when the amount of generated sequencing data was not yet critical. In FASTA and FASTQ files, each nucleotide found via sequencing is stored as an 8-bit ASCII character. As in a majority of cases, the nucleotide sequence only contains five symbols[24] ("A", "C", "G", "T", and "N"[25]), storing these symbols as ASCII symbols introduces redundancy. With the development of NGS tools, and the growing amount of sequencing data generated, well-known compression tools such as gzip have been adopted for the compression of FASTA respectively FASTQ files.

Regarding aligned data, amongst others (e.g. CRAM[26]), a file format known as SAM is in widespread use today. SAM files contain additional alignment and analysis information, but introduce even more redundancy. There exists a compressed "equivalent" of SAM, called BAM. Basically, BAM files are block-wise "gzipped" SAM files. An overview of the mentioned file formats is shown in Figure 2.7.

2.4.1. FASTA

Files in FASTA format contain a number of *records* which contain a read header and the sequence reads. The first character in the header ("`>`") is standardised to signal the start of a new read[27]. Typically, the header contains information about the originating experiment, the sequencing technology used, as well as other optional metadata. The sequence read contains a string made up of usually 5 symbols: "A", "C", "G","T", and "N"[28]. "N" is used when the sequencing machine could not take a decision about the base call. In the case of RNA sequencing, "T" is replaced by "U" (abbreviation for uracil).

[22] Actually, FASTQ exists in two different flavours as pointed out later in Section 2.4.2.

[23] Textual formats provide the advantage to be accessible via scripts and long-since established (Unix) command line tools. These features have been (amongst others) the key to their success.

[24] Generally speaking, the sequences may either contain protein sequences or nucleic acid sequences according to the IUPAC amino-acid and nucleic acid codes.

[25] The symbol "N" is used when the sequencing machine cannot make a decision about a base call.

[26] CRAM has been developed by the European Bioinformatics Institute (EBI) and yields overall better compression ratios than BAM. However, the majority of aligned data available exists in the form of large BAM files.

[27] Legacy FASTA files may use ";" instead.

[28] As already pointed out, the sequence can contain a protein or a nucleic acid sequence, conforming to the IUPAC standard codes.

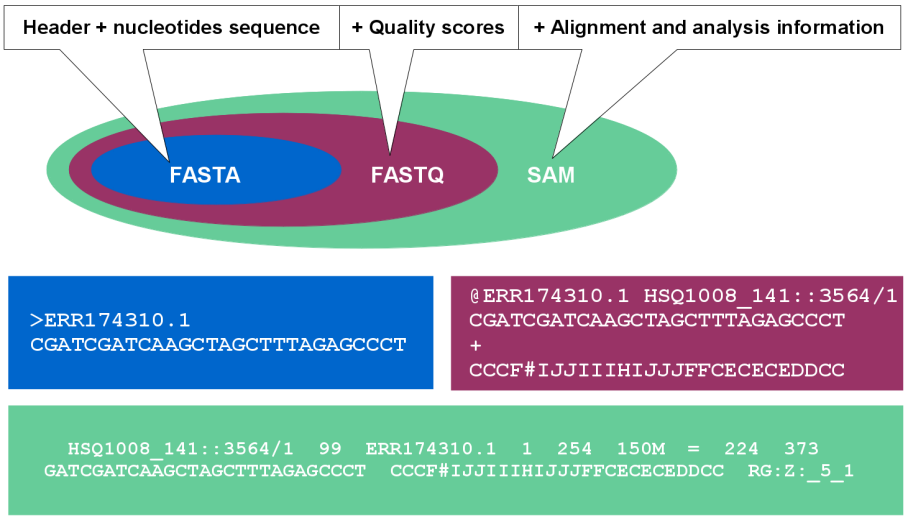


Figure 2.7.: *The main file formats for the representation and storage of genomic information data currently in use are depicted ordered by complexity. The corresponding rectangular boxes show examples for a single so-called* record *of each file format. The top part of the figure is oriented towards a figure presented in the MPEG document N15092 [AMC+15].*

2.4.2. FASTQ

The file format FASTQ is an extension to the FASTA format. It provides the additional ability to store quality scores. These are numeric values associated with each nucleotide in a sequence. However, the combination of nucleotide symbols and quality scores represent only a rough approximation to the underlying mechanisms of the employed sequencing technology.

The FASTQ format was originally developed at the Wellcome Trust Sanger Institute[29], but emerged as the de-facto standard for the storage of sequencing data. The most common file name extensions for FASTQ files are `.fq` and `.fastq`. The FASTQ format exists in two different flavours: the original Sanger FASTQ format, and the Solexa/Illumina FASTQ variants. Cock et al. provide a detailed summary about the commons and differences of the different FASTQ variants [CFG+10]. This section does not explain the details between the different FASTQ variants, but provides the important facts that are necessary to deal with the format in terms of compression.

FASTQ is a textual format. One FASTQ entry - also called *record* - consists of four lines:

1. **Read header/sequence identifier:** This line starts with an "@" character

[29] `https://www.sanger.ac.uk/`

and is comparable to the first line of a FASTA record. It contains a sequence identifier and an optional description[30].

2. **Sequence read:** The sequence as a string of ASCII symbols (usually "A", "C", "G", "T", and "N" are used for nucleic acid sequences).

3. **Description:** This line starts with a "+" and contains an *optional* description. Most of the available FASTQ files leave this line empty (it then only contains the "+" as a separator between the read and the quality scores).

4. **Quality scores:** This line contains one quality score for each nucleotide from line 2. Therefore, lines 2 and 4 must have the same length.

The read header and the sequence read are identical to the corresponding fields in the FASTA format, with the exception, that in FASTQ an "@" signals the start of a new record.

The quality scores are ASCII [Mac80] characters that are computed from the base calling *error* probability p. The original Sanger FASTQ format used the so-called *Phred quality score*, which is shown in Equation 2.57.

$$Q_{Phred} = -10 \cdot log_{10}(p) \tag{2.57}$$

The original Sanger format could encode Phred quality values from 0 up to 93, using the ASCII[31] codes from 33 ("!") to 126 ("~") as shown in Equation 2.58.

$$Q_{Sanger} = [-10 \cdot log_{10}(p)] + 33 \tag{2.58}$$

FASTQ file size: The approximate size of a FASTQ file holding a sequenced human genome (approximately 3.2 billion base pairs) with a coverage of $c \approx 200$ can be computed as: $3.2 \cdot 10^9 \cdot 8\ bit \cdot 2 \cdot 200 \cdot 1.15 \approx 1.5\ TiB$, whereas each nucleotide and each associated quality score is stored as an 8-bit ASCII character. The major factors for these huge file sizes are the sequencing depth (i.e. coverage) and the representation of nucleotides and quality scores as ASCII characters.

Illumina (respectively Solexa) used different metrics for the representation of quality scores in FASTQ files. Important to note is, that from Illumina 1.3 to 1.8, Phred quality scores in a range from 0 to 62 using the decimal ASCII codes 64 to 126 have been employed. Since 1.8, Illumina's processing pipelines produce Sanger quality scores, too.

[30] The sequence identifiers generated by Illumina machines are for example loosely standardised. They contain for example an instrument identifier, a run number on the instrument, and other information to uniquely identify the read.

[31] We talk about the decimal ASCII codes here.

As a short example: Having an error probability of $p = 0.1$ for a given position, then the corresponding ASCII character for the SANGER quality score is: $Q_{ASCII} = -10 \cdot log_{10}(0.1) + 33 = 10 + 33 = 43 \widehat{=} +$.

One more reason for the adoption of FASTA and FASTQ as de-facto standards is the ability to process these text files quite easily on "unixoid" platforms[32] (using e.g. Perl, Python, or shell scripts).

2.4.3. SAM

The SAM format consists of two main sections:

- An optional header section. All header lines start[33] with "@".
- An alignment section containing the so-called records.

In the scope of this report, the SAM header is not important. For a detailed description of the header syntax, please refer to the SAM format specification [SAM14].

The alignment section consists again of so-called records, whereas each record occupies one line, in contrast to FASTA respectively FASTQ files, where a record was spread among multiple lines. A SAM record is made up of 11 mandatory fields (columns), which are separated by tab stops. All fields after the 11th columns are optional. Table 2.2 gives an overview about these fields and provides short examples[34].

Figure 2.8 and Figure 2.9 show how sequence reads are formatted in SAM files. The figures have been taken from the SAM specification [SAM14].

Regarding SAM files and aligned reads, there is a certain terminology to be taken care of. The following items explain the most important terms and have in large part been taken from the SAM/BAM specification [SAM14] and the MPEG output document N15092 [AMC+15].

- **Template:** A DNA/RNA sequence part which is sequenced on a sequencing machine or assembled from raw sequences.
- **Segment:** A contiguous sequence or sub-sequence.
- **Read:** A raw sequence that comes off a sequencing machine. A read may consist of multiple segments.

[32] For the validation of the syntax of various FASTQ files, I adopted a script from Tyler Backman. The Perl script can be found in the samsplit repository (please see Appendix A for more detailed information).

[33] The SAM format is a bit sloppy here, as "@" is a possible character for quality scores, too.

[34] The table except for the sample content is taken from [SAM14].

Col.	Field	Type	Sample content	Description
1	QNAME	String	`HSQ108_141::3564/1`	Query template NAME
2	FLAG	Int	`99`	bitwise FLAG
3	RNAME	String	`ERR174310.1`	Reference sequence NAME
4	POS	Int	`1`	Mapping POSition
5	MAPQ	Int	`254`	MAPping quality
6	CIGAR	String	`150M`	CIGAR string
7	RNEXT	String	`=`	Ref. name of the mate/next read
8	PNEXT	Int	`224`	Position ot the mate/next read
9	TLEN	Int	`373`	observed Template LENgth
10	SEQ	String	`GATCGATCAAGCTAGCTT`	segment SEQuence
11	QUAL	String	`CCCFFIJJIIHIJJJFFC`	QUALity scores
12	OPT	String	`RG:Z:_5_1`	Optional information

Table 2.2.: *The first 11 fields are mandatory for each SAM record. The 12th field may contain optional columns, separated by tab stops. The nomenclature in this table was chosen according to the SAM specification [SAM14]. Column 4 ("Sample content") provides illustrative content for each field.*

- **Linear alignment:** An alignment of a read to a single reference sequence that may include insertions, deletions (indels), skips and clipping, but may not include direction changes (i.e. one portion of the alignment on forward strand and another portion of alignment on reverse strand). A linear alignment can be represented in a single SAM record.
- **Chimeric alignment:** An alignment of a read that cannot be represented as a linear alignment. A chimeric alignment is represented as a set of linear alignments that do not have large overlaps. Typically, one of the linear alignments in a chimeric alignment is considered the "representative" alignment and the others are called "supplementary" and are distinguished by the supplementary alignment flag.
- **Read alignment:** A linear alignment (one SAM record) or a chimeric alignment (several SAM records) that is the complete representation of the alignment of the read.
- **Multiple mapping:** The correct placement of a read may be ambiguous, e.g. due to repeats. In this case, there may be multiple read alignments for the same read. One of these alignments is considered primary. All the other alignments are considered "secondary". Typically the alignment designated primary is the best alignment, but the decision may be arbitrary.

```
Coor     12345678901234  5678901234567890123456789012345
ref      AGCATGTTAGATAA**GATAGCTGTGCTAGTAGGCAGTCAGCGCCAT

+r001/1        TTAGATAAAGGATA*CTG
+r002         aaaAGATAA*GGATA
+r003       gcctaAGCTAA
+r004                     ATAGCT.............TCAGC
-r003                            ttagctTAGGC
-r001/2                                      CAGCGGCAT
```

Figure 2.8.: *Reads aligned to a reference (`ref`). `r001/1` and `r001/2` represent a read pair, `r002` is a single read, `r003` is a chimeric read, and `r004` represents a split alignment (this read needs to be split in order to properly be mapped to the reference genome). Nucleotide symbols in uppercase are bases that match to the reference. Nucleotides which do not match are represented in lowercase. Dots represent a gap (unknown sequence) which separates a split alignment.*

```
@HD VN:1.5 SO:coordinate
@SQ SN:ref LN:45
r001   99 ref  7 30 8M2I4M1D3M = 37  39 TTAGATAAAGGATACTG *
r002    0 ref  9 30 3S6M1P1I4M *  0   0 AAAAGATAAGGATA    *
r003    0 ref  9 30 5S6M       *  0   0 GCCTAAGCTAA       * SA:Z:ref,29,-,6H5M,17,0;
r004    0 ref 16 30 6M14N5M    *  0   0 ATAGCTTCAGC       *
r003 2064 ref 29 17 6H5M       *  0   0 TAGGC             * SA:Z:ref,9,+,5S6M,30,1;
r001  147 ref 37 30 9M         =  7 -39 CAGCGGCAT         * NM:i:1
```

Figure 2.9.: *The aligned reads from Figure 2.8 have been formatted as* SAM *file (quality scores have been omitted here).*

Listing 2.1 shows a record from "real" SAM data. The data has been taken from the MPEG Genomic Information Database (file identifier `DH10B`[35]).

```
_5:1:3:5628:13832  147  EcoliDH10B.fa  4685987  254  150M  =
   4685803  -334  TCTGGCAGTGT...TAGTAAGTATTTTT  (3>>@::43:4...?GEAD
   <HGHFFFFD;B@  RG:Z:_5_1  BC:Z:1  XD:Z:150  SM:i:829  NM:i:0  AS:
   i:1657
```

Listing 2.1: *A record from "real"* SAM *data. In this example, optional information is present from the 12th column onwards. The columns `SEQ` and `QUAL` have been truncated to fit into the listing.*

[35]Please refer to Section 5.2 for a list of file identifiers.

2.4.4. BAM

The *Blocked GNU Zip Format* (BGZF) is "block compression implemented on top of the standard gzip [Deu96] file format" [SAM14]. A BGZF file is a series of concatenated so-called BGZF blocks. The maximum size for compressed as well uncompressed blocks is 64 kiB[36]. Each BGZF block is itself a gzip archive. This technique allows random access to the BGZF blocks.

Applying BGZF on a SAM file produces a so-called BAM file. The compressed data blocks in a BAM file can be accessed using virtual offsets into the BAM file. These offsets are unsigned 64-bit integers store in a so-called BAM index file with the suffix `.bai`[37]. The offsets are defined as `coffset<<6|uoffset`, where `coffset` is an unsigned byte offset into the BGZF file to the beginning of a BGZF block, and `uoffset` is an unsigned byte offset into the uncompressed data stream represented by that BGZF block (according to [SAM14]).

The SAM/BAM Format Specification [SAM14] provides more details about the BAM file structure.

[36]The SAM/BAM format specification [SAM14] mentions 64 Kb [sic!].
[37]Generating a BAM index file is explained in Appendix B.

3. Prior Work in Genome Compression and Storage

3.1. Problem Definition

Over the past few years, technological advances in sequencing have led to a faster and more cost-efficient approach to sequence individual genomes. Current predictions are, that in the not so distant future it will be common to sequence and analyse the entire genome of any patient much more frequently. Nevertheless, there remain many scientific and technological challenges to bring genome sequencing into widespread clinical practice.

Moreover, because of the enormous amount of sequencing data generated by NGS machines, which in addition is fastly and steadily growing, the storage and analysis of sequencing data is another problem which has to be taken care of. Novel processes and tools have to be developed to overcome the current limitations (in terms of storage space, processing speed, network bandwidth, and many more).

Stephens et al. [SLF+15] are projecting this development to the year 2025, and compared genomics with three other major generators of *Big Data*: astronomy, YouTube, and Twitter. For instance, Stephens et al. estimated the required storage space for the three mentioned domains in 2025: they estimate the field of astronomy to require approximately 1 EiB (per year), Twitter to require 1-17 PiB, YouTube to require 1-2 EiB, and genomics to even require 2-40 EiB a year. Including other estimates about data acquisition, analysis, and data distribution, their estimates show, that genomics is at least on par with the most demanding of all analysed domains. They conclude, that "now is the time for concerted, community-wide planning for the 'genomical' challenges of the next decade". As another conclusion, Stephens et al. identified a number of key technological needs for Big Data genomics. One need identified by Stephens et al. is improved data acquisition in terms of continued advances in sequencing technologies to "reduce costs, improve throughput, and achieve very high accuracy". Other identified needs are: the construction and maintenance of improved data centres and storage systems to aggregate large collections of genomic data, sophisticated data distribution through cloud-storage systems and a flexible infrastructure to perform analysis (for example pattern searching and matching) on huge collections of genomic data.

Regarding compression, there have been numerous efforts by various institutions. The result is a huge number of different tools and file formats only addressing

the specific needs of their authors. Standardised, but yet flexible tools, processing chains, and formats would be the key to satisfy the various needs of scientific and industrial communities.

To illustrate the mentioned issues, Figure 3.1 shows that in the time interval from 1996 to 2010 the amount of generated sequencing data (measured in kbases[1] per day) grows much faster than the available hard disk drive (HDD) storage capacity and much faster than the available computation power (measured in million instructions per second (MIPS)).

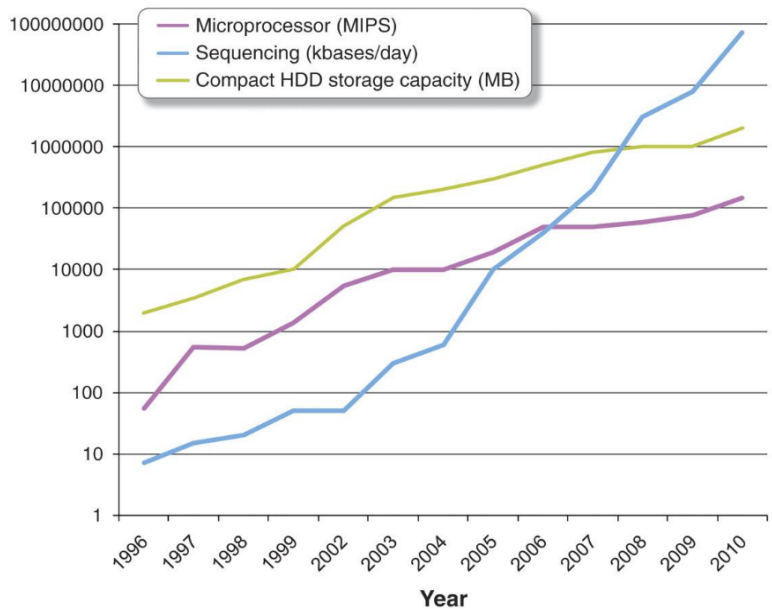


Figure 3.1.: *The amount of sequencing data generated by NGS machines outperforms the available storage and computation capacities. The chart has been taken from [WBL13] (the years 2000 and 2001 have been omitted by the authors).*

The Moving Pictures Experts Group (MPEG) [Chi15] first addressed these issues at their 109th meeting in Sapporo, Japan, in July 2014 [MAT+14]. MPEG issued first draft requirements for genome compression and storage at that meeting[2]. These requirements are being refined until now and are being further discussed in Section 3.3.

With a history as a standardisation body for the coding of digital audio and video data, MPEG focuses on the standardisation of storage and analysis processes for genomic data. NGS methods produce a huge amount of data, which is still stored in human-readable text files in various file formats such as FASTQ or SAM (the

[1]In this context, one kbase ("kilobase") refers to 1,000 bases.
[2]Please refer to the MPEG output document N14553 [MAT+14] for the draft requirements.

most common file formats for genomic information representation have been discussed in Section 2.4). Currently, these text files are compressed[3] e.g. using well-known dictionary-based compression algorithms from the Lempel-Ziv algorithm family (known as universal compression algorithms) [ZL77]. Due to the special structure of genomic information data (mainly sequence reads and quality scores), more sophisticated compression approaches promise high feasibility and should be investigated.

Before inducting more detailed explanations of the processes involved, an important distinction has to be made. The term "genome compression" denotes the process of encoding the entire genetic information of an individual organism. Concerning humans, the genetic information (from the perspective of a computer scientist) is contained in a string with a length of approximately 3.2 billion symbols. This string can be much longer for other organisms (up to more than 100 billion symbols) [AMC+14]. The topic this thesis is focusing on is more specifically referred to as the compression of sequence reads. This is the type of information directly produced in large volumes by high-throughput (NGS) machines.

3.1.1. Genomic Life Cycle

Sequencing is the process of expressing genomic information (more precise: the nucleotide sequence in a DNA or RNA molecule) usually as a sequence of characters (also known as string). Most sequencing devices output a string containing the nucleotide identifier, and a string containing the quality scores (one score for each nucleotide identifier), expressing the base-call accuracy.

During the process of alignment, the reads are arranged in order to reconstruct the donor genome (or parts of the donor genome). This can either be done by evaluating the similarities among different reads or with respect to an external (pre-constructed) DNA sequence. The reference-based alignment is also called *mapping*. Sequence assembly is the process of merging aligned fragments to reconstruct the (complete) original sequence. If an external reference sequence is used, the process is called *de-novo assembly*.

Figure 3.2 shows the so-called *genomic life cycle* and gives a brief overview about the steps performed on the way from DNA sequencing via assembly/alignment to analysis.

The majority of analysis programs available today depend on textual input. Thus, repeated compression and decompression is needed concerning the interface between storage and analysis. Loh et al. [LBB12] propose to overcome this inefficiency by "compressing sequence data in such a way as to allow direct computation on the

[3] As discussed in Section 2.4 the de-facto standards are either gzip for FASTQ files or BGZF for SAM files.

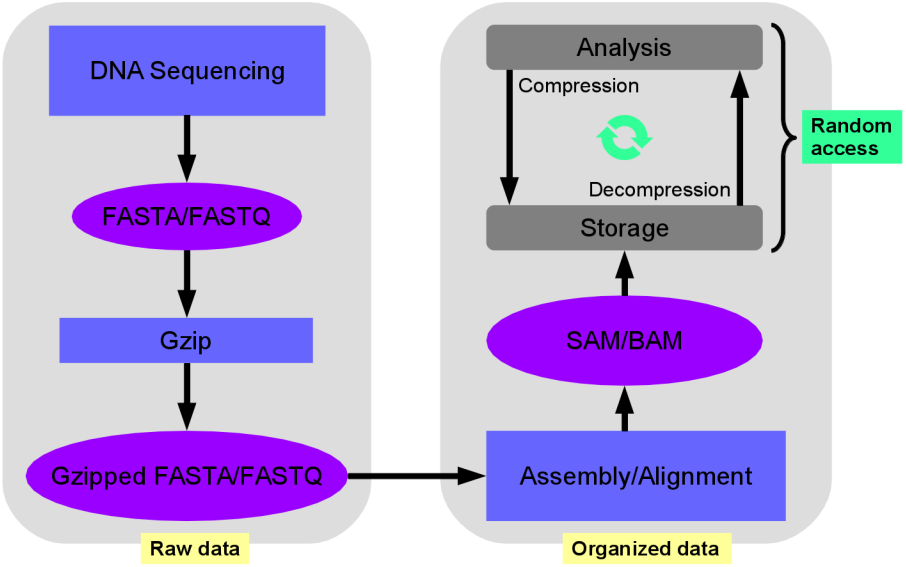


Figure 3.2.: *The* genomic life cycle *summarises the vital steps from DNA sequencing via assembly/alignment to analysis. Rectangular boxes denote processing steps; eclipses indicate file formats. The most important steps in this chart - storage and analysis - are denoted by rounded rectangular boxes. For the repeated steps of storage, decompression, analysis, and compression, random access to the compressed data is a major requirement.*

compressed data". They call this method "compressive genomics". This approach would require highly standardised processing and analysis pipelines and is only feasible in some special use cases.

Another approach is to design genomic information codecs in such a way, that efficient random access to the compressed data is possible. This could efficiently speed up the analysis of huge amounts of genomic information, as compression and decompression could be performed "on the fly" during analysis processes. This random access feature is of particular importance. A short practical example shall provide the rationale: When having compressed a 1.5 TiB FASTQ file with gzip [GA15], decompressing the `*.fq.gz` file takes about nine hours. Thus, due to the typically huge file sizes, random access is a key feature for any codec that might be successfully adopted by the scientific community.

Regarding the compression of genomic information data, another important discussion from the scientific community shall be mentioned here. As mentioned in Section 5.2, various efforts for the structured sequencing of human genomes and other genomes (for example from Theobroma cacao [ASA+11]) have been established over the last years. For the assembly of the sequence reads, in most cases, a reference genome is used. The same reference genome can be used for the compression of

sequence reads (and the associated metadata). However, each individual organism bears genetic differences, known as *single nucleotide variants* (SNV). Reference-based compression algorithms show a good performance but obviously have the requirement for a reference genome. The question arises, if equal compression ratios can be achieved with non-reference-based compression.

3.2. Standardization Efforts

As already mentioned, the Moving Pictures Experts Group (MPEG [Chi15]) took interest in the topic of genome compression during 2014. The MPEG members established a so-called *ad-hoc group* (AhG) at their 109th meeting in Sapporo, Japan, in July 2014. I collaborated in the standardisation effort since February 2015 by attending various teleconferences. Furthermore, I participated in the 112th MPEG meeting in Warsaw and compiled and presented an input document[4] to this meeting.

The main outcome of the AhG activities are (until now) the following:

- The AhG compiled a *genomic information database*[5] (GIDB) for the evaluation of different compression approaches.
- So-called *requirements* for genome compression have been issued. These requirements are discussed in detail in Section 3.3.
- The AhG started the evaluation of existing compression approaches. This shall lead to a *Call for Evidence* (CfE) and a subsequent *Call for Proposals* (CfP). This is the usual workflow within MPEG when compiling a novel standard or series of standards.

Apart from MPEG, other initiatives have been started, of course. The MPEG output document "White paper on genomic information compression and storage" [AMC+14] lists some of them. ISO initiatives mentioned are: the ISO Technical Committee (TC) 215 (Health Informatics) and the ISO TC 276 (Biotechnology). However, these ISO initiatives do not specifically address the issues of efficient data compression and the requirements having evolved with the rise of NGS technologies.

An non-ISO initiative that be mentioned here is the Pistoia Alliance Inc., which is a consortium of pharmaceutical industries, universities and research centres. Specifically, this initiative promoted the *SequenceSqueeze contest*[6] in 2011, which "aimed to spur development of new and novel methods of compressing sequence reads and their quality scores in a way that preserves 100% of the information whilst achieving much-improved linear (or, even better, non-linear) compression ratios". The winner

[4] Input document number: m36282 ("Approaches to SAM File Compression").
[5] An overview about the database is presented in Appendix D.
[6] `http://www.pistoiaalliance.org/projects/sequence-squeeze/`

of the contest was James Bonfield, an engineer from the Wellcome Sanger Institute, who is contributing to the MPEG AhG on genome compression.

Another initiative mentioned by Alberti et al. [AMC!15], is the Global Alliance for Genomics and Health[7]. The alliance brought four working groups into being, with one of them, the "Data Working Group", focusing on topics like file formats, benchmarking, containers and workflows, and many more.

As an outcome of the 112th MPEG meeting in Warsaw, the AhG on genome compression announced a "Seminar on Prospects on Genome Compression" [AM15] which will be held at the 113th meeting in October 2015. Speakers will be invited from several genomic research institutions, sequence machine manufacturers and cloud storage operators to get in touch with each other and get more input to the requirements for genome compression. Main topics will be: "new approaches, tools and algorithms to compress genome sequence data, objectives and issues of quality scores compression and impact on downstream analysis applications, new approaches to quality scores definition and processing, genome compression and genomic medicine applications" [AM15].

3.3. Requirements for Genomic Information Compression

MPEG issued a document[8] containing requirements for genome compression and storage at their 109th meeting [MAT+14]. These requirements are frankly a list of features that any future standard should contain. Table 3.1 is taken from the MPEG document N14553 [MAT+14] and shows the general requirements.

Furthermore, the MPEG document N14553 contains data security-related requirements, as sequenced (human) genomes contain highly private data worth protecting[9]. This is of course an important issue, but not further discussed in the scope of this report.

The MPEG document N15093 ("Requirements on Genome Compression and Storage") lists further requirements, separated as requirements for unmapped reads (as stored in FASTQ files) and as requirements for mapped respectively aligned reads (usually store in SAM files). Table 3.2 lists the most important requirements for unmapped reads.

For the compression of mapped reads, the AhG developed additional requirements

[7] `http://genomicsandhealth.org/`

[8] MPEG output document number N14553.

[9] For example, there are at least two major susceptibility genes for breast cancer. Inherited mutations in these genes lead to a high risk of e.g. breast cancer [EPD+07]. This information should only be of private concern.

ID	Requirement	Rationale
1	Maintain the data integrity produced by the sequencing technology.	It shall be possible to revert to the original representation of data (or equivalent) that was produced by the sequencing devices.
2	Compressed data should be structured into data access units.	Access to data shall work in terms of access units (blocks) that be accessed without the need to decompress the whole genomic information.
3	Access units should be able to contain variable size sequence lengths.	Sequences lengths heavily vary with the sequencing device and will keep changing with new sequencing technologies going to the market.
4	Data should be "queryable".	The data set structured according to the standard shall be accessible using queries such as: "search for the first match for a given sample" or "search all occurrences of a sample".
5	Stored sequence data should be extensible allowing adding or modifying sequencing information. Such modifications should be traceable, reversible and incremental.	An existing dataset shall support update, revert, history, and diff operations. For each operation support for integrity and correctness checks is recommended.
6	Compressed data formats should be able to integrate annotations - e.g. features on a given region, non-contiguous (this is the most important of all) regions of the data structure.	Annotations are metadata that are used for example to indicate relations between reads located at different spatial locations.
7	The nucleotide information shall support at least the 16 IUPAC codes but shall be able to encode more in the future.	This is required to support the coding of other genomic data such as amino-acids. For example basic "profiles" could support only the 16 IUPAC codes while extended ones would be able to additionally encode the 23 amino-acid codes.

Table 3.1.: *This table shows the general requirements for genome compression. The contents are taken from the MPEG output document N14553 [MAT+14]. In some cases, the rationales have been shortened.*

for compression. These are listed in Table 3.3.

ID	Requirement for compression
1	Lossless compression of read headers
2	Lossless compression of nucleotide sequences
3	Both lossless and lossy compression of quality scores
4	Compressed data shall be structured so that efficient random access and parallel processing are enabled and compression efficiency is preserved.
5	If the sequencing process produced paired-end or mate-reads, then pairing information shall be preserved.
6	Preserve the association among read header, nucleotide sequence, and quality scores.
7	The compression process shall support the assessment of integrity.

Table 3.2.: *The MPEG AhG compiled a list of requirements for unmapped reads [AMC*15]. This table shows the most important requirements in the scope of this thesis (some requirements have been shortened respectively altered and their rationales have been omitted in this table).*

ID	Requirement for compression
1	If a reference genome is used, it shall be a resource unambiguously identified.
2	Both reference based and non-reference based alignment shall be supported.
3	From aligned/mapped reads, it shall be possible to reconstruct the original unmapped reads, including pair information if present.
4	Access to indels (polymorphism that corresponds to the addition or removal of a number of bases in a DNA sequence) and differences with respect to the reference genome must be possible for selected segments of the alignment.
5	Uniquely identify each aligned read (a.k.a. alignment) and its mate pair if applicable.
6	Lossless compression of attributes associated to each read
7	Lossless compression of mapping/alignment information.
8	Support mapping quality expressed as “phred-scaled posterior probability that the mapping position” is wrong.
9	Optionally support the encoding of additional information structured as key-value pairs.

Table 3.3.: *Additional requirements for aligned data. The contents of the table taken from Alberti et al. [AMC*15] (some requirements have been shortened respectively altered and their rationales have been omitted in this table).*

3.4. Overview about Genome Compression

Over the past years, and advancing until now, numerous genomic information compression tools have been developed by industries and the scientific community. Dif-

ferent methods have been used in the currently available tools. However, comparing these different solutions is a difficult task. Actual implementations might not be available, a common test data set has not yet been defined, and output formats are proprietary or vary among the mentioned tools. The following items give an overview about the general approaches from the field of source coding used for genomic information compression. Generally speaking, there are four principal approaches to genomic information compression (specifically regarding the field of sequence read compression).

- **Naive bit encoding:** This simple approach is sometimes still in use. For a message composed using an alphabet containing the $K = 4$ textual symbols "A", "C", "G", and "T", this method stores a series of four symbols (each encoded with 2 bits) in one byte, yielding a compression ratio of 25%. However, this approach does not eliminate any redundancy that may be present in the data structure, and is only an optimal entropy coding scheme if the four symbol probabilities were equally distributed (i.e. each of the symbols would occur with a probability of $p = 1/K = 1/4$).
- **Dictionary-based:** These methods build up a dictionary of previously seen substrings during compression. Future occurrences of these substrings are then encoded as a reference to the dictionary. The dictionary can be either static or re-built during decompression.
- **Statistical methods**: These methods analyse the statistics and correlations in the input data and try to predict future symbols based on the constructed source model (which might be for example a Markov chain). In a second step, entropy coding algorithms (for example AC) are employed. The prediction process is computationally expensive but yields the best overall compression ratios.
- **Referential methods**: These methods encode positions and deviations with respect to some reference genome. Reference-based genome compression has e.g. been evaluated by Chern et al. [COM+12].

Recent genomic information compression tools can roughly be categorised in two classes: Either the tools compress raw reads (in the FASTQ format), or they compress aligned reads (provided in form of the SAM format). As currently the de-facto standard for compressing FASTQ files is gzip, respectively SAMtools (BAM) for SAM files, the primary goal of those tools is to replace the aforementioned. To provide an overview about existing tools, some of them are shown in Table 3.4 (all of them participated in the SequenceSqueeze competition).

The usual approach to FASTQ compression is to first separate the read headers, the nucleotide sequences, and the quality scores into three separate streams[10]. Then,

[10]The FASTQ description (third line in a record) is very rarely used and omitted in the majority of compression programs.

Tool Name	URL	Input Format
CRAM toolkit	cram-toolkit	BAM
DeeZ	deez	SAM/BAM
Goby	goby	FASTQ
Quip	quip	FASTQ, SAM/BAM
SAMtools	samtools	SAM/BAM
sam_comp	samcomp	SAM/BAM
SCALCE	scalce	FASTQ

Table 3.4.: *This table shows a collection of recent compression tools. All of them participated in the SequenceSqueeze competition. SAMtools is the software used for the en- and decoding of BAM files.*

they are compressed independently from each other. The key issue here is to find the best trade-off between complexity and efficiency. Similarly, the first step when compressing SAM files is to separate the records into twelve distinct streams. In a typical SAM file there are - in contrast to a typical FASTQ file - correlations between some fields, due to the alignment process and the coverage introduced by NGS sequencing technologies. The key issue here is to exploit intra- and inter-read dependencies.

Two tools from Table 3.4 shall be investigated in more detail here. Quip [JRPK12] is a tool mainly focusing on FASTQ compression. For the nucleotide sequences, it employs high-order Markov chains. "The nucleotide at a given position in a read is predicted using the preceding 12 positions" [JRPK12]. The drawback of this approach is that is requires a huge amount of data in order to train the model. Regarding the compression of quality score, the authors of Quip employ an order-3 Markov chain. However, due to the large alphabet of quality scores, they "bin the distal two positions", i.e. they quantize the quality scores used for prediction.

DeeZ [HNS14] "uses a unique compression method for each field of the SAM record in order to exploit its specific properties: read names are 'tokenized' and compressed by the use of delta encoding; quality scores are encoded using an order-2 arithmetic coder, etc.". In particular, the authors of DeeZ implemented a nucleotide sequence compressor which is based on "the observation that the vast majority of the nucleotide differences between each read and its mapping locus on the reference are shared with other reads mapped to the same locus" [HNS14]. The main characteristics (omitting everything else but the compression approaches for nucleotide sequence respectively quality score compression) of these two tools are compared in Table 3.5. Bonfield et al. [BM13] provide an overview incorporating some of the other tools mentioned in Table 3.4.

Tool	**RA**	**Sequence compression**	**Quality score compression**
Quip	No	Huge Markov model	Modified ("binned") order-3 prediction + AC
DeeZ	Yes	"Local Assembly"	Order-2 prediction + AC

Table 3.5.: *A comparison of the features of Quip and DeeZ as representatives of latest generation genomic information compression tools.*

4. Novel Approaches to Genome Compression

First, this section describes a compressor for nucleotide sequences which assumes aligned (i.e. position-sorted) data[1]. The "nucleotide compressor" combines the mapping positions (SAM field 4), the CIGAR strings (field 6), and the actual nucleotide sequences (field 10) to implicitly assemble local parts of the donor genome in order to compress the sequence reads. Aligning is done by programs called "aligners" or "multiple sequence aligners" as pointed out in Section 2.3.5. Due to the usually high coverage inherent in most NGS technologies, the process of alignment produces an output (i.e. sorted reads) that makes the redundancy introduced by the high coverage exploitable for compression. In contrast to other sequence compressors, the proposed compressor does not need a reference genome/sequence and is able to perform compression "on-the-fly" using solely a sliding window (i.e. a permanently updated short-time memory of arbitrary size) as context for the local implicit assembly.

Second, regarding the compression of quality scores, several predictive compression schemes are proposed (based on Markov chains) with a variable or fixed context to boost prediction performance. Specifically, as the prediction residues tend to fall into a two-sided geometric distribution, the entropy coding of quality scores could reasonably well be performed with a Rice code.

4.1. Statistical Analysis

For the statistical analysis of genomic information data, I developed a small program (written in C++), called "samsplit". A description of samsplit and its usage can be found in Appendix A. In short, samsplit takes a SAM file as input and separates the different fields/columns. The analysis has been mainly performed using various self-written MATLAB scripts. This software is shipped with this report and can be found within the samsplit repository in the subfolder `matlab/`.

[1] The data has to be sorted by SAM field 4, containing the mapping position. Please refer to Section 2.4.3 for a description of the SAM file format.

4.1.1. Read Entropy

First of all, to get an initial impression of the statistical properties of the data, in this case the nucleotide sequences, samsplit has been used to extract the sequence reads from various SAM files. The SAM files have been taken from the MPEG GIDB and are denoted with the file identifiers defined in Section 5.2. Subsequently, the entropy of the sequence reads has been computed[2]. The results are displayed in Table 4.1.

File	**Entropy** [$^{\text{bit}}/_{\text{symbol}}$]
e88HCC	2.0039
598HCC	2.0010
HCCmix	1.9975
DH10B	1.9982
9799#3	1.9981
K562	1.9945
9827#49	1.9996
NA12877	1.9796
NA12878	1.9791
NA12882	1.9824
NA12879	1.9556
NA12878	1.9595
NA12890	1.9907
NAchrom11	1.9969
sample-12	1.9421
sample-11	1.9775
sample-10	1.9658
Average entropy	1.9836

Table 4.1.: *The first 100,000 nucleotide sequences have been extracted from each file and their entropy was computed.*

The first two files have an entropy which is greater than 2 $^{bit}/_{symbol}$. These files thus have an alphabet at least containing the ASCII character "N" as a fifth character, as pointed out in Sections 2.2 and 2.3.4. Nevertheless, all files seem to have a reasonably equal frequency distribution among the four mainly used symbols ("A", "C", "G", and "T"). Due to this property, sophisticated entropy coding schemes would "overstep the mark". A simple two-bit encoding scheme as presented in Section 3.4 would be an adequate choice. However, as pointed out e.g. by Jones et al. [JRPK12], there is a predictive relationship among subsequent symbols that can be exploited to further reduce the entropy. Nevertheless, it should be kept

[2]Please refer to the following MATLAB code for this investigation: `test_environment.m`, `seq_entropy.m`, `tnt_read_ascii.m`, `tnt_entropy.m`.

in mind that SAM files contain another type of redundancy, due to the coverage mentioned in the introduction of this chapter. Thus, in general, prior to any entropy coding (which would yield an entropy of approximately 2 $^{bit}/_{symbol}$), there has to be a predictive coding step trying to eliminate the present redundancy.

The next section investigates how far well-known dictionary-based compression algorithms are able to predict and compress the nucleotide sequences.

4.1.2. Compression of Sequence Reads Using Universal Coding Algorithms

As pointed out in Section 2.2.3.4, dictionary-based coding algorithms are capable of removing redundancy from sources being afflicted with some kind of "memory", i.e. a predictive relationship. Three algorithms of this kind have been tested: gzip, bzip2, and lzma.

4.1.2.1. Gzip

Gzip is based on the so-called "DEFLATE" algorithm, which is in turn a combination of the LZ77 algorithm and Huffman coding [GA15]. The results[3] in Table 4.2 have been obtained using gzip version 1.6 on a machine with an Intel® Core™ 2 Quad CPU Q9650 @ 3.00 GHz and 8 GB main memory, running openSUSE 13.1 (x86_64), code name "Bottle". Timing measurements have been done with GNU time[4] 1.7.

The obtained compression ratios reach from 16.89% (`NA12878` and `NA12890`) up to 29.72% for `e88HCC`. For the following observations, a representative subset of the data has been used (`e88HCC`, `DH10B`, `K562`, `NA12877`, `NA12890`, `sample-11`).

4.1.2.2. bzip2

Bzip2 is based on the Burrows-Wheeler transform (BWT), combined with a move-to-front transform and Huffman coding. The results in Table 4.3 have been obtained using bzip2 version 1.0.6 on the same machine as in Section 4.1.2.1.

[3] In this and the following tables, t_c denotes the compression time and t_d denotes the decompression time. CR denotes the compression ratio, defined as $CR = {}^{CompressedSize}/_{UncompressedSize}$.
[4] Taking the "User time".

File ID	Size[B]	t_c [s]	Comp. size [B]	CR	t_d [s]
e88HCC	437936	17.36	130196	29.72%	2.52
598HCC	439956	17.67	129524	29.44%	2.51
HCCmix	441252	17.66	127312	28.85%	2.51
DH10B	411504	13.46	99860	24.26%	2.17
9799#3	368580	11.51	73856	20.03%	1.68
K562	31612	0.79	5296	20.03%	0.16
9827#49	363732	17.10	107928	29.67%	2.00
NA12877	351548	12.61	80952	23.02%	1.74
NA12878	348148	11.26	73444	21.09%	1.63
NA12882	349692	10.89	70516	20.16%	1.63
NA12879	364584	9.06	63844	17.51%	1.55
NA12878	363652	9.04	61444	16.89%	1.58
NA12890	364712	9.07	61636	16.89%	1.59
NAchrom11	432040	10.77	93744	21.69%	2.08
sample-12	336944	8.58	77604	23.03%	1.64
sample-11	338252	8.69	78804	23.29%	1.69
sample-10	337048	8.59	78684	23.34%	1.70

Table 4.2.: *The first 100,000 nucleotide sequence reads have been extracted from each file and they have been compressed using gzip.*

File ID	t_c [s]	Comp. size [B]	CR	t_d [s]
e88HCC	38.39	109588	25.02%	8.58
DH10B	41.28	80068	19.45%	7.76
K562	3.22	4464	12.89%	0.52
NA12877	34.62	64460	18.33%	7.18
NA12890	48.15	47400	12.99%	5.46
sample-11	43.40	63868	18.88%	6.08

Table 4.3.: *The first 100,000 nucleotide sequence reads have been extracted from each file and they have been compressed using bzip2.*

4.1.2.3. lzma

The Lempel-Ziv-Markov chain algorithm (LZMA) used an altered LZ77 algorithm in combination with Markov chain-based prediction and a range coder[5]. The results in Table 4.4 have been obtained using liblzma version 5.0.5.

A comparison can be viewed in Figure 4.1. Bzip2 yields the overall best compression results, whereas in general gzip has the fastest decompression. However, these compression algorithms are only capable of exploiting redundancy along the nucleotide

[5]Simply speaking, a range coder can be thought of as an arithmetic coder.

File ID	t_c [s]	Comp. size [B]	CR	t_d [s]
e88HCC	215.27	106828	24.39%	6.24
DH10B	178.78	81716	19.85%	4.55
K562	10.27	4296	13.58%	0.29
NA12877	154.04	64684	18.39%	3.67
NA12890	121.86	51036	13.99%	3.14
sample-11	117.26	67736	20.02%	3.72

Table 4.4.: *The first 100,000 nucleotide sequence reads have been extracted from each file and they have been compressed using LZMA.*

sequence. Direct correlations among subsequent records are not detected.

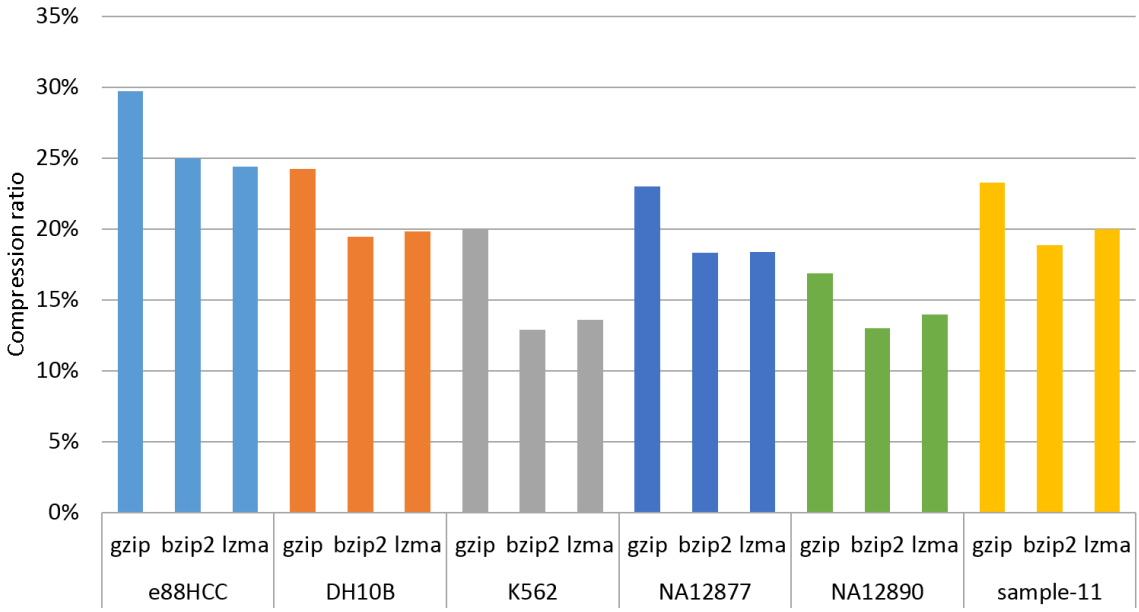


Figure 4.1.: *The performances of gzip, bzip2, and lzma as nucleotide sequence compressors are compared.*

4.1.3. Read Positions

A prerequisite for the proposed nucleotide sequence compressor is that the reads in a SAM file are sorted by their position. Figure 4.2 shows the positions of the first 100,000 reads of all files from the GIDB[6].

[6]Please refer to the MATLAB script `pos_visualization.m` from the SAMSPLIT software for this experiment. See Appendix A.

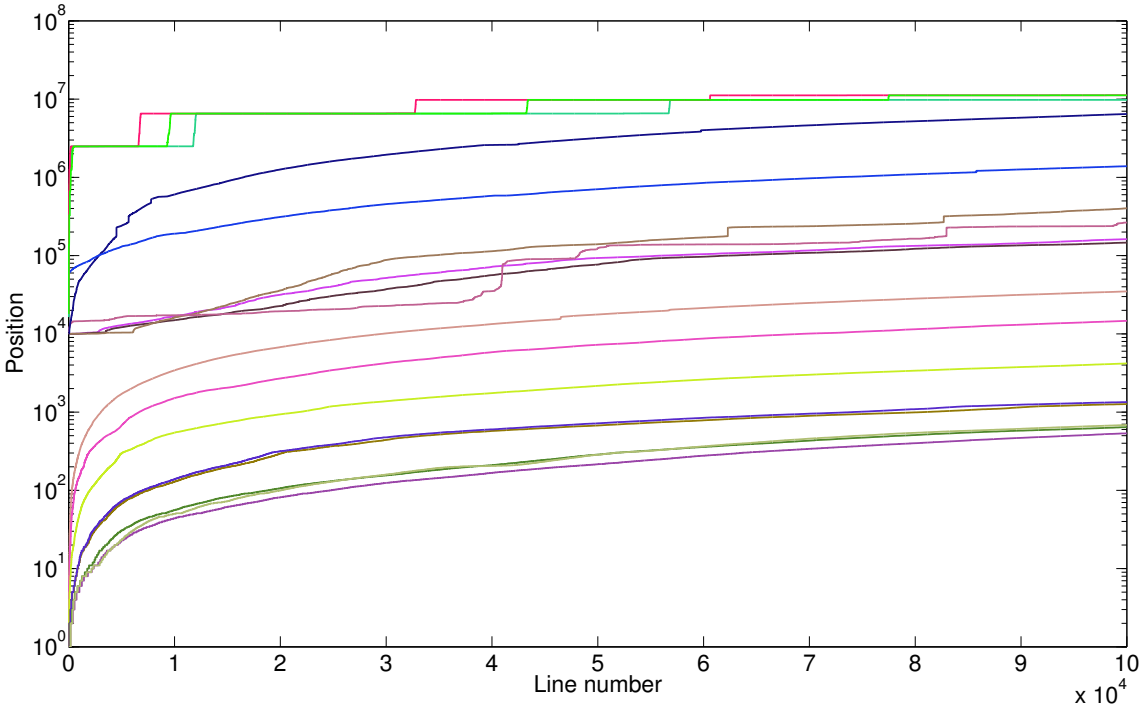


Figure 4.2.: *Positions of the first 100,000 reads for all files from the Genomic Information Database (GIDB). All reads are sorted by their position. Some samples show regions with a high slope which is identical to a low coverage in these regions. These regions might be difficult to compress with the proposed sequence compressor.*

4.1.4. Transform Coding for Illumina Reads

During the first work on sequence compression in the scope of this thesis, data from an Illumina machine has been evaluated. Due to the specific technology employed by Illumina machines (see Section 2.3.2), the produced sequence reads have a fixed length and a block of reads can be plotted as an image, as shown in Figure 4.3. The horizontal "cut" in Figure 4.3 represents a shift in the mapping position. "Dots" sticking out of the vertical lines are either errors produced by the sequencing machine or can be accounted on mapping errors.

The result of applying an 8×8 DCT on the (image) data is illustrated in Figure 4.4. The DCT coefficients clearly visualise the vertical correlation between consecutive reads and furthermore show, that read errors produce energy in higher frequency domains. Nevertheless, the two figures imply that compressing the nucleotide sequences in the "sequence space" rather than some "transform space" should be more beneficial.

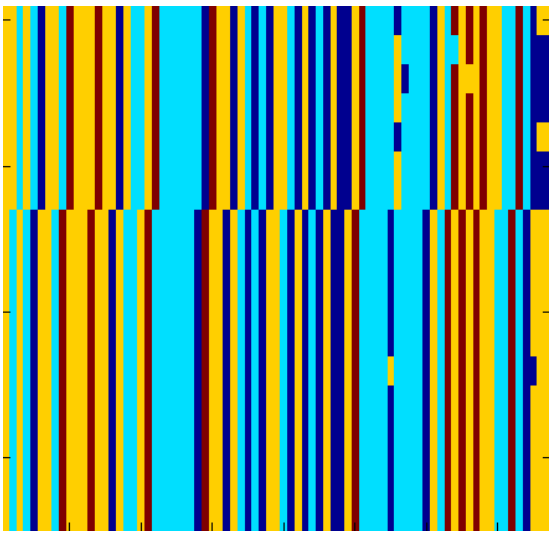


Figure 4.3.: *A block of sequence reads produced by an Illumina machine with a fixed length plotted as an image. The reads are plotted horizontally from top to bottom. The four colours (yellow, red, dark blue, light blue) represent the four nucleotides.*

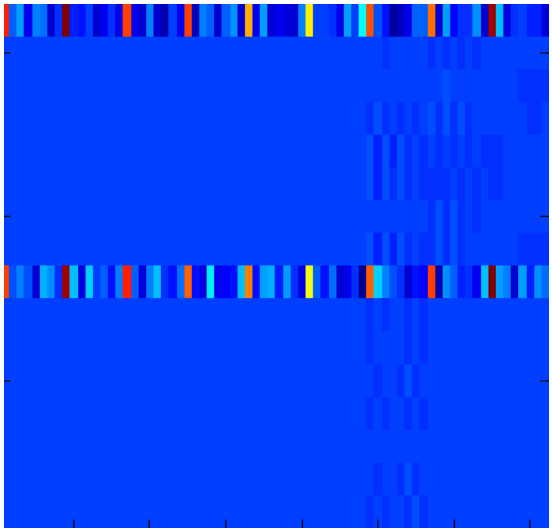


Figure 4.4.: *DCT coefficients produced by an 8×8 DCT. Sequencing errors respectively indels produce energy in higher frequency domains.*

4.1.5. Quality Scores

As for the sequence reads in Section 4.1.1, the entropy for the quality scores from all SAM files from the GIDB has been computed, too. The results are displayed in Table 4.5.

First of all, the quality scores have a much higher entropy. Again, the question arises, if the quality scores show some kind of correlation. This would enable the building prediction schemes. In order to investigate possible correlations within single quality score lines, a single quality score line has been randomly chosen from each file and the auto-correlation function has been computed. The results are displayed in Figure 4.5.

The auto-correlation functions are rapidly decreasing, but nevertheless do not resemble delta-impulses. This can be interpreted as that a single quality score value q_i at position i is correlated with the immediate preceding quality scores $q_{i-1}, q_{i-2}, \ldots,$ decreasing the further back we go. Thus, a MARKOV chain of some low order[7] N

[7]Because of the large quality score alphabet, only relatively small memory sizes are applicable.

File	Entropy [bit/symbol]
e88HCC	3.5517
598HCC	3.4679
HCCmix	3.4892
DH10B	4.1438
9799#3	3.2093
K562	3.5157
9827#49	4.0730
NA12877	3.9344
NA12878	4.0084
NA12882	3.8231
NA12879	4.0189
NA12878	3.9945
NA12890	3.9300
NAchrom11	4.3409
sample-12	4.5896
sample-11	4.6094
sample-10	4.6015
Average entropy	3.9589

Table 4.5.: *The first 100,000 quality score lines have been extracted from each file and their entropy was computed.*

(assuming correlations with N previous quality scores) should be suitable for the from now on so-called *intra-line prediction*.

To investigate possible *inter-line* dependencies, the first 200 quality score lines from each file from the GIDB have been separated. Subsequently, the quality scores have been written row-wise to a single stream for each file. Then, the auto-correlation functions of all the signals have been computed[8]. The resulting auto-correlation functions can be viewed in Figure 4.6.

The periodic peaks in the auto-correlation functions, approximately at a lag of the length of a quality score line imply, that different quality score lines in a block are in some way "similar". To verify, that modelling the quality score source with a Markov chain is generally suitable, a MATLAB simulation has been developed. A Markov chain with a memory holding the $N = 2$ previous quality scores was employed. The current symbol was predicted using a maximum-likelihood predictor, as presented in Section 2.2.4. For each file from the GIDB, the first 25,000 quality score lines have been processed. This should suffice to train the Markov model. Table 4.6 shows the achieved entropy reductions.

[8]To compute the auto-correlation functions of the signals in this section, the MATLAB function `xcorr` has been used.

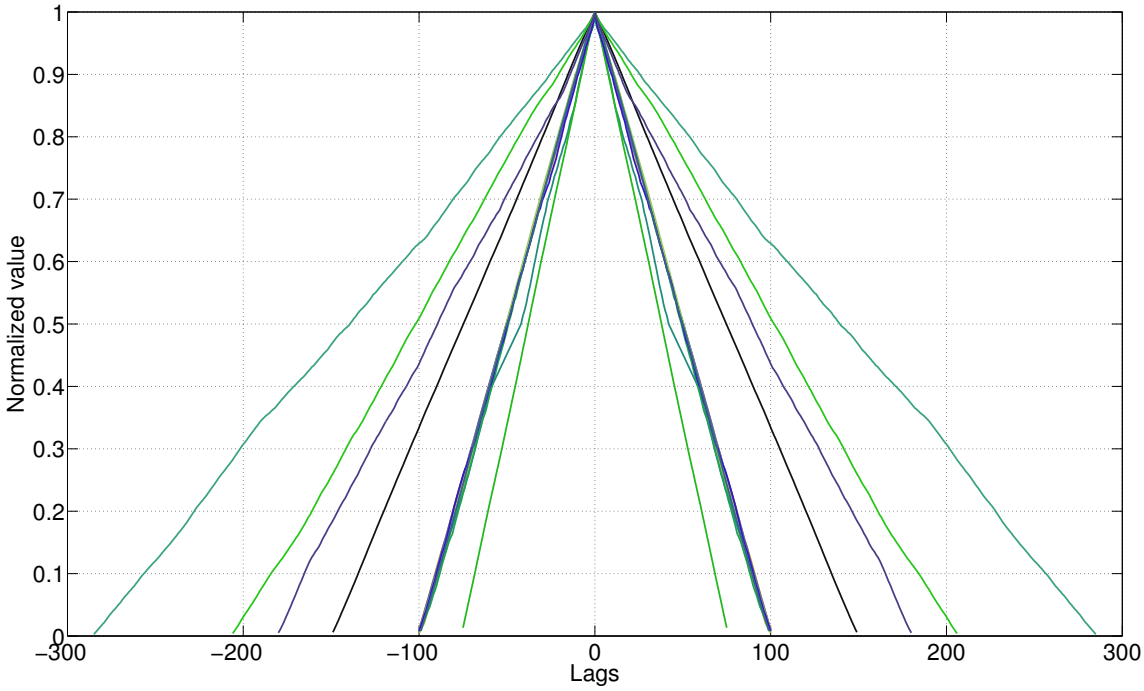


Figure 4.5.: *Auto-correlation functions for randomly chosen quality score lines for each file from the Genomic Information Database.*

For the files with the identifiers `9799#3` and `9827#49` no entropy reduction was achieved. This could either be due to the statistics of the quality scores themselves or due to the relatively small amount of training data (only 25,000 quality score lines). Nevertheless, on average, the entropy could be reduced by approximately 0.28 $^{bit}/_{symbol}$.

4.2. Block-based Compression of Nucleotide Sequences

Current nucleotide compressor implementations (e.g. Quip [JRPK12] and sam_comp [BM13]) use prediction schemes combined with subsequent arithmetic coding to compress the nucleotide sequences. In order to exploit the redundancy present in the data, prediction models such as the maximum-likelihood predictor need *a priori* knowledge (in order to train the prediction model). This practically excludes random access to the compressed data, as the prediction performance increases with the amount of data the prediction model can be trained on.

The authors of DeeZ [HNS14] made a new approach, which they call "local assembly". Their scheme is based on "the of the observation that the vast majority of the

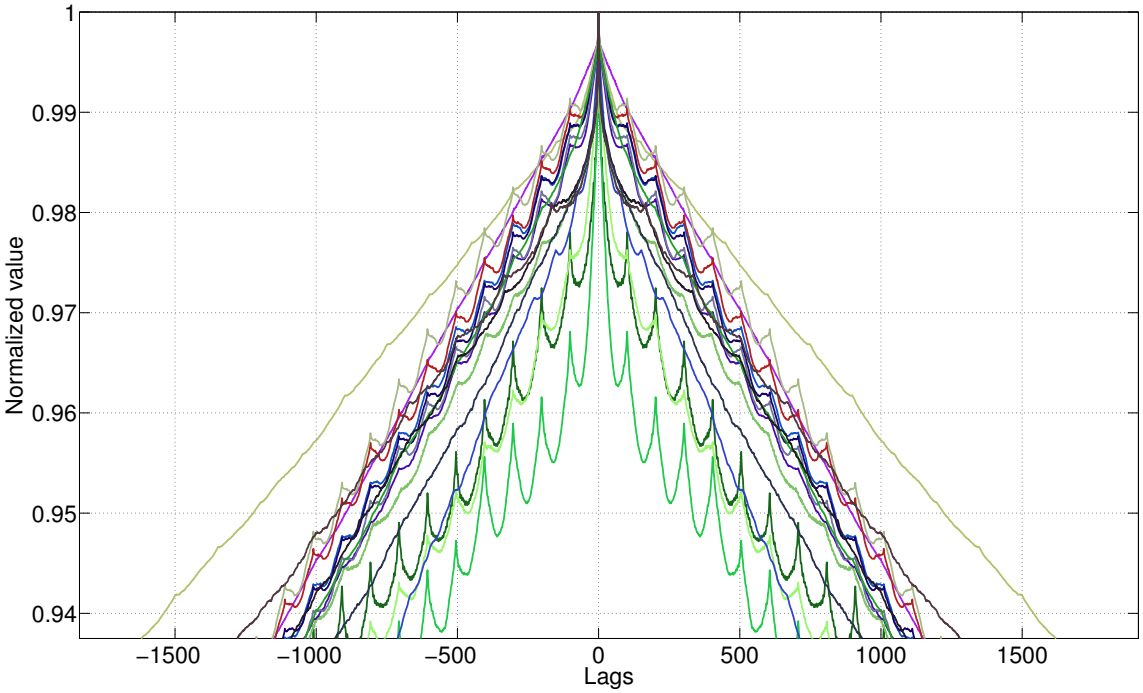


Figure 4.6.: *Auto-correlation functions for the first 200 quality score lines for each file from the GIDB. Image has been zoomed in.*

nucleotide differences between each read and its mapping locus on the reference are shared with other reads mapped to the same locus". DeeZ obtains a "consensus" of the reads mapping to a specific locus and encodes all reads with respect to this consensus. DeeZ nevertheless performs the local assembly with respect to an external reference (given in FASTA format). Furthermore, obtaining the full consensus is computationally expensive.

The authors of Quip [JRPK12] implemented an assembly-based compression scheme for nucleotide sequences, too. They use by default the first 2.5 million reads to assemble so-called "contigs", which are then used in place of a reference sequence to encode aligned reads. This algorithm does not require a reference – this means that the resulting compressed files are entirely self-contained – but has the drawback that the best match of a specific read to the previously assembled reference has to be found in the fixed non-adaptive contig repository.

4.2.1. Proposed Compression Mechanism

In this section, a compression scheme using a sliding window to perform a "local block-based implicit assembly" is proposed. In contrast to DeeZ, the compressor does not require a reference genome and is able to work on smaller block granularities

File ID	$\mathbf{H}(\mathbf{Q})$ [bit/symbol]	$\mathbf{H}(\hat{\mathbf{Q}})$ [bit/symbol]	$\mathbf{H}(\mathbf{Q}) - \mathbf{H}(\hat{\mathbf{Q}})$ [bit/symbol]
e88HCC	3.7150	3.5146	0.2004
598HCC	3.6203	3.3679	0.2524
HCCmix	3.5910	3.2229	0.3681
DH10B	4.2428	3.8133	0.4295
9799#3	3.1990	3.2645	-0.0655
K562	3.6866	3.6519	0.0347
9827#49	4.0281	4.2134	-0.1853
NA12877	3.9975	3.9603	0.0372
NA12878	4.0285	4.0040	0.0245
NA12882	3.8530	3.7398	0.1132
NA12879	3.8684	3.7181	0.1503
NA12878	3.8279	3.6081	0.2198
NA12890	3.7790	3.5247	0.2543
NAchrom11	4.2597	3.9589	0.3008
sample-12	4.5804	3.7225	0.8579
sample-11	4.6362	3.7453	0.8909
sample-10	4.6226	3.7449	0.8777
Average	3.9727	3.6926	0.2800

Table 4.6.: *The first 25,000 quality score lines have been predicted with a 2nd-order* Markov *model.*

as it uses a sliding window as context (of course smaller block sizes have a negative impact on the compression ratio). In contrast to Quip, the compressor does not need any reads to assemble "contigs" in advance. The compression is performed using only the currently available nucleotide sequences in the sliding window. The compression scheme therefore is less complex.

The approach is based on correlations between consecutive reads. All reads in a properly aligned SAM file are aligned to their mapping position. Thus, all reads mapping to the same position should be similar, except for sequencing respectively mapping errors or highly allelic regions. These aberrations would be coded in the CIGAR string. Figure 4.7 and Figure 4.8 illustrate the structure of mapped sequence reads. Consecutive reads are plotted line by line. If SAM files are sorted by mapping position, a read mapping to a different position than the read before leads to a step in the plot. If plotted over a large block, this leads to the "staircase" property visible in the plotted figure.

The proposed compression scheme encodes sequence reads block-wise exploiting the joint coding of the mapping positions, the CIGAR strings and the sequence reads itself, whereas the block size might be fixed or variable and of arbitrary size (which naturally includes the compression of the input file as a single block). The first

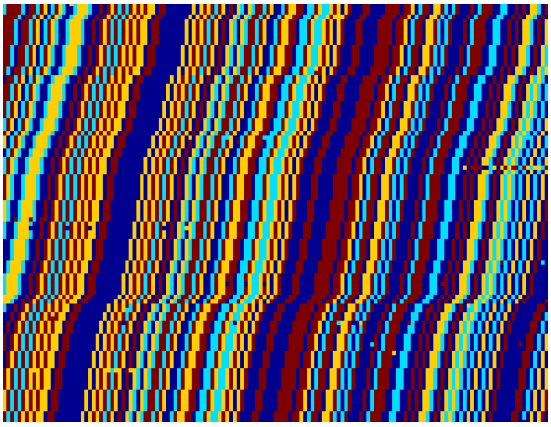


Figure 4.7.: *Consecutive nucleotide sequences from a SAM file. The different colours represent the nucleotides.*

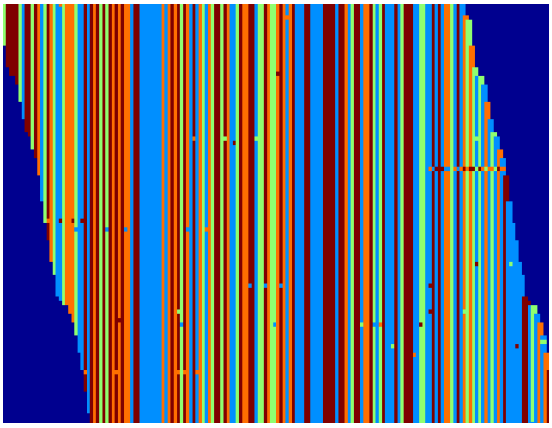


Figure 4.8.: *The nucleotide sequences after the shift according to their mapping position. The noisy "dots" represent genome mutations or sequencing errors described in the CIGAR string.*

sequence read seq_1 in a block is encoded without prediction, as well as the corresponding mapping position and the CIGAR string. A subsequent read $seq_i, i > 1$ is then aligned to a previous read $seq_j, j < i$ using its CIGAR string $cigar_i$ and its mapping position pos_i. This process yields the *expanded* read exp_i. Thereafter, the difference d_i (containing the differences from the expanded/aligned read exp_i to read exp_j) is computed and passed to an entropy coder, e.g. an AC. At the decoder, read seq_i can be reconstructed from exp_j by applying d_i to reconstruct the nucleotide sequence, the CIGAR string, and the mapping position. The encoder structure of the nucleotide sequence compressor is shown in Figure 4.9.

```
POS  CIGAR       SEQ
7    8M2I4M1D3M  TTAGATAAAGGATACTG  < read 1
9    3S6M1P1I4M  AAAAGATAAGGATA     < read 2

       6789...  < POS
        8M       2I4M   1D3M
        TTAGATAAAGGATA_CTG  < read 1 expanded
       AAAAGATAAGGGATA      < read 2 expanded
       3S 6M     1P1I4M
```

Listing 4.1: *Working principle of the* **expand** *function. The reads are positioned according to their mapping position ("POS"). Afterwards, the CIGAR string is used to unwind both sequences. They would both now map onto a virtual reference sequence.*

The alignment of read seq_i to read seq_j is done with a function called `expand`. The

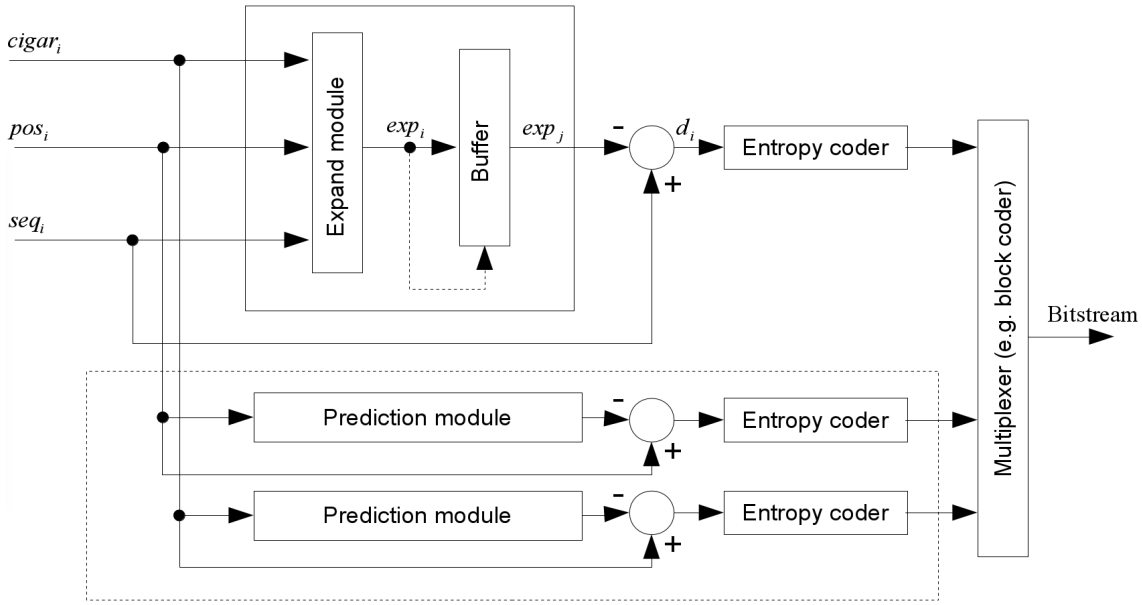


Figure 4.9.: *Possible encoder structure of the proposed nucleotide sequence compressor.*

output of `expand` is called exp_i. The CIGAR strings and mapping positions of both reads are used to unwind the sequence such that they would both directly align[9]. In general, unwinding the sequence consists of a series of insertions, deletions, and modification operations as described by the CIGAR string. Consequently, `expand` condenses the position, the CIGAR string, and the sequence into a joint representative code word. As an alternative, the last step could be omitted and these SAM fields could be processed separately[10]. A short example of the working principle of `expand` is given in Listing 4.1.

In the proposed algorithm, the context read seq_j used to encode read seq_i is by default the adjacent read seq_{i-1}. This context read might be highly erroneous or not properly aligned. This issue is addressed by keeping track of N previous reads (this can be illustrated as "sliding window"). Sequence reads which are not aligned might alternatively be directly passed to the entropy coder or retained and encoded separately later, e.g. at the end of each block. The selected matching read might be signalled or estimated at the decoder. A possible implementation of the proposed algorithm in pseudocode is shown in Algorithm 1.

[9]Please refer to the SAM specification [SAM14], Section 1.4ff for a detailed explanation of the CIGAR string syntax.

[10]This has been done in order to compare the proposed compression algorithm with other tools.

4.3. Predictive Coding of Quality Scores

The quality scores yield the largest entropy among all fields present in a typical SAM file, mainly due to their large alphabet. The authors of fqzcomp [BM13] state amongst other things that any quality score q_i has a strong correlation to the direct previous quality scores $(q_{i-1}, q_{i-2}, \ldots)$, decreasing the further back we go. This assumption is verified in Figure 4.5. Current compression implementations exploit this observation by employing a Markov chain with the memory ranging over the direct predecessors in the quality score stream. In the following context, this is referred to as "intra-line" prediction.

Current SAM file compressors model the quality score source as a Markov chain, predicting the current symbol with a maximum likelihood predictor (introduced in Section 2.2.4.2) using the N direct predecessors. In each state $(q_1, \ldots, q_{N-1})$ the symbol with the greatest conditional probability is selected as prediction value $q_N^{(p)}$. The maximum likelihood predictor consumes a huge amount of memory since we have to keep track of $K^N \cdot N$ relative symbol frequencies (having an alphabet of size K). This model needs a large amount of data to train in order to tightly fit the model to the data.

4.3.1. Proposed Compression Schemes

As an alternative to building the prediction memory during the decoding process, the possibility to store the prediction table (generated during compression) e.g. in the file header to speed up decompression is proposed. The prediction tables for the individual blocks may then be encoded, e.g. delta-encoded with subsequent AC. This enables random access to the compressed data while retaining the compression performance of the prediction model (e.g. Markov model) by providing the fully trained prediction memory, or parts of it, (built from the full input data at the encoder) to the decoder, which can then be directly used from the start of the decoding process.

The following sections describe alternatives to predictive quality score compression which might be combined to yield the best compression performance.

4.3.1.1. Extended Line Context

The observation has been made, that in some datasets subsequent quality score lines are "similar" in terms of a small Levensthein [Lev66] distance between pairs of quality score lines. This can be addressed by using a so-called *line context*, i.e. a structure holding some number of previous lines from the current block (this is known as "sliding window"). The prediction context is divided into one part

containing the memory locations on the current line ("intra-line memory") and one part containing the memory locations on some specific line from the line context ("inter-line memory") as depicted in Figure 4.10.

```
DECEEEECDDDDDDDDDDDDDDEDCDDDDDDD
                ⋮
DDDDDDDCDDDDDBDDDDDDDDEEEEDDD
```

Figure 4.10.: *Visualisation of the intra-line (dark blue) and inter-line (light blue) context used to predict the current symbol (green).*

Choosing the proper line from the line context to boost prediction performance is crucial. On possibility would be to compute an identification vector for each quality score line (or parts of it) in the line context. As an example, this could be a three-dimensional vector, containing e.g. the length, the arithmetic mean, and the variance of a quality score line. In the process of encoding line i, the one specific line $\hat{i}$ from the line context with the least identification vector distance is selected, whereas any known vector distance measurement might be applied, e.g. the three-dimensional Euclidean distance. The vector components might be individually weighted. Consecutively, the intra-line memory is placed on line i and the inter-line memory is placed on the selected line $\hat{i}$ from the line context.

4.3.1.2. Dictionary-based Substring Matching

To encode the quality score $q_{i,j}$ in column j and line i, another possibility is to select the substring $\underline{\kappa}$ of N preceding symbols: $\underline{\kappa} = (q_{i,j-N-1}, \ldots, q_{i,j-1})$. Then, a similar substring $\hat{\underline{\kappa}} = (q_{\xi,\eta}, \ldots, q_{\xi,\eta+N}) \equiv \underline{\kappa}$ could be searched for in the line context. Furthermore, $q_{\xi,\eta+N+1}$ could be used as the prediction value $q_{i,j}^{(p)}$ for $q_{i,j}$. Alternatively, the prediction value $q_{i,j}^{(p)}$ could be calculated as a mean or median or as an arbitrary combination from M substring matches $\hat{\underline{\kappa}}_m$, $1 \leq m \leq M$. This principle is briefly pointed out in Figure 4.11.

```
DECEEEECDDDDDDDDDDDDDDEDCDDDDDDD
                ⋮
DDDDDDDCDDDDDBDEECDDDDEEEEDDD
```

Figure 4.11.: *The green symbol is predicted from the blue symbol, if a matching substring containing $M = 5$ symbols (red) is found in the line context.*

4.4. Quantized Quality Scores

Due to the high entropy of quality scores, lossy compression should be investigated. Currently, this topic is investigated in the scientific community, e.g. by Benoit et al. [BLL+15] and Ochoa et al. [OAB+13]. In the scope of this thesis, lossy quality score compression has not been evaluated. Nevertheless, a short overview of achievable compression ratios shall be given here. The observations have been done using a MAX-LLOYD quantizer[11] [Llo82, Max60].

A training set of 10,000 quality score *values* has been used for the quantizer. The code book size has been varied from $2^1 = 2$ up to $2^5 = 32$. Subsequently, the first 100,000 quality score lines have been quantized. The obtained entropy and the mean squared distortion of the quantized signals are shown in Figure 4.12. It can be concluded, that rates of less than 2 $^{bit}/_{symbol}$ should be achievable at an acceptable distortion. However, it is not yet fully clear, which distortion metrics should be used (the impact on downstream analyses of lossy quality score compression has e.g. been evaluated by Kozanitis et al. [KSK+11]).

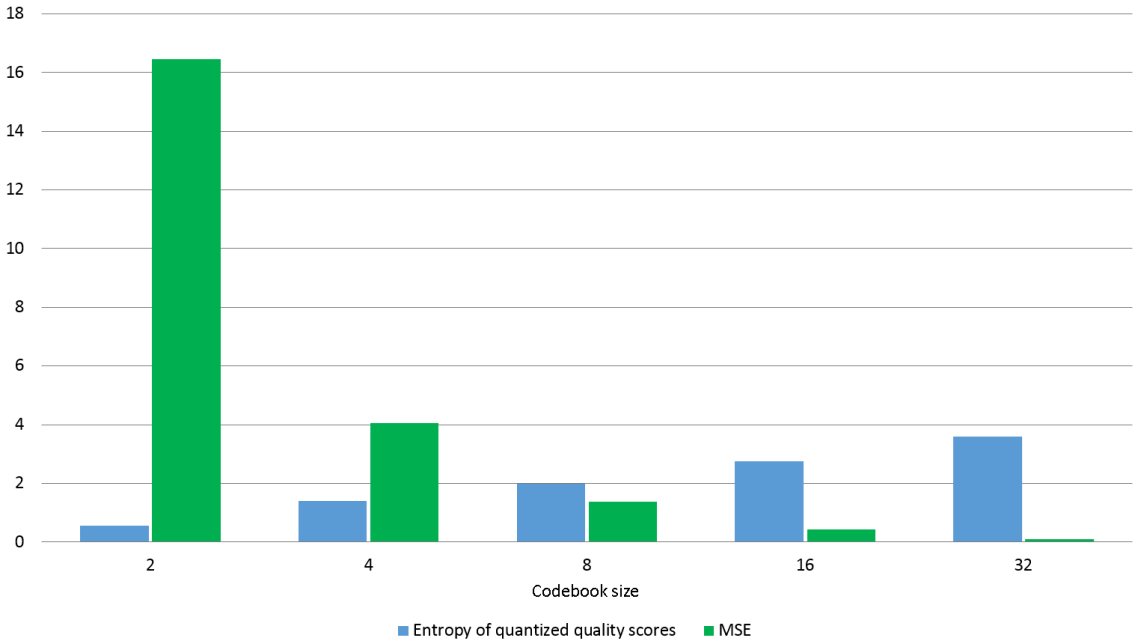


Figure 4.12.: *The first 100,000 quality score lines have been quantized with a* MAX-LLOYD *quantizer. The first 10,000 quality score values have been used to train the code book.*

[11] Actually, the MATLAB functions `lloyd` and `quantiz` have been used.

Algorithm 1 *Pseudocode implementation of the proposed nucleotide sequence encoder.*

```
1:  Input:
2:  L                                          ▷ Number of SAM records
3:  N_b                                        ▷ Block size
4:  t                                          ▷ Threshold
5:  pos[L]
6:  cigar[L]
7:  seq[L]
8:
9:  Output:
10: binary stream                              ▷ From entropy coder
11:
12: new circbuf(N)                             ▷ Circular buffer of size N (sliding window)
13:
14: l ← 0                                      ▷ Line counter
15: b ← N_b                                    ▷ Block line counter
16: while l < L do
17:     if b == N_b then                       ▷ Start new block
18:         clear_circbuf();
19:         reset_entropy_coder();
20:         entropy_coder(pos[l], cigar[l], seq[l]);
21:         circbuf.push(pos[l], cigar[l], seq[l]);
22:         b ← 1
23:     else
24:         if cigar(l) ≠ "*" then             ▷ Sequence is mapped
25:             e_min ← ∞
26:             repeat
27:                 seq_exp = expand(pos[l], cigar[l], seq[l])
28:                 (pos_ref, cigar_ref, seq_ref, ref_index) = circbuf.get_next()
29:                 ref_exp = expand(pos_ref, cigar_ref, seq_ref)
30:                 d = compute_differences(seq_exp, ref_exp);
31:                 if entropy(d) < e_min then
32:                     e_min ← entropy(d)
33:                     ref_index_best ← d
34:                 end if
35:             until entropy(d) > t & circbuf.has_next()
36:             entropy_coder(ref_index_best, d_best)
37:         else
38:             entropy_coder(pos[l], cigar[l], seq[l])
39:         end if
40:         circbuf.push(pos[l], cigar[l], seq[l])
41:     end if
42:     b ← b + 1
43:     l ← l + 1
44: end while
```

5. Results

5.1. Software Framework

In order to evaluate the compression approaches presented in Chapter 4, a software framework called tsc (C99-compliant [C9907]) has been developed. Currently, the software is able process SAM files. Support for FASTQ files might be added in the future. First, tsc splits the SAM file into separate streams for each field of the SAM records. These fields are described in detail in Section 2.4.3. Second, specialised compression algorithms (called "sub-block codecs") can then be invoked for each field or for a combination of fields. As pointed out in Appendix C, new compression modules for e.g. quality score compression can easily be mounted[1]. Furthermore, the software provides detailed timing and compression statistics for all sub-block codecs. Due to the modular structure and the statistic features implemented, the tsc framework could serve as a "test suite" for evaluating different compression approaches[2] or as a starting point for a standardisation effort.

The program structure (only intended to serve as an overview about the framework) is shown in Figure 5.1.

In order to ensure random access to the compressed data, the compression is then performed block-wise. To accomplish this goal, a binary file format (with the file name extension `tsc`) has been developed. The basic layout of a tsc file is shown in Figure 5.2. Generally, the file format is extensible and could serve as a container format for arbitrary (compressed) genomic information data. Currently, three distinct sub-block codecs are implemented, as shown in Figure 5.1.

A tsc file contains a header with general information about the compressed file. The detailed contents of the tsc header (version 1.0) have been summarised in Table 5.1.

The SAM header is simply copied to the tsc file as shown in Table 5.2.

The structure of a tsc block header, prepended to each block, is shown in Table 5.3. The file pointer offsets (`bh_fpos` and `bh_fpos_nxt`) can be used to seek through the compressed file in order to randomly access compressed blocks.

More information on the structure of the tsc software is provided in Appendix C.

[1] The compression modules only must provide a suitable interface as briefly describe in Appendix C.

[2] Presumably, a combination of compression algorithms from various sources will yield the best overall compression ratio.

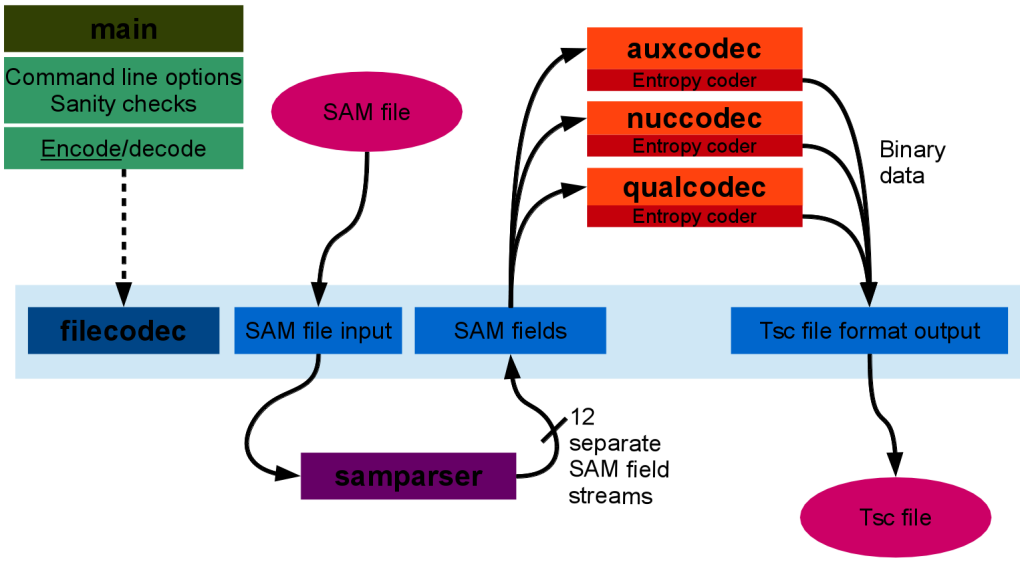


Figure 5.1.: *The most important module in the tsc framework is the so-called* `filecodec`*, which in turn calls the parser and the sub-block codecs (which implement the specialised compression algorithms/codecs).*

Data type	Description
`unsigned char fh_magic[6]`	"tsc–" + '\0'
`unsigned char fh_version[6]`	VERSION + '\0'
`uint64_t fh_blk_lc`	no. of lines per block
`uint64_t fh_blk_n`	number of blocks

Table 5.1.: *Data stored in the tsc header (version 1.0).*

5.2. MPEG Genomic Information Database

The MPEG AhG on genome information compression and storage focused on the selection of the most appropriate file formats for genomic information representation after the 110th MPEG meeting in Strasbourg. Prior to the selection of one or more file formats currently used by the scientific community, the AhG identified the types of genomic information data relevant for compression. In general, this are on the one hand unmapped reads as directly produced by the sequencing machines, and on the other hand aligned reads with their associated metadata. Finally, the AhG selected SAM respectively its compressed equivalent BAM as file format for aligned reads and FASTQ as file format for unmapped reads[3]. The relation among these file formats has been shown in Figure 2.7 in Section 2.4.

[3]One exception in the genomic information database is Oxford Nanopore data, which was only available in the form of Fast5 files. However, Fast5 files can be easily transcoded to FASTQ using for example `fast5tofastq` (`https://github.com/gt1/fast5tofastq`).

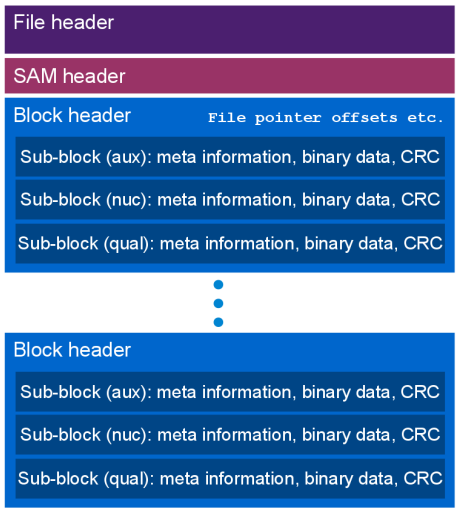


Figure 5.2.: *A tsc file consists of an arbitrary number of independent blocks to ensure random access to specific blocks. The file header contains generic information about the tsc file. The SAM header is simply copied from the input file. This is feasible, as the SAM header has a negligible size compared to the alignment section.*

Data type	**Description**
`uint64_t sh_sz`	SAM header size
`unsigned char *sh_data`	SAM header data

Table 5.2.: *The SAM header stored in a tsc file.*

The genomic information database (GIDB[4]) as a reference collection of genomic data was then compiled by the AhG according to specific criteria to ensure a wide variety of challenges for existing and future compression tools. Table 5.4 gives an overview about the criteria and their rationales. Some specific characteristics incorporated into the GIDB shall be highlighted here:

- Sequencing data produced by different technologies has been incorporated into the database, as for example Illumina machines produce fixed-length reads that are rather short and have few errors whereas for example PacBio technology yields variable read lengths.
- Bacteria have for example been added because they are usually aligned against the closest known homologue[5].

[4]The database consists of a selection of mainly "gzipped" FASTQ files and BAM files and has a size of approximately 6.4 TiB and has thus been distributed via hard disks on the MPEG meetings. Nevertheless, all files are publicly available and can be downloaded on the web [AMC+15]. Within the TNT cluster, the data is stored in `/data/genome/`.

[5]Homology between DNA sequences is defined in terms of shared ancestry.

Data type	Description
`uint64_t bh_fpos`	fp offset to the beginning of -this- block
`uint64_t bh_fpos_nxt`	fp offset to the beginning of the -next- block
`uint64_t bh_blk_cnt`	block count, starting with 0
`uint64_t bh_line_cnt`	no. of lines in block
`uint64_t bh_chr_cnt`	chromosome number
`uint64_t bh_pos_min`	smallest position contained in block
`uint64_t bh_pos_max`	largest position contained in block

Table 5.3.: *The tsc block header ensures random access by storing a file pointer to the beginning of the next block. It is thus possible to seek through the compressed file.*

- Regarding the genome size, plants have been added to the GIDB as they are usually multiploid in contrast to the diploid human genome.

Organism	Criterion
Homo sapiens	Different coverage levels
Bacteria	Eukaryotic versus prokaryotic life forms
Plants	Polyploidy

Type of experiment	Attributes
Metagenomic	No reference (nor homologous reference) known
Cancer cell lines	Genetic variations

Technology	Attributes
Illumina	Fixed length, short, few errors
PacBio	Variable length, long, high error rate
IonTorrent	In between

Table 5.4.: *Main differentiations used for compiling a database of genomic information data. The table has been constructed using information from various MPEG documents [Chi15].*

The SAM files from the GIDB are listed in Table 5.5. For the sake of simplicity in the scope of this thesis, short file identifiers have been defined as shown in Table 5.5.

Apart from the MPEG database, there are a number of other publicly available repositories providing genomic information data for scientific research. The International Nucleotide Sequence Database Collaboration (INSDC[6]) is for example a joint effort of the DNA Data Bank of Japan[7] (DDBJ), the European Bioinformatics

[6] `http://www.insdc.org/`
[7] `http://www.ddbj.nig.ac.jp/`

File	Identifier
G15511.HCC1143.1.sam	e88HCC
G15511.HCC1143_BL.1.sam	598HCC
HCC1954.mix1.n80t20.sam	HCCmix
MiSeq_Ecoli_DH10B_110721_PF.sam	DH10B
9799_7#3.sam	9799#3
K562_cytosol_LID8465_TopHat_v2.sam	K562
9827_2#49.sam	9827#49
NA12877_S1.sam	NA12877
NA12878_S1.sam	NA12878
NA12882_S1.sam	NA12882
NA12879_S1.sam	NA12879
NA12878_S1.sam	NA12878
NA12890_S1.sam	NA12890
NA21144.chrom11...20130415.sam	NAchrom11
sample-2-12_sorted.sam	sample-12
sample-2-11_sorted.sam	sample-11
sample-2-10_sorted.sam	sample-10

Table 5.5.: *In the context of this thesis, the defined abbreviated identifiers for the SAM files from the MPEG genomic information database are used.*

Institute[8] (EBI) and the National Center for Biotechnology Information[9] (NCBI). The largest contributor to the INSDC is the 1000 Genomes Project, an international research consortium [Siv08]. The NCBI furthermore manages a public repository known as Gene Expression Omnibus (GEO[10]) [EDL02].

5.3. Results: Block-based Compression Of Nucleotide Sequences

In the scope of this thesis, only the performance of the proposed nucleotide sequence compressor from Section 4.2 has been evaluated. The evaluation of the predictive coding schemes for quality score compression is still in progress. In order to obtain complete compressed (binary) tsc files, the quality scores and the remaining SAM fields have been solely compressed with an AC. The detailed compression results (including the CPU and memory specifications etc.) can be found in Appendix E. For

[8] https://www.ebi.ac.uk/
[9] http://www.ncbi.nlm.nih.gov/
[10] http://www.ncbi.nlm.nih.gov/geo/

the evaluation of the compression performance, only a subset[11] of the files mentioned in Section 5.2 has been used, composed of DH10B, K562, 9827#49, and NA12878. As the tsc software is still being developed, full runs could only be produced on these datasets. The obtained compression ratios regarding the nucleotide sequences are summarised in Table 5.6. Furthermore, Table 5.6 contains the compression ratios yielded by Quip [JRPK12] respectively DeeZ [HNS14] for the given datasets.

File	Identifier	tsc	Quip	DeeZ
MiSeq_Ecoli_DH10B_110721_PF.sam	DH10B	**3.4%**	18.3%	3.9%
K562_cytosol_LID8465_TopHat_v2.sam	K562	20.1%	19.6%	**4.6%**
9827_2#49.sam	9827#49	22.5%	26.7%	7.7%
NA12878_S1.sam	NA12878	**2.7%**	6.2%	7.3%

Table 5.6.: *The compression ratios (CR) for nucleotide sequences yielded by tsc, Quip, and DeeZ. The best compression ratios are written in a bold font.*

As shown in Table 5.6, tsc shows the best compression performance for the datasets DH10B and NA12878. By selecting appropriate context lines from the sliding window to encode the current nucleotide sequence, the proposed algorithm outperforms the "local assembly" approach by DeeZ, which is by far computationally more expensive. Tsc does not perform well on datasets with a low coverage or bad alignments, as in the datasets K562 and 9827#49. This has already been predicted in Section 4.1.3. Nevertheless, in these cases, tsc is still on par with Quip, which employs an order-12 prediction with subsequent AC as pointed out in Section 3.4.

Figure 5.3 shows the data distribution in the (compressed) tsc file for DH10B. The nucleotide sequences ("Nuc") have been compressed with the proposed algorithm. The quality scores ("QUAL") have been compressed solely using an AC (without employing any prediction). The remaining SAM fields have been summarised in "Aux" and have been compressed with an AC, too. The data distributions for the other tsc files from Table 5.6 look similar, as pointed out in Appendix E. It can be concluded, that the quality scores consume a huge amount of space in the compressed file. Generally, the relative space consumed by (compressed) quality scores even increases if the data now present in "Aux" were compressed with more sophisticated schemes, as for example in DeeZ [HNS14]. Therefore, lossy quality score compression should be investigated, as pointed out in Chapter 6.

Furthermore, Figure 5.4 shows the time shared between the different compression steps. Only a small time amount is consumed by input/output operations etc. ("Remaining"). The largest time share is used for the predictive coding.

[11] The compression results have been obtained by compressing the first 100,000 reads of each dataset. A comparison with the compression ratios obtained by compressing the complete datasets showed no significant differences.

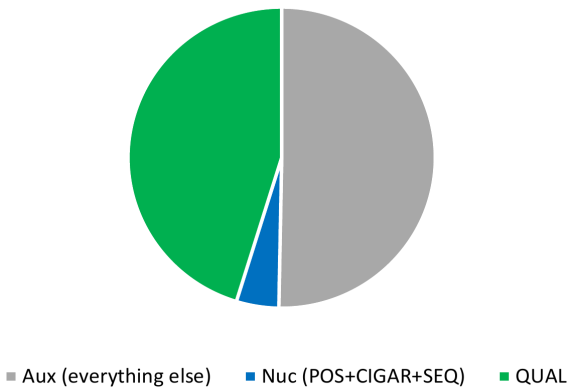


Figure 5.3.: *Data distribution in a (compressed) tsc file (`DH10B`). The "Aux" field obtains about 50% of the compressed file size , the "QUAL" field consumes about 45%, and the compressed nucleotide sequences only consume about 5% of the compressed file size.*

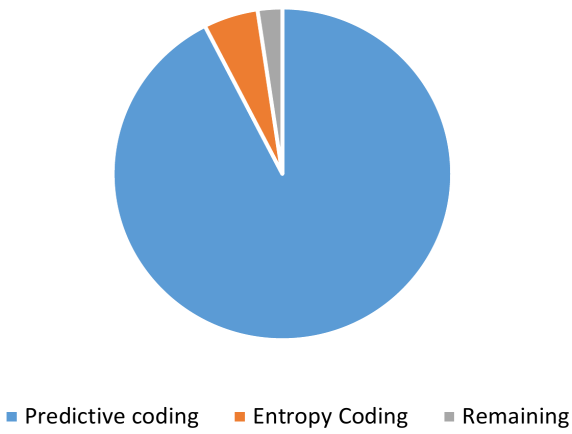


Figure 5.4.: *The timing statistics obtained by compressing the dataset `DH10B` with tsc. Only a small time amount is consumed by input/output operations etc. ("Remaining"). The largest time share is occupied by the predictive coding steps.*

6. Summary and Outlook

The introduction to this report (Chapter 1) highlighted the fact that due to the development of next-generation sequencing (NGS) technologies since the 1990s, and the huge amount of data generated by those, the development of specialised compression systems for genomic information data is required to close the gap between sequencing and storage costs. This issue has been examined and evaluated throughout the following sections.

First, Chapter 2 introduced the theoretical foundations necessary for the understanding of this report. The first sections in this Chapter (Sections 2.1 (Statistics) and 2.2 (Source Coding)) presented the theory of probability and discrete random processes as well as the field of source coding with an emphasis on predictive coding schemes and entropy coding algorithms.

Referring to the NGS technologies already mentioned in the introduction, Section 2.3 then introduced the history of DNA sequencing in more detail. First, a brief overview about the "first-generation" sequencing technologies was presented. Second, the rise of the next-generation sequencing technologies was introduced. The "sequencing-by-synthesis" [RKP+96] technology has been specifically highlighted, as the Solexa/Illumina technology incorporating this approach is responsible for more than 90 percent of all sequencing data produced today. Furthermore, Section 2.3 presented a detailed comparison of the most important sequencing technologies, explaining the technical specifications which are relevant for the compression of genomic information data. Subsequently, different types for the representation of genomic information (using either nucleotide sequences combined with quality scores or the representation using the IUPAC notation) have been introduced. The rationales behind the fact that most NGS sequencing technologies do not (yet) use the IUPAC representation have especially been highlighted. Moreover, Section 2.3.5 explained the processes of alignment respectively assembly. Together with the high sequencing depth inherent in most NGS technologies, these processes ensure that the redundancy present in NGS sequencing data is directly accessible for compression algorithms. This redundancy is exploited by the proposed nucleotide sequence compressor from Section 4.2.

Finally, Section 2.4 introduced the most widely used file formats for raw reads (FASTQ) and aligned respectively mapped reads (SAM). The differences between these file formats have been highlighted in detail. Specifically, the different "flavours" of quality scores have been presented. The most important parts of the SAM specification [SAM14], have been exposed. Chapter 2 closed with a description of the BAM file format, being the de-facto standard for the compression of SAM

files.

Chapter 3 opened with a generic problem definition of genome compression and storage. The scientific field known as "genomics" has been introduced as one of the major generators of Big Data [SLF+15], identifying a number of key technological needs for "Big Data genomics". Among these is the need for high performance compression of genomic information data. Section 3.1 furthermore mentioned the standardisation efforts started by MPEG and presented the "genomic life cycle", which is summarising the aforementioned components from sequencing via assembly respectively alignment to compression and analysis. The genomic life cycle highlights in particular the need for random access to the compressed data, due to the large file sizes and the repeated steps of compression, decompression, and analysis.

Section 3.2 addressed the standardisation efforts started yet, mainly by the MPEG ad-hoc group (AhG) on genome compression. Nevertheless, other important initiatives have been mentioned as well. Concentrating on the MPEG initiatives, Section 3.3 presents the requirements for genomic information compression developed and issued by the MPEG AhG in detail. These requirements served as a guideline for the development of novel approaches to genomic information compression in Chapter 4. Chapter 3 closed with a general overview about genome compression in Section 3.4. The four principal approaches towards genomic information compression (namely naive bit encoding, dictionary-based compression, compression using statistical methods, and referential methods) have been introduced first. Hereafter, recent genomic information compression tools have been examined in more detail. Specifically, two of them (Quip [JRPK12] and DeeZ [HNS14]) have been closely examined and compared.

Having described the *status quo* in genome sequencing, compression and associated topics, Chapter 4 ultimately presented the novel approaches to genome compression developed in the scope of this thesis. First, Section 4.1 investigated the compression of sequence reads using universal (i.e. dictionary-based) coding algorithms, but concluded that these approaches are only capable of exploiting redundancy along single nucleotide sequences. Redundancy generated by a high sequencing depth in combination with the processes of alignment respectively assembly is not covered by these approaches. As a response to these issues, Section 4.2 presented the proposed compression mechanism for nucleotide sequences. In contrast to already existing approaches, the proposed compressor does not require a reference genome nor reference sequences and is able to work on small block granularities, thus ensuring improved random access to the compressed data. The redundancy generated by a high sequencing depth is exploited by the joint coding of the mapping positions, the CIGAR strings, and the sequence reads.

Regarding the compression of quality scores, the possibility of employing Markov chains for prediction was presented in Section 4.1.5. Based on the analysis of autocorrelation functions, novel starting points regarding quality score compression have

been pointed out. Section 4.3 presents several novel approaches for the compression of quality scores. The approaches try to exploit inter-line correlations by employing e.g. a MARKOV chain with an "extended line context" (Section 4.3.1.1) or a "dictionary-based substring matching" (Section 4.3.1.2) approach. Furthermore, due to the high entropy of the quality scores, there is a need for the investigation of lossy quality score compression. Section 4.4 briefly shows that a reasonable entropy reduction can be achieved by employing a MAX-LLOYD quantizer [Llo82, Max60]. However, the impact of lossy quality score compression on downstream analysis applications has not yet been fully evaluated [OAB+13] and appropriate distortion metrics have to be defined and examined.

The development of novel compression approaches led to a software framework which was developed in the scope of this thesis. This framework was presented at the beginning of Chapter 5 in Section 5.1. It has to be pointed out, that the framework and the associated file format have been specifically developed to ensure random access to the compressed data. Moreover, the software has been designed with a modular structure, allowing to mount dedicated compression modules, for e.g. quality scores.

The proposed nucleotide sequence compressor has been evaluated in Section 5.3, after introducing the genomic information database (GIDB) in Section 5.2. It has been shown, that the proposed compression algorithm (implemented in the software framework "tsc") yields superior compression ratios (compared to Quip [JRPK12] and even DeeZ [HNS14]) for some datasets. However, tsc does not perform as well as DeeZ on datasets with a low coverage or bad alignments. This has already been predicted in Section 4.1.3. Nevertheless, in these cases, tsc is still on par with Quip. Regarding the proposed nucleotide sequence compressor, some aspects are still subject to future research. For example, the compression algorithm might be improved by exploiting another SAM field, the mapping quality. This information could be used in order to improve the selection of a proper context read in the sliding window.

Another important topic is the lossy compression of quality scores. This has already been briefly highlighted in Section 4.4 and has been investigated in recent research projects. For example, Ochoa et al. [OAB+13] presented an approach based on rate distortion theory. They showed that "it is possible to achieve a significant reduction in size with little compromise in performance on downstream applications (e.g. alignment)" [OAB+13]. Furthermore, Benoit et al. [BLL+15] investigated the lossy transformation of quality scores using DE BRUIJN graphs with an emphasis on downstream analyses. According to these recent papers and the results obtained in the scope of this thesis, lossy quality score compression should be a major topic in future research initiatives.

A. Samsplit

Samsplit is a collection of software for the (statistical) investigation of SAM files. The main program is called *samsplit* and is explained in Section A.1. Other software contained within the samsplit repository are some MATLAB scripts, mainly for the statistical analysis of SAM files. Some of these script have some kind of generic use and have been condensed in a library called *tntlib*. The MATLAB scripts as well as `tntlib` are explained in Section A.2.

Samsplit is hosted in a Git repository [CS14] and accessible from within the TNT cluster via: `git@git:jvoges/samsplit.git`. Samsplit is licensed under the GNU General Public License (GPL) [Smi07]. Additionally, the samsplit software collection is shipped on the DVD accompanying this thesis.

The Git repository furthermore contains the Perl scripts `validatefastq.pl` for the syntax check of FASTQ files, and `run_samsplit.pl` to run samsplit on numerous files.

The current organisation structure of the repository can be viewed in Listing A.1.

```
samsplit/
   matlab/             // MATLAB scripts and tntlib
   run_samsplit.pl     // Perl script to process numerous SAM files
   src/                // source code for samsplit
   tests/              // various SAM test files
   validatefastq.pl    // FASTQ validation
```

Listing A.1: *The repository structure of samsplit.*

A.1. Samsplit

Samsplit is a small program (written in C++). Its main (and so far only) purpose is to parse a SAM file and to split it into separate streams for the twelve fields (or columns). A SAM file typically contains a *header*; these are all lines starting with the ASCII symbol "@". These lines are discarded by samsplit. Furthermore, a SAM file contains typically a huge amount of lines (ending with the ASCII symbol `\n`) called *records*. Each record contains twelve fields (separated by tab stops - ASCII symbol `\t`) according to the SAM specification [SAM14].

Samsplit takes a SAM file as input and separates the twelve columns (according to the options given). A typical call to extract the sequence reads can be viewed in

Listing A.2.

```
$ samsplit test.sam --field=10 > test.seq
```

Listing A.2: *Samsplit is used to extract the sequence reads (column 10) from* `test.sam`*. The raw reads are written to* `seq.sam`*.*

The currently available command line options for samsplit version 1.0 are listed in Table A.1.

Short option	Long option	Description
`-f`	`--field`	SAM field (column) to process (0 for full record, otherwise 1-12)
`-h`	`--help`	Print help
`-n`	`--nrecords`	Number of records to process (0 for whole file)

Table A.1.: *Samsplit options*

A.2. MATLAB Scripts and Tntlib

For the statistical evaluation of SAM files, several short MATLAB scripts have been developed in the scope of this thesis. Most of them are referenced in the above text, mainly in Section 4.1. These scripts can be found in the folder `matlab/`. The MATLAB scripts that proved a more generic use have been encapsulated as a small library called *tntlib*[1]. Tntlib contains for example functions to generate a Walsh-Hadamard matrix (`tnt_whtmtx.m`) [Hor07], or functions to efficiently read ASCII files line by line (`tnt_read_ascii.m`).

The scripts used for the statistical analyses (mainly in Section 4.1 can be found in the folder `matlab/analysis/`. Either `test_environment.pl` or `test_environment.m` can be invoked to start statistical analysis with one of the scripts briefly explained in Table A.2.

[1]The functions in tntlib can be made available to any other MATLAB code with `addpath('path/to/tntlib');`.

File	**Description**
`pos_visualization`	Returns the positions from a SAM file as row vector
`qual_entropy`	Computes the entropy of the quality scores in a SAM file
`qual_lloyds`	Quantizes the quality scores in a SAM file
`qual_markov`	Reducing the quality score entropy by using a Markov model
`qual_stats`	Generate statistics for the quality scores in a SAM file
`seq_entropy`	Computes the entropy of the nucleotide sequences in a SAM file

Table A.2.: *MATLAB scripts for the analysis of SAM files*

B. SAM/BAM File Processing

An important tool to investigate and manipulate aligned reads expressed in SAM or BAM files is the SAMtools toolkit [Gen15]. A good overview about the SAM format is given by Li et al. in their paper describing SAMtools [LHW+09]. Apart from this thesis, the MPEG document "Database for Evaluation of Genome Compression and Storage" [AMC+15] provides a similar overview about the use of SAMtools.

B.1. Getting SAMtools

SAMtools can be downloaded from `http://www.htslib.org/download/` and built according to the instructions provided on that web page. Internally, SAMtools uses HTSlib[1], which can be downloaded on that same web page.

The SAMtools toolkit provides a variety of tools (for example conversion from BAM to FASTQ format with `bam2fq`). However, only the tools `view` and `index` have been used in this work. The following sections provide brief explanations and examples for the most common use cases.

B.2. Accessing SAM/BAM Content

The SAMtools command `view` can be used to view the contents of SAM files and even of compressed BAM files. Listing B.1 shows an example of how to access BAM files with SAMtools.

```
$ samtools view -f 4 foo.bam | more
```

Listing B.1: *SAMtools is used to extract all reads with the flag value 4 (these reads failed to map to the reference genome) from the file `foo.bam`. The output is piped to `more`.*

Furthermore, SAMtools can be used to access a specific part of a BAM file. An example is shown in Listing B.2.

[1] HTSlib is a library written in C for processing high-throughput sequencing data such as SAM and BAM files. HTSlib (and SAMtools) furthermore provide support for a file format known as CRAM [ENA15, CCB12, FLCB11]. We do not investigate CRAM here, as it has not been chosen by the MPEG AhG for the evaluation of genomic information compression, and thus has not been used in this work.

```
$ samtools view foo.bam -region chr4:3000000-3001000
```

Listing B.2: *Only a specific region on chromosome 4 is extracted from the BAM file.*

B.3. BAM Indexing

Large BAM files sometimes have a companion file containing offsets into the BAM file. This companion file with the suffix `bai` (and the same name as the BAM file) serves as a table of contents for the BAM file. Listing B.3 shows how to create an such an index file with SAMtools.

```
$ samtools index foo.bam
```

Listing B.3: *SAMtools is used to generate an index file `foo.bai` for the BAM file `foo.bam`.*

B.4. SAM/BAM Conversion

Many downstream analysis application require BAM files as input. As BAM files are in general block-wise gzipped SAM files, they provide random access to specific blocks. The compression of a SAM file to a BAM file is shown in Listing B.4.

```
$ samtools view -b -S -o foo.bam foo.sam
```

Listing B.4: *SAMtools is used to compress a SAM file. The flag `-b` indicates the compression to a BAM file, the flag `-S` tells SAMtools that the input is a SAM file, and `-o` specifies the name of the output file.*

An example for the conversion from BAM to SAM format is shown in Listing B.5.

```
$ samtools view -h foo.bam -o foo.sam
```

Listing B.5: *SAMtools is used to decompress the file `foo.bam`.*

C. The Tsc Framework

The compression approaches presented in Chapter 4 have been implemented in a software called *tsc*. The tsc file structure and the general design of the framework have already been discussed in Section 5.1.

Tsc is hosted in a Git repository [CS14] and accessible from within the TNT cluster via: `git@git:jvoges/tsc.git`. Additionally, the tsc software collection is shipped on the DVD accompanying this thesis. The current organisation structure of the repository can be viewed in Listing C.1.

```
tsc/
   BUILD        // Building instructions
   framework/   // The tsc -framework- for distribution
   gidb/        // Files concerning the genomic information database
   src/         // Source code
   tests/       // Various test files
   TODO         // Self-explanatory
```

Listing C.1: *The repository structure of tsc.*

The following paragraphs illustrate the functionality of the most important modules during the compression process (all files mentioned from now on can be found in the directory `src/`.). First, the main function in `tsc.c` performs sanity checks. When everything is safe, the *file codec* is invoked as shown in Listing C.2.

```
...
fileenc_t* fileenc = fileenc_new(tsc_in_fp, tsc_out_fp);
fileenc_encode(fileenc);
fileenc_free(fileenc);
...
```

Listing C.2: *The main function calls the filecodec.*

Subsequently, `fileenc_encode()` (see `filecodec.c`) calls the SAM parser and invokes the dedicated sub-block codecs as shown in Listing C.3. The SAM parser fills the structure `samrecord_t` which then contains all fields (i.e. columns) of a single record. The different fields can then be added to one of the sub-block codecs (lines 14-28). Currently, as explained in Chapter 4, the positions, the CIGAR strings, and the nucleotide sequences themselves are jointly encoded by the `nuccodec`. The `qualcodec` encodes the quality scores. In order to encode the entire SAM file, everything else is passed to the `auxcodec`. Upon having processed all lines for a block, the entropy coders are invoked (lines 6-8). Adding or removing other sub-block codecs can be easily done. Any novel sub-block codec (e.g. `mycodec`) must provide only

two methods (`mycodec_add_record()` and `mycodec_write_block()`) for encoding, respectively two associated methods for decoding.

```
...
samrecord_t* samrecord = &(fileenc->samparser->curr);
while (samparser_next(fileenc->samparser)) {
    if (bh_line_cnt >= fh_blk_lc) {
        ...
        auxenc_write_block(fileenc->auxenc, fileenc->ofp);
        nucenc_write_block(fileenc->nucenc, fileenc->ofp);
        qualenc_write_block(fileenc->qualenc, fileenc->ofp);
        ...
        bh_blk_cnt++;
        bh_line_cnt = 0;
    }
    ...
    auxenc_add_record(fileenc->auxenc,
                      samrecord->str_fields[QNAME],
                      samrecord->int_fields[FLAG],
                      samrecord->str_fields[RNAME],
                      samrecord->int_fields[MAPQ],
                      samrecord->str_fields[RNEXT],
                      samrecord->int_fields[PNEXT],
                      samrecord->int_fields[TLEN],
                      samrecord->str_fields[OPT]);
    nucenc_add_record(fileenc->nucenc,
                      samrecord->int_fields[POS],
                      samrecord->str_fields[CIGAR],
                      samrecord->str_fields[SEQ]);
    qualenc_add_record(fileenc->qualenc,
                       samrecord->str_fields[QUAL]);
    ...
    bh_line_cnt++;
    ...
}
...
```

Listing C.3: *The filecodec invokes the SAM parser and the sub-block codecs.*

D. Genomic Information Database

Genomic Information Database

Table taken from ISO/IEC JTC 1/SC 29/WG 11 N15092 (Geneva, CH - February 2015)

ID	Sequencing method	Size	File type	Coverage	Origin	Comments	Path (prepend /data/genome/ within the TNT cluster)
Homo Sapiens							
ERP001775	Illumina HiSeq	~2 TB	FastQ	200x	http://www.ebi.ac.uk/ena/data/view/ERP001775	This is the largest dataset. Can be transcoded from gzip to 7zip if needed.	human/illumina/ERP001775
ERP001960	Illumina HiSeq	~120 GB each genome	BAM	30x	http://www.ebi.ac.uk/ena/data/view/ERP001960	3 genomes selected: SAMEA1573614 SAMEA1573618 SAMEA1573617	human/illumina/ERP001960/tmp/NA12878_S1.sam human/illumina/ERP001960/tmp/NA12879_S1.sam human/illumina/ERP001960/tmp/NA12890_S1.sam
ERP002490	Illumina HiSeq	265 GB	BAM	30-40x	http://www.ebi.ac.uk/ena/data/view/ERP002490	The insert size (distance between the pair of reads) is much larger than usual (about 2K bases instead of 300).	human/illumina/ERP002490/tmp/NA12877_S1.sam human/illumina/ERP002490/tmp/NA12878_S1.sam human/illumina/ERP002490/tmp/NA12882_S1.sam
Low coverage ERR317482 WGS	Illumina HiSeq 2000	6.1 GB	BAM	1.9x	http://www.ebi.ac.uk/ena/data/view/ERR317482 ftp://ftp.1000genomes.ebi.ac.uk/vol1/ftp/technical/reference/phase2_reference_assembly_sequence/hs37d5.fa.gz	Also used as a low coverage test in the Scramble paper.	human/illumina/ERR317482WGS/tmp/9827_2#49.sam
Low coverage NA21144.chrom11	Illumina HiSeq 2000	1 GB	BAM	7.5x	ftp://ftp.1000genomes.ebi.ac.uk/vol1/ftp/data/NA21144/alignment/NA21144.chrom11.ILLUMINA.bwa.GIH.low_coverage.20130415.bam ftp://ftp.1000genomes.ebi.ac.uk/vol1/ftp/technical/reference/phase2_reference_assembly_sequence/hs37d5.fa.gz	Used in Scramble paper. Processed through the GATK pipeline, which makes the auxiliary data bulkier. (Can be stripped off easily if desired.)	human/illumina/NA21144.chrom11/tmp/NA21144.chrom11.ILLUMINA.bwa.GIH.low_coverage.20130415.sam
PacBio_CHM1htert_54x	Pacific Biosciences SMRT	150 GB	FastQ	54x	http://datasets.pacb.com/2014/Human54x/fast.html	Available as zipped FastQ.	human/pacbio
IonTorrent	Ion Torrent	1.3 GB each file	BAM		http://www.ebi.ac.uk/ena/data/view/ERX276880 http://www.ebi.ac.uk/ena/data/view/ERX276881 http://www.ebi.ac.uk/ena/data/view/ERX276882	http://www.ebi.ac.uk/ena/data/warehouse/search?query=%22instrument_platform=%22ION_TORRENT%22%20%20AND%20submitted_format=%22BAM%22%22&domain=read	human/IonTorrent/tmp/sample-2-10_sorted.sam human/IonTorrent/tmp/sample-2-11_sorted.sam human/IonTorrent/tmp/sample-2-12_sorted.sam
RNAseq		16 GB	BAM		http://www.ebi.ac.uk/arrayexpress/files/E-MTAB-1728/K562_cytosol_LID8465_TopHat_v2.bam		human/RNASeq/tmp/K562_cytosol_LID8465_TopHat_v2.sam

Bacteria							
ERX593919 (E. Coli)	Oxford Nanopore	60 GB	Gzipped Fast5		http://www.ebi.ac.uk/ena/data/view/ERX593919	To be converted from Fast5.	--
ERX593921 (E. Coli)	Oxford Nanopore	46 GB	Gzipped Fast5		http://www.ebi.ac.uk/ena/data/view/ERX593921	To be converted from Fast5.	bacteria/oxfordnano
DH10B (E.Coli)	Illumina	1.3 GB	BAM		ftp://webdata:webdata@ussd-ftp.illumina.com/Data/SequencingRuns/DH10B/MiSeq_Ecoli_DH10B_110721_PF.bam https://raw.githubusercontent.com/allanroscoche/PathTree/master/data/DH10B_WithDup_FinalEdit_validated.fasta	Used in the Deez paper.	bacteria/DH10B/tmp/MiSeq_Ecoli_DH10B_110721_PF.sam
ERA2690369799_7#3.bam (E.Coli)	Illumina	2.3 GB	BAM		ftp://ftp.sra.ebi.ac.uk/vol1/ERA269/ERA269036/bam/ http://www.ncbi.nlm.nih.gov/nuccore/NC_000913.2?report=fasta&format=text	Used in the Scramble paper.	bacteria/ERA269036/tmp/9799_7#3.sam
Metagenomic							
Human gut	Illumina Genome Analyzer II	10 GB	FastQ		http://www.ebi.ac.uk/ena/data/view/ERA000116	3 samples picked: SAMEA728920 SAMEA728635 SAMEA728854	metagenomic/humangut
Cancer cell lines							
Mutation/Variation Calling Benchmark 4 at CGHub		255 GB	BAM	60x	https://cghub.ucsc.edu/datasets/benchmark_download.html	TCGA BENCHMARK CELL LINE: HCC1143 NORMAL 60x f0eaa94b-f622-49b9-8eac-e4eac6762598	cancercells/f0eaa94b-f622-49b9-8eac-e4eac6762598/tmp/G15511.HCC1143_BL.1.sam
Mutation/Variation Calling Benchmark 4 at CGHub		305 GB	BAM	50x	https://cghub.ucsc.edu/datasets/benchmark_download.html	TCGA BENCHMARK CELL LINE: HCC1143 TUMOR 50x ad3d4757-f358-40a3-9d92-742463a95e88	cancercells/ad3d4757-f358-40a3-9d92-742463a95e88/tmp/G15511.HCC1143.1.sam
Mutation/Variation Calling Benchmark 4 at CGHub		130 GB	BAM		https://cghub.ucsc.edu/datasets/benchmark_download.html	UCSC ARTIFICIAL MIXED SAMPLE: 80% HCC1954BL 20% HCC1954 360b4736-6c5e-48df-af58-c1cf51609350	cancercells/360b4736-6c5e-48df-af58-c1cf51609350/tmp/HCC1954.mix1.n80t20.sam
Plants							
T. Cacao	Illumina	8.2 GB	FastQ	10x	http://www.ncbi.nlm.nih.gov/sra/SRX288435		--

E. Tsc Benchmark Results

Local path	/data/genome/human/illumina/ERR317482WGS/tmp/9827_2#49.sam
File format	SAM
Organism	Homo sapiens
Misc	The first 100,000 alignments have been used

Summary

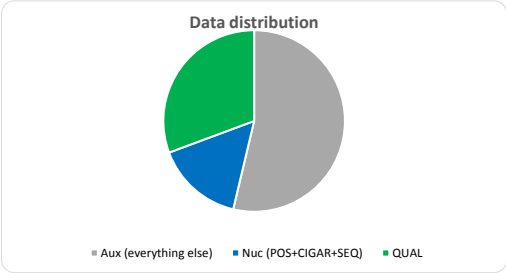
	File size (B)	File size (GiB)	Ratio	Factor	Time (us)	Time (h)	Parameters	Misc	Platform specification
Original File	360427371	0,34	100,00%	1,00					
Tsc	161678231	0,15	44,86%	2,23	741570270	0,21	FILECODEC_BLK_LC 10000 NUCCODEC_WINDOW_SZ 10		Intel® Xeon® CPU E5405 @ 2.00 GHz, 39GiB RAM, openSUSE 12.3 (Dartmouth) (x86_64)

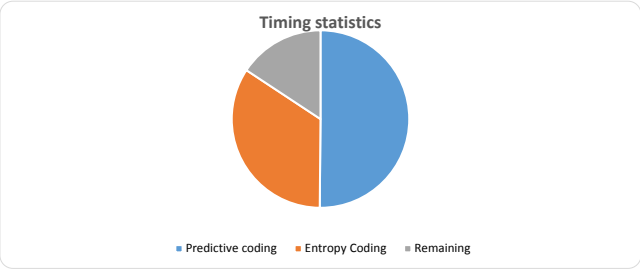
Data distribution

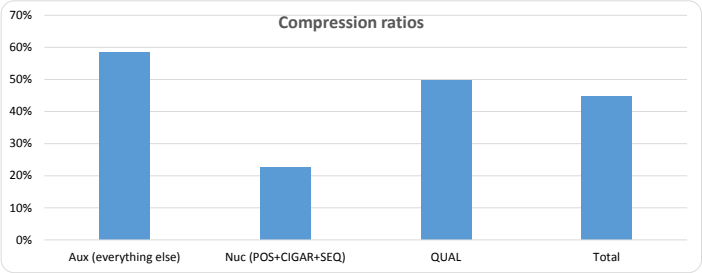
	SAM size (B)	SAM size (GiB)	Ratio	Tsc size (B)	Tsc size (GiB)	Ratio	Compression ratio	Compression factor	Method
File format	0	0,00	0,00%	5628	0,00	0,00%			
SAM header	29279	0,00	0,01%	29279	0,00	0,02%	100,00%	1,00	
Aux (everything else)	148387525	0,14	41,17%	86824841	0,08	53,70%	58,51%	1,71	Predictive coding: nothing, entropy coding: 0-order arithmetic coder
Nuc (POS+CIGAR+SEQ)	112021467	0,10	31,08%	25260964	0,02	15,62%	22,55%	4,43	Predictive coding: local sequence alignment with short-time memory reference, entropy coding: 0-order arithmetic coder
QUAL	99989100	0,09	27,74%	49557519	0,05	30,65%	49,56%	2,02	Predictive coding: nothing, entropy coding: 0-order arithmetic coder
Total	360427371	0,34	100,00%	161678231	0,15	100,00%	44,86%	2,23	

Tsc timing statistics

	Time (us)	Time (h)	Ratio
Total	741570270	0,21	100,00%
Predictive coding	371774640	0,10	50,13%
Entropy Coding	253233893	0,07	34,15%
Remaining	116561737	0,03	15,72%
Misc	0	0,00	0,00%







Local path	/data/genome/bacteria/DH10B/tmp/MiSeq_Ecoli_DH10B_110721_PF.sam
File format	SAM
Organism	Bacteria
Misc	The first 100,000 alignments have been used

Summary

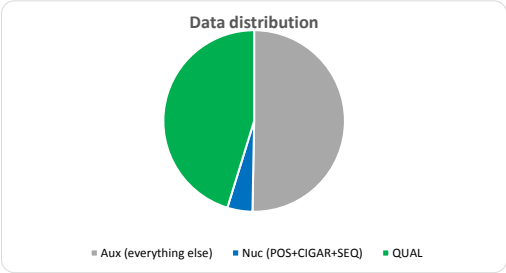
	File size (B)	File size (GiB)	Ratio	Factor	Time (us)	Time (h)	Parameters	Misc	Platform specification
Original File	409375634	0,38	100,00%	1,00					
Tsc	122698208	0,11	29,97%	3,34	1056081405	0,29	FILECODEC_BLK_LC 20000 NUCCODEC_WINDOW_SZ 10		Intel® Xeon® CPU E5405 @ 2.00 GHz, 39GiB RAM, openSUSE 12.3 (Dartmouth) (x86_64)

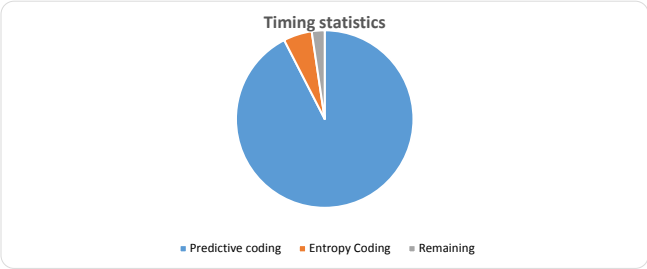
Data distribution

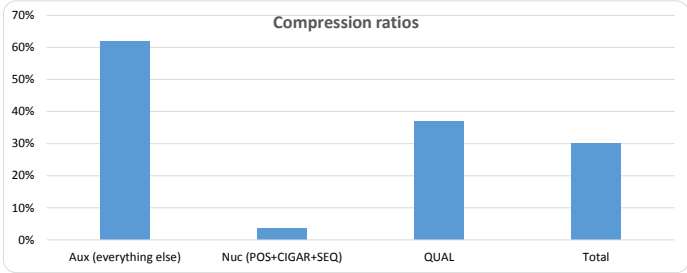
	SAM size (B)	SAM size (GiB)	Ratio	Tsc size (B)	Tsc size (GiB)	Ratio	Compression ratio	Compression factor	Method
File format	0	0,00	0,00%	5628	0,00	0,00%			
SAM header	197	0,00	0,00%	197	0,00	0,00%	100,00%	1,00	
Aux (everything else)	99699744	0,09	24,35%	61711510	0,06	50,30%	61,90%	1,62	Predictive coding: nothing, entropy coding: 0-order arithmetic coder
Nuc (POS+CIGAR+SEQ)	159676293	0,15	39,00%	5485868	0,01	4,47%	3,44%	29,11	Predictive coding: local sequence alignment with short-time memory reference, entropy coding: 0-order arithmetic coder
QUAL	149999400	0,14	36,64%	55495005	0,05	45,23%	37,00%	2,70	Predictive coding: nothing, entropy coding: 0-order arithmetic coder
Total	409375634	0,38	100,00%	122698208	0,11	100,00%	29,97%	3,34	

Tsc timing statistics

	Time (us)	Time (h)	Ratio
Total	1056081405	0,29	100,00%
Predictive coding	976340483	0,27	92,45%
Entropy Coding	54922435	0,02	5,20%
Remaining	24818487	0,01	2,35%
Misc	0	0,00	0,00%







Local path	/data/genome/human/illumina/ERP001960/tmp/NA12878_S1.sam
File format	SAM
Organism	Homo sapiens
Misc	The first 100,000 alignments have been used

Summary

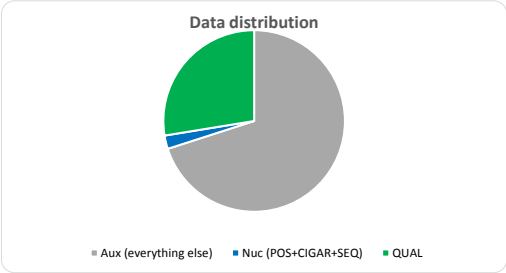
	File size (B)	File size (GiB)	Ratio	Factor	Time (us)	Time (h)	Parameters	Misc	Platform specification
Original File	360371641	0,34	100,00%	1,00					
Tsc	123395952	0,11	34,24%	2,92	66820930381	18,56	FILECODEC_BLK_LC 20000 NUCCODEC_WINDOW_SZ 10		Intel® Xeon® CPU E5620 @ 2.40 GHz, 94GiB RAM, openSUSE 13.1 (Bottle) (x86_64)

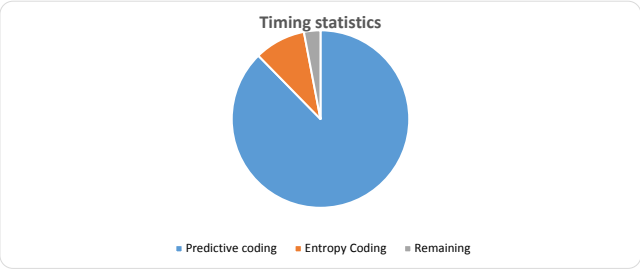
Data distribution

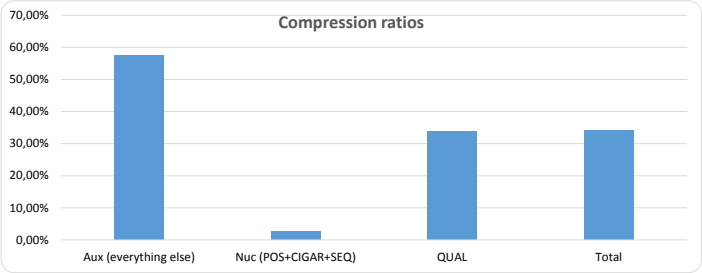
	SAM size (B)	SAM size (GiB)	Ratio	Tsc size (B)	Tsc size (GiB)	Ratio	Compression ratio	Compression factor	Method
File format	0	0,00	0,00%	5628	0,00	0,00%			
SAM header	696	0,00	0,00%	696	0,00	0,00%	100,00%	1,00	
Aux (everything else)	150486207	0,14	41,76%	86417617	0,08	70,03%	57,43%	1,74	Predictive coding: nothing, entropy coding: 0-order arithmetic coder
Nuc (POS+CIGAR+SEQ)	108887465	0,10	30,22%	2948748	0,00	2,39%	2,71%	36,93	Predictive coding: local sequence alignment with short-time memory reference, entropy coding: 0-order arithmetic coder
QUAL	100997273	0,09	28,03%	34023263	0,03	27,57%	33,69%	2,97	Predictive coding: nothing, entropy coding: 0-order arithmetic coder
Total	360371641	0,34	100,00%	123395952	0,11	100,00%	34,24%	2,92	

Tsc timing statistics

	Time (us)	Time (h)	Ratio
Total	66820930381	18,56	100,00%
Predictive coding	58572428778	16,27	87,66%
Entropy Coding	6243013640	1,73	9,34%
Remaining	2005487963	0,56	3,00%
Misc	0	0,00	0,00%







Local path	/data/genome/human/RNAseq/tmp/K562_cytosol_LID8465_TopHat_v2.sam.short.sam
File format	SAM
Organism	Homo sapiens
Misc	The first 100,000 alignments have been used

Summary

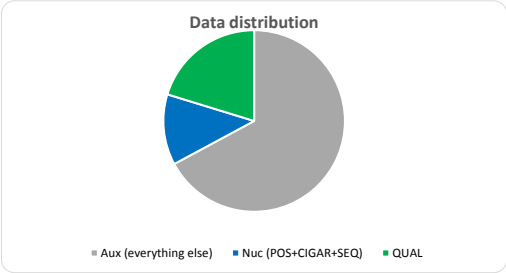
	File size (B)	File size (GiB)	Ratio	Factor	Time (us)	Time (h)	Parameters	Misc	Platform specification
Original File	31167813	0,03	100,00%	1,00					
Tsc	13593736	0,01	43,61%	2,29	30943206	0,01	FILECODEC_BLK_LC 20000 NUCCODEC_WINDOW_SZ 10		Intel® Xeon® CPU E5-2690 v2 @ 3.00 GHz, 126GiB RAM, openSUSE 13.1 (Bottle) (x86_64)

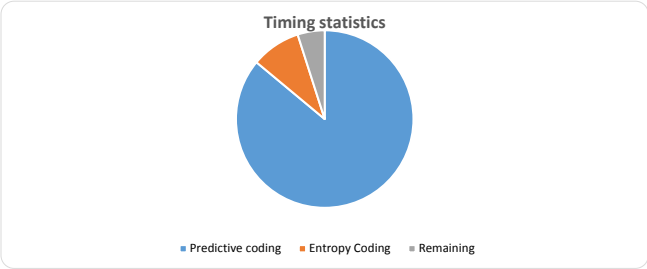
Data distribution

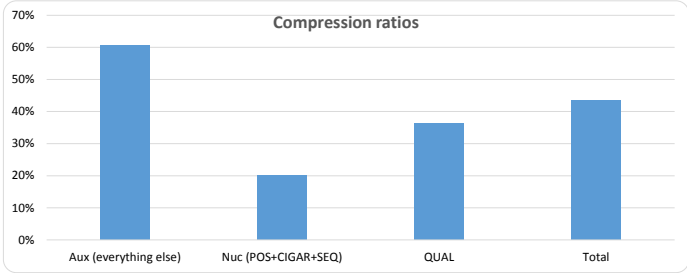
	SAM size (B)	SAM size (GiB)	Ratio	Tsc size (B)	Tsc size (GiB)	Ratio	Compression ratio	Compression factor	Method
File format	0	0,00	0,00%	588	0,00	0,00%			
SAM header	3576	0,00	0,01%	3576	0,00	0,03%	100,00%	1,00	
Aux (everything else)	15057730	0,01	48,31%	9124121	0,01	67,12%	60,59%	1,65	Predictive coding: nothing, entropy coding: 0-order arithmetic coder
Nuc (POS+CIGAR+SEQ)	8513043	0,01	27,31%	1712317	0,00	12,60%	20,11%	4,97	Predictive coding: local sequence alignment with short-time memory reference, entropy coding: 0-order arithmetic coder
QUAL	7593464	0,01	24,36%	2753134	0,00	20,25%	36,26%	2,76	Predictive coding: nothing, entropy coding: 0-order arithmetic coder
Total	31167813	0,03	100,00%	13593736	0,01	100,00%	43,61%	2,29	

Timing statistics

	Time (us)	Time (h)	Ratio
Total	30943206	0,01	100,00%
Predictive coding	26625257	0,01	86,05%
Entropy Coding	2793129	0,00	9,03%
Remaining	1524820	0,00	4,93%
Misc	0	0,00	0,00%







Bibliography

[Gol66] S. W. Golomb: Run-length encodings.
IEEE Transactions on Information Theory, (1966).

[WC53] Watson, J. D. and Crick, F. H. C.: Molecular structure of nucleic acids: A structure for deoxyribose nucleic acid.
Nature 171, pages 737–738, (1953).
doi:10.1038/171737a0.

[COM+12] B. G. Chern, I. Ochoa, A. Manolakos, A. No, K. Venkat, and T. Weissman: Reference Based Genome Compression.
arXiv:1204.1912v1 [cs.IT] 9 Apr 2012, (2012).
Available: `http://arxiv.org/pdf/1204.1912.pdf`. Accessed: 2015-08-01.

[Llo82] Stuart P. Lloyd: Least Squares Quantization in PCM.
IEEE Transactions on Information Theory, Vol. IT-28, 1982, pp. 129–137, March, (1982).

[Max60] Joel Max: Quantizing for Minimum Distortion.
IRE Transactions on Information Theory, Vol. IT-6, pp. 7–12, March, (1960).

[Sha48] Claude E. Shannon: A Mathematical Theory of Communication.
The Bell System Technical Journal, Vol. 27, pp. 379-423, 623-656, July, October, (1948).

[MAT+14] Marco Mattavelli, Claudio Alberti, Yann Thoma, Heinz Stockinger, Jaime Delgado, Ioannis Xenarios: Issues in genome compression and storage.
ISO/IEC JTC1/SC29/WG11 Requirements Subgroup, (2014).
MPEG document ID: N14553. Online. Accessed: 2015-08-28.

[Chi15] Dr. Leonardo Chiariglione: MPEG — The Moving Picture Experts Group website.
ISO/IEC JTC1/SC29/WG11 (MPEG), (2015).
URL: `http://mpeg.chiariglione.org`. Accessed: 2015-07-22.

[WBL13] Sebastian Wandelt, Marc Bux, and Ulf Leser: Trends in Genome Com-

pression.
Knowledge Management in Bioinformatics, Institute for Computer Science, Humboldt-Universität zu Berlin, Germany, (2013).
This paper gives an excellent introduction and overview to the topic of genome compression. However, is is slightly outdated.

[TLS10] Waibhav Tembe, James Lowey, and Edward Suh: G-SQZ: compact encoding of genomic sequence and quality data.
Bioinformatics, Vol. 26, No. 17, 2010, pages 2192–2194, (2010).
doi:10.1093/bioinformatics/btq346.

[DG11] Sebastian Deorowicz, and Szymon Grabowski: Compression of DNA sequence reads in FASTQ format.
Bioinformatics, Vol. 27, No. 6, pages 860–862, (2011).
doi:10.1093/bioinformatics/btr014.

[ZL77] Jacob Ziv, Fellow, IEEE, and Abraham Lempel, Member, IEEE: A Universal Algorithm for Sequential Data Compression.
IEEE Transactions on Information Theory, Vol. IT-23, NO. 3, pp. 337-343, May, (1977).
The classical paper introducing dictionary-based source coding.

[GA15] Jean-loup Gailly and Mark Adler: The gzip home page.
Jean-loup Gailly and Mark Adler, (2015).
URL: `http://www.gzip.org`. Accessed: 2015-07-27.

[JRPK12] Daniel C. Jones, Walter L. Ruzzo, Xinxia Peng, and Michael G. Katze: Compression of next-generation sequencing reads aided by highly efficient de novo assembly.
Nucleic Acids Research, pp. 1–9, (2012).
doi:10.1093/nar/gks754. Quip home page: `http://homes.cs.washington.edu/~dcjones/quip/` (accessed: 2015-07-27). Supplementary data and source code: `https://github.com/dcjones/quip` (accessed: 2015-07-27).

[BM13] James K. Bonfield, Matthew V. Mahoney: Compression of FASTQ and SAM Format Sequencing Data.
PLoS One (www.plosone.org), Volume 8, Issue 3, e59190, pages 1-10, March, (2013).
Published online 2013 Mar 22. doi: 10.1371/journal.pone.0059190.

[HNS14] Faraz Hach, Ibrahim Numanagić, S. Cenk Sahinalp: DeeZ: reference-based compression by local assembly.
Nature Methods, Vol. 11, No. 11, pages 1082-1084, November, (2014).
Supplementary note: `http://www.nature.com/nmeth/journal/v11/n11/extref/nmeth.3133-S1.pdf` (accessed: 2015-07-27). doi:10.1038/nmeth.3133.

[SAM14] The SAM/BAM Format Specification Working Group: Sequence Alignment/Map Format Specification.
The SAM/BAM Format Specification Working Group, (2014).
Available: `https://github.com/samtools/hts-specs`. *For this report printing version 90b9d9e has been used.*

[VMO15] Jan Voges, Marco Munderloh, and Jörn Ostermann: Predictive Coding of Genomic Information Data.
In the United States Patent and Trademark Office: Application for Provisional Letters Patent, (2015).
EFS ID: 22642136. Application Number: 62180146.

[KSK+11] Christos Kozanitis, Chris Saunders, Semyon Kruglyak, Vineet Bafna, and George Varghese: Compressing Genomic Sequence Fragments Using SlimGene.
Journal of Computational Biology, Volume 18, Number 3, 2011, pages 401-413, (2011).
doi: 10.1089/cmb.2010.0253.

[RP71] Robert F. Rice and James R. Plaunt: Adaptive Variable-Length Coding for Efficient Compression of Spacecraft Television Data.
IEEE Transactions on Communication Technology, Vol. COM-19, No. 6, pages 889-897, December, (1971).

[VM15] Jan Voges, Marco Munderloh: Approaches to SAM File Compression.
ISO/IEC JTC1/SC29/WG11 (MPEG), (2015).
MPEG input document m36282 to the 114th MPEG meeting in Warsaw.

[C9907] ISO/IEC JTC1/SC22/WG14 (C): ISO/IEC 9899:TC3.
ISO/IEC JTC1/SC22/WG14 (C), (2007).
The latest publicly available version of the C99 standard is the combined C99 + TC1 + TC2 + TC3, WG14 N1256, dated 2007-09-07. This is a WG14 working paper, but it reflects the consolidated standard at the time of issue. Available: `http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1256.pdf`. *Accessed: 2015-07-28.*

[Deu96] L. Peter Deutsch: DEFLATE Compressed Data Format Specification version 1.3.
Network Working Group, (1996).
Available: `https://tools.ietf.org/pdf/rfc1951.pdf`. *Accessed: 2015-07-28.*

[Huf52] David A. Huffman: A Method for the Construction of Minimum-Redundancy Codes.
Proceedings of the I.R.E., pages 1098-1101, September, (1952).

[LLL+12] Lin Liu, Yinhu Li, Siliang Li, Ni Hu, Yimin He, Ray Pong, Danni Lin, Lihua Lu, and Maggie Law: Comparison of Next-Generation Sequencing Systems.
Journal of Biomedicine and Biotechnology, Volume 2012, Article ID 251364, 11 pages, (2012).
doi:10.1155/2012/251364.

[QSC+12] Michael A Quail, Miriam Smith, Paul Coupland, Thomas D Otto, Simon R Harris, Thomas R Connor, Anna Bertoni, Harold P Swerdlow and Yong Gu: A tale of three next generation sequencing platforms: comparison of Ion Torrent, Pacific Biosciences and Illumina MiSeq sequencers.
BMC Genomics 2012, 13:341, (2012).
doi:10.1186/1471-2164-13-341.

[SLF+15] Zachary D. Stephens, Skylar Y. Lee, Faraz Faghri, Roy H. Campbell, Chengxiang Zhai, Miles J. Efron , Ravishankar Iyer, Michael C. Schatz, Saurabh Sinha, Gene E. Robinson: Big Data:Astronomical or Genomical?.
PLoS Biol 13(7):e1002195, (2015).
doi:10.1371/journal.pbio.1002195.

[CLR04] Lei Chen, Shiyong Lu, and Jeffrey Ram: Compressed Pattern Matching in DNA Sequences.
Proceedings of the 2004 IEEE Computational Systems Bioinformatics Conference (CSB 2004), (2004).

[CPT11] Phillip E. C. Compeau, Pavel A. Pevzner, and Glenn Tesler: How to apply de Bruijn graphs to genome assembly.
Nature Biotechnology, Volume 29, Number 11, pages 987-991, November, (2011).

[MO11] Prof. Dr.-Ing. H.-G. Musmann and Prof. Dr.-Ing. Jörn Ostermann: Statistische Methoden der Nachrichtentechnik - Skript zur Vorlesung.
Institut für Informationsverarbeitung, Leibniz Universität Hannover, (2011).
The statistics dossier from the lecture "Statistische Methoden der Nachrichtentechnik" at the "Institut für Informationsverarbeitung".

[MO12] Prof. Dr.-Ing. H.-G. Musmann and Prof. Dr.-Ing. Jörn Ostermann: Quellencodierung - Skript zur Vorlesung.
Institut für Informationsverarbeitung, Leibniz Universität Hannover, (2012).
The source coding dossier from the lecture "Quellencodierung" at the "Institut

für Informationsverarbeitung".

[MO*12] Prof. Dr.-Ing. H.-G. Musmann and Prof. Dr.-Ing. Jörn Ostermann: Informationstheore - Skript zur Vorlesung.
Institut für Informationsverarbeitung, Leibniz Universität Hannover, (2012).
The dossier on information theory from the lecture "Informationstheorie" at the "Institut für Informationsverarbeitung".

[Mar06] A. A. Markov: An Example of Statistical Investigation of the Text Eugene Onegin Concerning the Connection of Samples in Chains.
Science in Context 19(4), pages 591–600, (2006).
Lecture at the physical-mathematical faculty, Royal Academy of Sciences, St. Petersburg, 23 January 1913. doi:10.1017/S0269889706001074.

[GS97] Charles M. Grinstead and J. Laurie Snell: Introduction to Probability.
American Mathematical Society, (1997).
2nd edition.

[Mac80] Charles E. Mackenzie: Coded Character Sets, History and Development.
Addison-Wesley, (1980).
ISBN: 0-201-14460-3.

[AMC+15] Claudio Alberti, Marco Mattavelli (EPFL), Leonardo Chiariglione (CEDEO), Ioannis Xenarios, Nicolas Guex, Heinz Stockinger, Thierry Schuepbach, Pascal Kahlem, Christian Iseli, Daniel Zerzion, Dmitry Kuznetsov (SIB), Yann Thoma, Enrico Petraglio (HEIG-VD), Cenk Sahinalp, Ibrahim Numanagic (Simon Fraser University), James Bonfield (Wellcome Trust Sanger Institute), Vadim Zalunin (EBI), Jaime Delgado (UPC): Database for Evaluation of Genome Compression and Storage.
ISO/IEC JTC1/SC29/WG11 Requirement Subgroup, (2015).
MPEG document ID: w15092. Online. Accessed: 2015-08-28.

[CS14] Scott Chacon and Ben Straub: Pro Git.
Apress; 2nd ed. 2014 edition (November 12), (2014).
ISBN-13: 978-1484200773. Available: `https://git-scm.com/book/de/v1`.

[Smi07] Brett Smith: A Quick Guide to GPLv3.
Free Software Foundation, Inc., (2007).
Available: `http://www.gnu.org/licenses/quick-guide-gplv3.pdf`.

[RY01] K.R. Rao and P.C. Yip: The transform and data compression handbook.
CRC Press LLC, (2001).
ISBN: 0-8493-3692-9.

[GHG+15] Daniel F. Gudbjartsson, Hannes Helgason, Sigurjon A. Gudjonsson et

al.: Large-scale whole-genome sequencing of the Icelandic population.
Nature Genetics 47, 435–444, (2015).
doi:10.1038/ng.3247. Available: `http://www.nature.com/ng/journal/v47/n5/full/ng.3247.html`. *Accessed: 2015-09-02.*

[Lun15] Christopher Lund: Die Allmacht des Forschers.
DER SPIEGEL 36/2015, (2015).

[Zim15] Carl Zimmer: Snapshot of Icelandic DNA Shows New Gene Mutations Tied to Diseas.
The New York Times, March 26, 2015, page A6 (New York edition), (2015).

[Hor07] K. J. Horadam: Hadamard Matrices and Their Applications.
Princeton University Press, (2007).

[Gen15] Genome Research Limited: SAMtools.
Online, (2015).
The SAMtools software is available via `http://www.htslib.org/`. *Accessed: 2015-09-04.*

[LHW+09] Heng Li, Bob Handsaker, Alec Wysoker, Tim Fennell, Jue Ruan, Nils Homer, Gabor Marth, Goncalo Abecasis, Richard Durbin, and 1000 Genome Project Data Processing Subgroup: The Sequence Alignment/Map format and SAMtools.
Bioinformatics, Vol. 25, No. 16, pp. 2078–2079, (2009).
doi:10.1093/bioinformatics/btp352.

[ENA15] European Nucleotide Archive (ENA): CRAM toolkit.
Online, (2015).
Available: `http://www.ebi.ac.uk/ena/software/cram-toolkit`. *Accessed: 2015-09-04.*

[CCB12] Guy Cochrane, Charles E. Cook, and Ewan Birney: The future of DNA sequence archiving.
GigaScience 1:2, (2012).
doi:10.1186/2047-217X-1-2.

[FLCB11] Markus Hsi-Yang Fritz, Rasko Leinonen, Guy Cochrane, and Ewan Birney: Efficient storage of high throughput DNA sequencing data using reference-based compression.
Genome Research 21, pp. 734-740, (2011).

[Siv08] Nayanah Siva: 1000 Genomes project.

Nature Biotechnology, Vol. 26, Number 3, page 256, (2008).
doi:10.1038/nbt0308-256b

[EDL02] Ron Edgar, Michael Domrachev, and Alex E. Lash: Gene Expression Omnibus: NCBI gene expression and hybridization array data repository.
Nucleic Acids Research, Vol. 30 No. 1, pp. 207-210, (2002).
Available: `http://www.ncbi.nlm.nih.gov/pmc/articles/PMC99122/pdf/gkf066.pdf`. *Accessed: 2015-09-04.*

[LBB12] Po-Ru Loh, Michael Baym, and Bonnie Berger: Compressive genomics.
Nature Biotechnology, Volume 30, Number 7, pp. 627-630, July, (2012).
Available: `http://www.nature.com/nbt/journal/v30/n7/pdf/nbt.2241.pdf`. *Accessed: 2015-09-05.*

[ASA+11] Xavier Argout, Jerome Salse, Jean-Marc Aury, et al.: The genome of Theobroma cacao.
Nature Genetics, Volume 43, Number 2, February, (2011).
Available: `http://www.nature.com/ng/journal/v43/n2/pdf/ng.736.pdf`. *Accessed: 2015-09-05.*

[EPD+07] Douglas F. Easton, Karen A. Pooley, Alison M. Dunning, et al.: Genome-wide association study identifies novel breast cancer susceptibility loci.
Nature, June 28, 447(7148), pp. 1087–1093, (2007).
doi:10.1038/nature05887. Available: `http://www.ncbi.nlm.nih.gov/pmc/articles/PMC2714974/pdf/ukmss-5232.pdf`. *Accessed: 2015-09-05.*

[Kah11] Scott D. Kahn: On the Future of Genomic Data.
Science, Vol. 331, pp. 728-729, (2011).
Available: `http://web.stanford.edu/class/cs262/papers/FutureOfGenomicData.pdf`. *Accessed: 2015-09-06.*

[AMC+14] Claudio Alberti, Marco Mattavelli, Leonardo Chiariglione, Ioannis Xenarios, Nicolas Guex, Heinz Stockinger, Thierry Schuepbach, Pascal Kahlem, Christian Iseli, Daniel Zerzion, Dmitry Kuznetsov, Yann Thoma, Enrico Petraglio: White paper on genomic information compression and storage.
ISO/IEC JTC1/SC29/WG11 Requirements Subgroup, (2014).
Online. Accessed: 2015-09-06.

[Ill15] Illumina, Inc.: An Introduction to Next-Generation Sequencing Technology.
Illumina, Inc., (2015).
Online. Accessed: 2015-09-06.

[SNC77] F. Sanger, S. Nicklen, and A. R. Coulson: DNA sequencing with chain-terminating inhibitors.
Proc. Nati. Acad. Sci. USA, Vol. 74, No. 12, pp. 5463-5467, December, (1977).

[SSHP93] Mark P. Sawicki, Ghassan Samara, Michael Hurwitz, Edward Passaro, Jr.: Human Genome Project.
The American Journal Of Surgery, Volume 165, February, (1993).

[RKP+96] Mostafa Ronaghi, Samer Karamohamed, Bertil Pettersson, Mathias Uhlén, Pål Nyrén: Real-Time DNA Sequencing Using Detection of Pyrophosphate Release.
Analytical Biochemistry, Volume 242, Issue 1, pages 84–89, (1996).
doi:10.1006/abio.1996.0432.

[CFG+10] Peter J. A. Cock, Christopher J. Fields, Naohisa Goto, Michael L. Heuer, and Peter M. Rice: The Sanger FASTQ file format for sequences with quality scores, and the Solexa/Illumina FASTQ variants.
Nucleic Acids Research, Vol. 38, No. 6, pp. 1767–1771, (2010).
doi:10.1093/nar/gkp1137.

[AMC*15] Claudio Alberti, Marco Mattavelli, Leonardo Chiariglione, Ioannis Xenarios, Nicolas Guex, Heinz Stockinger, Thierry Schuepbach, Pascal Kahlem, Christian Iseli, Daniel Zerzion, Dmitry Kuznetsov, Yann Thoma, Enrico Petraglio, Cenk Sahinalp, Ibrahim Numanagic, James Bonfield , Vadim Zalunin, Jaime Delgado, Walter Allasia: Requirements on Genome Compression and Storage (Draft).
ISO/IEC JTC1/SC29/WG11 Requirements Subgroup, (2015).
MPEG document ID: N15093. Online. Accessed: 2015-09-09.

[AMC!15] Claudio Alberti, Marco Mattavelli, Leonardo Chiariglione, Ioannis Xenarios, Nicolas Guex, Heinz Stockinger, Thierry Schuepbach, Pascal Kahlem, Christian Iseli, Daniel Zerzion, Dmitry Kuznetsov, Yann Thoma, Enrico Petraglio, Jaime Delgado: Investigation on genomic information compression and storage.
ISO/IEC JTC1/SC29/WG11 Requirements Subgroup, (2015).
MPEG document ID: N15094. Online. Accessed: 2015-09-09.

[AM15] Claudio Alberti, Marco Mattavelli: Seminar on Prospects on Genome Compression Standardization.
ISO/IEC JTC1/SC29/WG11 Requirements Subgroup, (2015).
MPEG document ID: N15347. Online. Accessed: 2015-09-13.

[Ans73] F. J. Anscombe: Graphs in Statistical Analysis.
The American Statistician, Vol. 27, No. 1, pp. 17-21, February, (1973).

doi:10.2307/2682899.

[MSW03] Detlev Marpe, Heiko Schwarz, and Thomas Wiegand: Context-Based Adaptive Binary Arithmetic Coding in the H.264/AVC Video Compression Standard.
IEEE Transaction on Circuits and Systems for Video Technology, Vol. 13, No. 7, pp. 620-636, July, (2003).

[PM93] William B. Pennebaker and Joan L. Mitchell: JPEG Still Image Data Compression Standard.
Springer US, (1993).
ISBN: 978-0-442-01272-4.

[Bro12] Stuart M. Brown: Sequencing-by-Synthesis: Explaining the Illumina Sequencing Technology.
Blog article in in Next Generation Sequencing, 30th of August, (2012).
Online.

[BS95] James K. Bonfield and Rodger Staden: The application of numerical estimates of base calling accuracy to DNA sequencing projects.
Nucleic Acids Research, Vol. 23, No. 8, pp. 1406-1410, (1995).

[LD09] Heng Li and Richard Durbin: Fast and accurate short read alignment with Burrows–Wheeler transform.
Bioinformatics, Vol. 25, No. 14, pp. 1754–1760, (2009).
doi:10.1093/bioinformatics/btp3240.

[LTPS09] Ben Langmead, Cole Trapnell, Mihai Pop, and Steven L. Salzberg: Ultrafast and memory-efficient alignment of short DNA sequences to the human genome.
Genome Biology 10:R25, (2009).
doi:10.1186/gb-2009-10-3-r25.

[LS12] Ben Langmead and Steven L. Salzberg: Fast gapped-read alignment with Bowtie 2.
Nature Methods, Vol. 9, No. 4, pp. 357–359, April, (2012).
doi:10.1038/nmeth.1923.

[KMKM02] Kazutaka Katoh, Kazuharu Misawa, Kei-ichi Kuma, and Takashi Miyate: MAFFT: a novel method for rapid multiple sequence alignment based on fast Fourier transform.
Nucleic Acids Research, Vol. 30, No. 14, pp. 3059-3066, (2002).
doi:10.1093/nar/gkf436.

[Edg04] Robert C. Edgar: MUSCLE: multiple sequence alignment with high accuracy and high throughput.

Nucleic Acids Research, Vol. 32, No. 5, pp. 1792-1797, (2004).
doi:10.1093/nar/gkh340.

[BLL+15] Gaëtan Benoit, Claire Lemaitre, Dominique Lavenier, Erwan Drezen, Thibault Dayris, Raluca Uricaru, and Guillaume Rizk: Reference-free compression of high throughput sequencing data with a probabilistic de Bruijn graph.
BMC Bioinformatics, 16:288, (2015).
doi:10.1186/s12859-015-0709-7.

[OAB+13] Idoia Ochoa, Himanshu Asnani, Dinesh Bharadia, Mainak Chowdhury, Tsachy Weissman, and Golan Yona: QualComp: a new lossy compressor for quality scores based on rate distortion theory.
BMC Bioinformatics, 14:187, (2013).
doi:10.1186/1471-2105-14-187.

[Lev66] Vladimir I. Levenshtein: Binary codes capable of correcting deletions, insertions, and reversals.
Soviet Physics Doklady 10 (8), pages 707–710, (1966).

Glossary

alignment The process of aligning sequence reads by finding regions of similarity. If the alignment is done with respect to a (pre-constructed) reference sequence, the process is called mapping. Aligned reads are called "alignment", too. XXIX, XXX, 29, 30, 34, 41, 42, 46, 48, 51, 77, 78
arithmetic coding Form of entropy coding. Encodes an entire message as a single number. v, XXX, 17
assembly The process of merging multiple alignments to reconstruct the (complete) donor DNA sequence. If performed without the use of an external reference, this process is called de-novo assembly. XXX, XXXII, 1, 29, 30, 41, 42, 51, 77, 78

BAM Compressed version of SAM. Conversion between the two formats is done with SAMtools. The most important conversion commands are explained in Appendix B. V, VI, 3, 31, 34, 37, 47, 70, 71, 77
base pair A pair of nucleobases contributing to the structure of DNA. Abbreviated as bp. v, XXXII, 23, 24, 26, 27
Big Data Broad term for huge and complex data sets that require the development of novel data processing schemes. 39, 78
bit The word *bit* is a portmanteau of "binary digit" and represents the basic unit of information in information theory. It can have two values (0 respectively 1). iii, XXIX, XXXII, 3, 13, 47, 52
byte One *byte* consists of 8 bits and can therefore represent the (decimal) values 0 through 255. iii, 47

cell nucleus Organelle in eukaryotic cells containing the chromosomes. XXXI, 23
chromosome Structured macromolecule containing most of the genetic information in the form of DNA. VI, XXIX, 23, 24
codec Device able to (en-)code respectively decode digital data signals. The word *codec* is a portmanteau of the words "coder" and "decoder". 42
codon Triplet of nucleobases coding the amino-acids for protein synthesis in the ribosomes. 23
compression See source coding. XXX, 39, 41–43, 47, 56, 70
correlation Correlations between different data sets can be evaluated with correlation coefficients (e.g. PCC). Correlations can indicate an exploitable predictive relationship. 47, 57
coverage The coverage refers to the (average) number of times a nucleotide has been sequenced. The coverage c can be computed as $c = {}^{N \cdot \bar{l}}/_{l_G}$ where N denotes the number of sequence reads, $\bar{l}$ denotes the average read length, and l_G denotes the length of the original genome. XXXII, 2, 30, 33, 48, 51, 53, 56

de-novo assembly Sequence assembly *without* using the information from an external reference. XXIX, 1, 29, 41
decompression See compression. 41, 42

eigenvector The vector $\underline{v}$ is an eigenvector of the square matrix $\underline{A}$, if $\underline{Av} = \lambda\underline{v}$, with λ being the corresponding eigenvalue. 22
electropherogram Plot produced by automated DNA sequencing machines. 28
entropy The SHANNON entropy (entropy in terms of information theory) is the average information contained in a message. 13, 52, 53, 57–59, 66, 79
entropy coding Lossless data compression scheme, trying to eliminate the redundancy present in a sequence of symbols emitted by a source. See arithmetic coding. XXIX
Eukaryote Organism whose cells contain a nucleus and other organelles. XXX
eukaryotic See Eukaryote. XXIX, 23, 24

FASTA File format to store raw sequencing data. Contains sequence identifiers and strings representing the actual nucleotide sequences. XXX–XXXII, 30–34, 60
FASTQ File format. Extension to the FASTA format. Contains in addition an (optional) description and a string of quality scores, one score for each nucleotide. XXXI, XXXII, 1–3, 30–34, 40–42, 44, 47, 48, 69–71, 77

gene A portion of DNA, coding cellular processes. XXX, XXXI, 22–24
genetics A field of biology. Denotes the study of genes and associated topics such as heredity. 22, 23
genome The entire genetic material of an organism. Consists of DNA or RNA in case of RNA viruses. XXX, XXXII, 1, 2, 23, 25–27, 29–31, 33, 39, 42, 44, 46, 47, 51, 60, 72, 78
genome compression The process of encoding the entire genetic information of an individual organism. 41, 43, 44, 47
genomics The scientific field of applying sequencing methods and bioinformatics to analyse the function and structure of genomes. 1, 2, 4, 23, 39, 78

i.e. Latin: "id est". VII, 3, 11, 12, 19–21, 26, 27, 33, 35, 47, 48, 51, 64, 78
indel A blend (portmanteau) of the words "insert" and "deletion". Designates an additional or missing element in a nucleotide sequence with respect to a reference sequence. 29, 35, 46, 57

mapping Sequence alignment, performed with reference to a (pre-constructed) reference sequence. XXIX, 29, 41, 46
mate pair Two sequence reads, extracted by a sequencing machine from the same (long) strand of DNA. XXXI, 27
molecule A group of two or more atoms. XXXI, 1, 23, 24, 30
Moving Pictures Experts Group Also known as ISO/IEC JTC1/SC29/WG11 (Coding of audio, picture, multimedia and hypermedia information). Abbreviated as MPEG. A working group of ISO/IEC that develops international standards in the field of coding audio, video, and related data. vi, 2, 40, 43

next-generation sequencing Sequencing (NGS) machines developed since the 1990s, based on massively parallel analysis and yielding a high throughput and reduced cost. vi, 1, 24, 25, 77
nucleic acid Macromolecule, made up of nucleotides. Depending on the nucleotides, the polymer is called DNA or RNA. XXXI
nucleobase Within bioinformatics, also simply known as base. Part of a nucleotide. XXIX, XXXI, 23, 27
nucleotide Sub-units of nucleic acids such as DNA and RNA. A nucleotide contains a 5-carbon sugar, a phosphate group, and a nitrogenous base. If the sugar is deoxyribose, the nucleotides form DNA. If the sugar is ribose, the resulting macromolecule is RNA. XXXI, 1, 14, 20, 23, 27, 28, 41, 51, 52, 54, 55, 59, 60, 62
nucleotide sequence The sequence of nucleotides in a nucleic acid. Within DNA, the four possible nucleobases are adenine, cytosine, guanine, and thymine, usually represented by the letters A, C, G, and T. VII, XXXII, 1, 15, 21, 27–29, 31, 46–48, 51, 52, 56, 59, 61, 73–75, 77–79
nucleus Please refer to cell nucleus. XXX, 24

organelle Functional subunit within a cell. XXX, 23

paired ends Two sequence reads, obtained by sequencing both ends of a single (short) DNA strand. The orientation is the opposite of mate pairs. 27
paired read Depending on the sequencing technology used, two reads can form a pair with the information, that both reads have been produced from the same strand of DNA with a known distance (about 500-1,000 bp) between them. 24
phenotype Observable characteristics of an organism. Genotypes in conjunction with environmental influences conduct the phenotype. 23, 27
protein Large molecule consisting of a chain of amino-acids. Performing important functions determined by the genes. XXIX, XXXII, XXXIII, 1, 23, 24
pyrosequencing DNA sequencing method bases on sequencing by synthesis. Relying on detection of pyrophosphate release on nucleotide incorporation in contrast to chain termination with dideoxynucleotides (Sanger sequencing). 26, 29

quality score Numerical value assigned to each base call by sequencing machines. Expresses the base-call accuracy. VII, XXX, XXXII, 1, 2, 28–30, 32–34, 36, 41, 46–48, 51, 57–60, 64–66, 69, 73, 74, 77–79

random access In terms of data compression, random access designates the ability to directly access (and decompress) a compressed data element (e.g. a block) from a set of elements, without needing to decompress other data. 2, 4, 37, 42, 46, 59, 69, 71, 78, 79
read See sequence read. I, II, V, XXIX, XXXI, 1, 2, 24, 26, 27, 29–31, 33–35, 42–44, 46, 47, 51, 52, 54–57, 60–62, 70, 71, 74, 77, 79
read header A read header in a FASTA or FASTQ file contains a sequence identifier and an optional description associated with each sequence read. XXXII, 31–33, 46, 47

record Atomic unit in FASTA, FASTQ, and SAM files. Typically, a record contains a sequence read combined with different amounts of meta information. 31, 32, 34, 35, 55, 69

redundancy In terms of information theory, a process or signal can be regarded as a combination of *relevant*, *irrelevant*, and *redundant* information. Within the context of source coding, redundant information can and should be eliminated to reduce bandwidth requirements for transmission. In the context of channel coding (error-prone transmission over noisy channels), redundancy is added to enable error detection and/or error correction. XXX, 2, 3, 15, 26, 27, 31, 51, 53, 54, 59, 77, 78

reference-based assembly Sequence assembly with respects to (parts of) an external reference genome. 1

ribonucleic acid In RNA viruses, the genome is composed of RNA. In eukaryotic cells, as messenger RNA (mRNA), it carries genetic information to the ribosomes for proteine synthesis. vi

ribosome Complex molecular structure serving as the site of protein synthesis. XXIX, XXXII, XXXIII, 23

SAM Sequence Alignment/Map Format, file format used to store aligned sequencing data. Contains the data from the FASTQ format with additional alignment and meta information. I, II, V–VII, XXIX, XXXII, 1–3, 30, 31, 34–37, 40, 41, 44, 47, 48, 51–53, 55, 57, 61–64, 69–74, 77, 79

sequence identifier See read header. 32

sequence read The readout of a continuous part of a segment of DNA. NGS sequencing machines produce (short) reads (with lengths in the order of 100 base pairs) containing the actual nucleotide sequence and optional meta information (such as quality scores). XXIX–XXXII, 1, 2, 29–31, 33, 41, 47, 61–63, 78

sequencing The process of expressing genomic information (more precise: the nucleotide or base sequence in a DNA molecule) usually as a sequence of characters (also known as sequence reads). Additional meta information (e.g. quality scores) might be produced. XXX–XXXII, 1, 2, 4, 7, 22–27, 30, 39–42, 48, 71, 77, 78

sequencing by synthesis The process of taking a single strand of DNA and synthesising the complementary strand enzymatically. XXXI, 25, 26

sequencing depth See coverage. 30, 33, 77, 78

single nucleotide variant The genome of an individual organism may have differences with respect to other genomes from the same species. Differences concerning only a single nucleotide are named SNV. vi

source coding Broadly speaking, source coding or data compression denotes the process of encoding information using less bits. XXIX, 47

strand DNA is made up of two biopolymer strands forming the double helix structure. XXXI, XXXII, 23, 24, 26, 27

string A sequence of characters (in the field of computer science). 19, 28, 41

transcription The process of copying the genetic information from DNA to mRNA. 23

translation Translation of the base sequence carried by mRNA into an amino-acid sequence respectively protein with the help of ribosomes. 23

Index

Bayes' theorem, 8
Crick, 1
Fourier transform, 12
Golomb code, 19
Huffman coding, 16
Kraft's inequality, 15
Markov source, 20
Pearson product-moment correlation coefficient, 10
Rice code, 19
Sanger, 1
Sanger sequencing, 25
Watson, 1

AC, 17–19, 47, 62, 73, 74
ad-hoc group, 2, 43
adenine, 23
AhG, V, 2, 29, 43, 44, 46, 70, 71, 78
alignment, 29, 30, 34, 41, 42, 46, 48, 51, 77, 78
alphabet, 13
amino-acid, 23
Arithmetic coding, 17
arithmetic coding, 16, 17
arithmetic mean, 65
ASCII, II, 28, 31, 33, 34, 52
assembly, 1, 29, 30, 41, 42, 51, 77, 78
auto-correlation, 12
auto-correlation series, 12

BAM, V, VI, 3, 31, 34, 37, 47, 70, 71, 77
base calling *error* probability, 33
base pair, 23, 24, 26, 27
base-call accuracy, 41
BGZF, 37, 41
Big Data, 39, 78
binary tree, 16
bit, iii, 3, 13, 47, 52
bp, 24
BWA, 29
BWT, 53
byte, iii, 47

CABAC, 18
Call for Evidence, 43
Call for Proposals, 43
cdf, 9
cell nucleus, 23
central moment, 10
CF, iii
CfE, 43
CfP, 43
chromosome, VI, 23, 24
codec, 42
codon, 23
compression, 39, 41–43, 47, 56, 70
conditional probability, 8, 20
context-adaptive binary arithmetic coding, 18
continuous random process, 11
continuous random variable, 10
correlation, 10, 11, 47, 57
covariance, 10
covariance matrix, 22
coverage, 2, 30, 33, 48, 51, 53, 56
CR, iii, 74
cross correlation, 11
cross-correlation, 11, 12
cumulative distribution function, 9
cytosine, 23

DCT, 21, 22, 56, 57
DDBJ, 72
de-novo assembly, 1, 29, 41
decision content, 14, 15

decompression, 41, 42
DeeZ, 60
DFT, 22
dictionary, 19
dictionary-based, 19
diploid, 72
discrete Fourier transform, 22
discrete cosine transform, 21
discrete random process, 11, 12
discrete random variable, 9
DNA, XXIX, 1, 7, 22–25, 27, 29, 30, 34, 41, 42, 46, 77
DVD, iv, I, VII

EBI, 31, 73
eigenvector, 22
electropherogram, 28
elementary event, 7
ensemble, 7, 13, 14
entropy, 13, 20, 52, 53, 57–59, 66, 79
entropy coding, 16, 19
ergodic, 11
Euclidean distance, 65
eukaryotic, 23, 24
EZN, iv

Fast5, 70
FASTA, 30–34, 60
FASTQ, 1–3, 30–34, 40–42, 44, 47, 48, 69–71, 77
fqzcomp, 64
Frederick Sanger, 25

gene, 22–24
genetics, 22, 23
genome, XXXII, 1, 2, 23, 25–27, 29–31, 33, 39, 42, 44, 46, 47, 51, 60, 72, 78
genome compression, 41, 43, 44, 47
genomic information, 7
genomic information compression, 46
genomic information database, 43
genomic life cycle, 41, 42
genomics, 1, 2, 4, 23, 39, 78
GEO, 73
geometric distribution, 18, 19
GIDB, 5, 43, 52, 55–58, 60, 71, 72, 79
guanine, 23

HDD, 40
heredity, 22
HEVC, 18
human genome, 14
human genome project, 1, 25

i.e., VII, 3, 19–21, 26, 27, 33, 35, 47, 48, 51, 64, 78
Illumina, 26, 27, 33, 56, 57
Illumina Inc., 27
indel, 29, 35, 46, 57
information, 13
information content, 13
information theory, 14
INSDC, 72, 73
IonTorrent, 27
irrelevance, 3
ISO, 43
IUPAC, 28, 29, 31, 77

joint central moment, 10
joint ensemble, 7
joint moment, 10

Karhunen-Loeve-Transform, 22
KLT, 22
Kolmogoroff, 7

Life Technologies, 27
linear dependence, 11
linear mean, 10
Lossless compression, 3
LZMA, 54, 55
LZW, 19

mapping, 29, 41, 46
mapping position, 62
mate pair, 27
maximum likelihood predictor, 64
maximum-likelihood predictor, 20, 58, 59
message, 13

MIPS, 40
molecule, 1, 23, 24, 30
moment, 10
Moving Pictures Experts Group, 2, 40, 43
MPEG, iii, iv, V, 2, 5, 18, 29, 32, 34, 36, 40, 43–46, 52, 70–73, 78
mRNA, 23
multiploid, 72

NCBI, 73
network bandwidth, 39
next-generation sequencing, 1, 24, 25, 77
NGS, 1, 24, 25, 27, 28, 31, 39–41, 43, 48, 51, 77
nucleobase, 23, 27
nucleotide, 1, 14, 20, 23, 27, 28, 32, 41, 51, 52, 54, 55, 59, 60, 62
nucleotide sequence, VII, 1, 15, 21, 27–29, 31, 46–48, 51, 52, 56, 59, 61, 73–75, 77–79
nucleus, 24

organelle, 23
orthogonal, 21

PacBio, 27
Pacific Biosciences, 27
paired ends, 27
paired read, 24
PCC, 10
pdf, 10, 11
phenotype, 23, 27
Phred quality score, 33
Phred quality scores, 28
pmf, 9–11, 16
power spectral density, 12
prefix-free code, 15, 20
prefix-free codes, 15
probability, 77
probability density function, 10
probability mass function, 9
processing speed, 39
protein, 1, 23, 24
pyrosequencing, 26, 29

quality score, VII, XXXII, 1, 2, 28–30, 32–34, 36, 41, 46–48, 51, 57–60, 64–66, 69, 73, 74, 77–79
quantization, 3
Quip, 59, 61

RA, 49
random access, 2, 4, 37, 42, 46, 59, 69, 71, 78, 79
rate distortion theory, 79
read, I, II, V, 1, 2, 24, 26, 27, 29–31, 33–35, 42–44, 46, 47, 51, 52, 54–57, 60–62, 70, 71, 74, 77, 79
read header, 31–33, 46, 47
record, 31, 32, 34, 35, 55, 69
redundancy, 2, 3, 15, 26, 27, 31, 51, 53, 54, 59, 77, 78
reference genome, 42
reference-based assembly, 1
requirements, 43
ribosome, 23
RNA, 1, 23, 27, 31, 34, 41

SAM, I, II, V–VII, 1–3, 30, 31, 34–37, 40, 41, 44, 47, 48, 51–53, 55, 57, 61–64, 69–74, 77, 79
SAMtools, V, VI
second central moment, 10
sequence identifier, 32
sequence read, XXIX, 1, 2, 29–31, 33, 41, 47, 61–63, 78
SequenceSqueeze, 47
SequenceSqueeze contest, 43
sequencing, 1, 2, 4, 7, 22–27, 30, 39–42, 48, 71, 77, 78
sequencing by synthesis, 25, 26
sequencing depth, 30, 33, 77, 78
SNV, 43
Solexa, 33
SOLiD, 25, 26
source coding, 3, 47
source coding theorem, 14, 16

SpS, iii
standard deviation, 10
states, 12
stationary, 11
statistical independence, 11, 13
statistically independent, 8, 12, 21, 22
storage size, 1
storage space, 39
strand, 23, 24, 26, 27
string, 19, 28, 41

thymine, 23
transcription, 23
transform coding, 21
transform coefficients, 21, 22
translation, 23
transmission costs, 1

uncertainty, 13
unitary, 21
Unix, 19, 31
uracil, 23, 31

variance, 10, 65

List of Figures

2.1. Probability mass function and cumulative distribution function 9
2.2. Arithmetic coding 18
2.3. Predictive coding 21
2.4. From a cell to DNA 24
2.5. Electropherogram 28
2.6. From sequencing to assembly 30
2.7. File formats 32
2.8. Aligned reads 36
2.9. Reads formatted as SAM file 36

3.1. Sequencing progress versus computation and storage capacities 40
3.2. Genomic life cycle 42

4.1. Comparison of universal coding algorithms 55
4.2. Read positions 56
4.3. Illumina reads 57
4.4. Illumina reads - DCT 57
4.5. Auto-correlation of single quality score lines 59
4.6. Inter-line correlations 60
4.7. Aligned nucleotide sequences 62
4.8. Shifted nucleotide sequences 62
4.9. Nucleotide sequence encoder structure 63
4.10. Extended line context 65
4.11. Dictionary-based substring matching 65
4.12. Quantized quality scores 66

5.1. Tsc program structure 70
5.2. Tsc file structure 71
5.3. Tsc data distribution 75
5.4. Tsc timing statistics 75

List of Tables

1.1. De-facto compression ratios for FASTQ and SAM files 3

2.1. Comparison of NGS methods 27
2.2. SAM fields 35

3.1. General genome compression requirements 45
3.2. Compression requirements for unmapped reads 46
3.3. Additional compression requirements for aligned data 46
3.4. Selection of genomic information compression tools 48
3.5. Comparison of Quip and DeeZ 49

4.1. Nucleotide sequence entropies 52
4.2. Nucleotide sequence compression with gzip 54
4.3. Nucleotide sequence compression with bzip2 54
4.4. Nucleotide sequence compression with lzma 55
4.5. Quality score entropies 58
4.6. Markov chains and quality scores 61

5.1. Tsc Header 70
5.2. SAM header in a tsc file 71
5.3. Tsc block header 72
5.4. Genomic information database criteria 72
5.5. File identifiers 73
5.6. Benchmark results 74

A.1. Samsplit options II
A.2. MATLAB scripts for the analysis of SAM files III

List of Algorithms

1. Nucleotide sequence encoder 67

List of Listings

1. Supplementary material iv

2.1. Snapshot from a SAM file 36

4.1. Working principle of `expand` 62

A.1. Samsplit repository I
A.2. Extracting sequence reads with samsplit II

B.1. Viewing BAM Content V
B.2. Accessing a specific segment of a BAM files VI
B.3. Indexing a BAM file VI
B.4. SAM to BAM compression VI
B.5. Decompressing a BAM file VI

C.1. Tsc repository VII
C.2. Calling the filecodec VII
C.3. Sub-block codecs VIII